Sliper QML C++ — Kaynak Kod Dokumu

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(sliper VERSION 1.0)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_PREFIX_PATH "C:/Qt/6.7.3/mingw_64")

find_package(Qt6 REQUIRED COMPONENTS Core Quick Charts Widgets Sql PrintSupport Network Multimedia)

qt_standard_project_setup()
qt_policy(SET QTP0001 NEW)

qt_add_executable(sliper
    main.cpp
    src/Backend.h
    src/Backend.cpp
    src/SensorManager.h
    src/SensorManager.cpp
    src/Calculator.h
    src/Calculator.cpp
    src/Database.h
    src/Database.cpp
    src/ReportManager.h
    src/ReportManager.cpp
    src/WifiManager.h
    src/WifiManager.cpp
    src/VoiceCommandManager.h
    src/VoiceCommandManager.cpp
    src/VoiceRecognitionWorker.h
    src/VoiceRecognitionWorker.cpp
)

set_source_files_properties(qml/Translations.qml PROPERTIES
    QT_QML_SINGLETON_TYPE TRUE
)

qt_add_qml_module(sliper
    URI sliper
    VERSION 1.0

    QML_FILES
        qml/Main.qml
        qml/Translations.qml
        qml/pages/LoginPage.qml
        qml/pages/DashboardPage.qml
        qml/pages/OlcumPage.qml
        qml/pages/SonuclarPage.qml
        qml/pages/GecmisPage.qml
        qml/pages/KalibrasyonPage.qml
        qml/components/SuTerazisi.qml
        qml/components/TarihSecici.qml

    RESOURCES
        qml/assets/logo.png
        qml/assets/egim_sensor.png
        qml/assets/loadcell.png
        qml/assets/mesafe_sensor.png
        qml/assets/slipergif.mp4
)

target_link_libraries(sliper PRIVATE
    Qt6::Core
    Qt6::Quick
    Qt6::Charts
    Qt6::Widgets
    Qt6::Sql
    Qt6::PrintSupport
    Qt6::Network
    Qt6::Multimedia
)

# --- Sesli komut (Vosk, offline Turkce konusma tanima) ---
set(VOSK_DIR "${CMAKE_SOURCE_DIR}/third_party/vosk")
set(VOSK_MODEL_DIR "${CMAKE_SOURCE_DIR}/resources/vosk-model-small-tr-0.3")

target_include_directories(sliper PRIVATE ${VOSK_DIR})
target_link_libraries(sliper PRIVATE "${VOSK_DIR}/libvosk.lib")
```

```cmake
# libvosk.dll kendi MinGW calisma zamanini (libstdc++/libwinpthread/libgcc_s)
# gerektirir. Vosk paketiyle gelen bu DLL'leri DEGIL, projenin kullandigi
# Qt MinGW derleyicisiyle ayni surumden gelenleri kopyaliyoruz; aksi halde
# surum uyusmazligi uygulamanin ENTRYPOINT_NOT_FOUND ile cokmesine yol acar.
get_filename_component(MINGW_BIN_DIR "${CMAKE_CXX_COMPILER}" DIRECTORY)

add_custom_command(TARGET sliper POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different
        "${VOSK_DIR}/libvosk.dll"
        "${MINGW_BIN_DIR}/libstdc++-6.dll"
        "${MINGW_BIN_DIR}/libwinpthread-1.dll"
        "${MINGW_BIN_DIR}/libgcc_s_seh-1.dll"
        "$<TARGET_FILE_DIR:sliper>"
    COMMENT "Vosk kutuphaneleri kopyalaniyor"
)

add_custom_command(TARGET sliper POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_directory
        "${VOSK_MODEL_DIR}"
        "$<TARGET_FILE_DIR:sliper>/vosk-model-small-tr-0.3"
    COMMENT "Vosk Turkce ses tanima modeli kopyalaniyor"
)
```

main.cpp

```cpp
#include <QQmlContext>
#include <QApplication>
#include <QQmlApplicationEngine>
#include <QDebug>
#include <QtGlobal>
#include "src/Backend.h"
#include "src/SensorManager.h"
#include "src/Calculator.h"
#include "src/Database.h"
#include "src/ReportManager.h"
#include "src/WifiManager.h"
#include "src/VoiceCommandManager.h"

int main(int argc, char *argv[])
{
    qputenv("QT_QUICK_CONTROLS_STYLE", "Basic");

    QApplication app(argc, argv);
    QApplication::setApplicationName("SLIPER");

    Backend backend;
    SensorManager sensorManager;
    Calculator calculator;
    Database database;
    ReportManager reportManager;
    WifiManager wifiManager(&sensorManager, &database);
    VoiceCommandManager voiceCommandManager;

    QQmlApplicationEngine engine;

    engine.rootContext()->setContextProperty("backend", &backend);
    engine.rootContext()->setContextProperty("sensorManager", &sensorManager);
    engine.rootContext()->setContextProperty("calculator", &calculator);
    engine.rootContext()->setContextProperty("database", &database);
    engine.rootContext()->setContextProperty("reportManager", &reportManager);
    engine.rootContext()->setContextProperty("wifiManager", &wifiManager);
    engine.rootContext()->setContextProperty("voiceCommandManager", &voiceCommandManager);

    QObject::connect(
        &engine,
        &QQmlApplicationEngine::objectCreationFailed,
        &app,
        []() {
            qWarning() << "QML nesnesi olusturulamadi!";
            QCoreApplication::exit(-1);
        },
        Qt::QueuedConnection);

    engine.loadFromModule("sliper", "Main");

    return app.exec();
}
```

qml/Main.qml

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtQuick.Window 6.7
import QtQuick.Layouts 6.7
import "pages"

ApplicationWindow {
    id: pencere
    visibility: Window.Maximized
    title: "SLIPER Analiz"

    background: Rectangle {
        color: "#060d17"
    }

    StackLayout {
        id: anaStack
        anchors.fill: parent
        currentIndex: 0

        LoginPage {
            onGirisBasarili: anaStack.currentIndex = 1
        }

        DashboardPage {
            onCikisYapildi: anaStack.currentIndex = 0
        }
    }
}
```

qml/Translations.qml

```
pragma Singleton
import QtQuick

QtObject {
    // true = Türkçe, false = English
    property bool turkish: true

    function toggle() {
        turkish = !turkish
    }
}
```

qml/components/SuTerazisi.qml

```
import QtQuick 6.7
import sliper

Item {
    id: root
    property real egimX: 0
    property real egimY: 0
    property real maxAci: 15

    readonly property var metinler: ({
        duzMetni: { tr: "Düz", en: "Level" },
        ayarlaniyorMetni: { tr: "Ayarlanıyor...", en: "Adjusting..." }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    width: 200
    height: 200

    readonly property real yaricap: (width / 2) - 20
    readonly property bool duz: Math.abs(egimX) < 0.5 && Math.abs(egimY) < 0.5

    readonly property real kabarcikOfsetX: Math.max(-1, Math.min(1, egimY / maxAci)) * yaricap
    readonly property real kabarcikOfsetY: Math.max(-1, Math.min(1, egimX / maxAci)) * yaricap

    Rectangle {
        anchors.fill: parent
        radius: width / 2
        color: "#0f1420"
        border.color: "#1e2a3f"
        border.width: 2
    }

    Rectangle {
        anchors.centerIn: parent
        width: root.yaricap * 1.2
        height: width
        radius: width / 2
        color: "transparent"
        border.color: "#1e2a3f"
        border.width: 1
    }

    Rectangle {
        anchors.centerIn: parent
        width: parent.width - 20
        height: 1
        color: "#1e2a3f"
    }

    Rectangle {
        anchors.centerIn: parent
        width: 1
        height: parent.height - 20
        color: "#1e2a3f"
    }

    Rectangle {
        id: kabarcik
        width: 28
        height: 28
        radius: 14
        color: root.duz ? "#16a34a" : "#dc2626"
        border.color: root.duz ? "#4ade80" : "#f87171"
        border.width: 2

        x: (root.width / 2) - (width / 2) + root.kabarcikOfsetX
        y: (root.height / 2) - (height / 2) + root.kabarcikOfsetY

        Behavior on x { NumberAnimation { duration: 150; easing.type: Easing.OutQuad } }
        Behavior on y { NumberAnimation { duration: 150; easing.type: Easing.OutQuad } }
        Behavior on color { ColorAnimation { duration: 200 } }
        Behavior on border.color { ColorAnimation { duration: 200 } }
    }

```

```
    Text {
        anchors.top: parent.bottom
        anchors.horizontalCenter: parent.horizontalCenter
        anchors.topMargin: 8
        text: root.duz ? root.txt("duzMetni") : root.txt("ayarlaniyorMetni")
        color: root.duz ? "#4ade80" : "#6b7280"
        font.family: "Segoe UI"
        font.pixelSize: 12
        font.bold: root.duz
    }
}
```

qml/components/TarihSecici.qml

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import sliper

Item {
    id: root
    width: 130
    height: 36

    readonly property var metinler: ({
        placeholderMetni: { tr: "gg.aa.yyyy", en: "dd.mm.yyyy" },
        temizleButon: { tr: "Temizle", en: "Clear" }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    // Kontrollü bileşen: seçili tarih dışarıdan (ör. gecmisSayfasi.baslangicTarihi)
    // bağlanır, kullanıcı bir gün seçtiğinde sadece tarihSecildi sinyali yayılır —
    // root.secilenTarih burada asla doğrudan atanmaz ki dışarıdaki binding kopmasın.
    property string secilenTarih: ""
    property string placeholderText: txt("placeholderMetni")
    property date gorunenAy: new Date()

    readonly property int hucreBoyutu: 28
    readonly property int hucreAraligi: 4
    readonly property int gridGenisligi: 7 * hucreBoyutu + 6 * hucreAraligi

    signal tarihSecildi(string tarihStr)

    function ikiHane(n) {
        return n < 10 ? "0" + n : "" + n
    }

    function tarihStr(d) {
        return ikiHane(d.getDate()) + "." + ikiHane(d.getMonth() + 1) + "." + d.getFullYear()
    }

    function gunSayisi(yil, ay) {
        return new Date(yil, ay + 1, 0).getDate()
    }

    // Takvim ızgarasını (önceki/sonraki aydan taşan günler dahil, Pazartesi başlangıçlı,
    // her zaman 6 hafta = 42 hücre) saf QtQuick ile hesaplar; Qt.labs.calendar modülüne
    // bağımlı değildir çünkü bu Qt kurulumunda mevcut değil.
    function ayGunleriniHesapla(ay) {
        var yil = ay.getFullYear()
        var ayIndeks = ay.getMonth()
        var ilkGun = new Date(yil, ayIndeks, 1)
        var haftaBaslangicKaymasi = (ilkGun.getDay() + 6) % 7 // Pazartesi = 0
        var baslangic = new Date(yil, ayIndeks, 1 - haftaBaslangicKaymasi)

        var gunler = []
        for (var i = 0; i < 42; i++) {
            gunler.push(new Date(baslangic.getFullYear(), baslangic.getMonth(), baslangic.getDate() + i))
        }
        return gunler
    }

    property var gunHucreleri: ayGunleriniHesapla(gorunenAy)

    readonly property var ayAdlari: [
        { tr: "Ocak", en: "January" }, { tr: "Şubat", en: "February" }, { tr: "Mart", en: "March" },
        { tr: "Nisan", en: "April" }, { tr: "Mayıs", en: "May" }, { tr: "Haziran", en: "June" },
        { tr: "Temmuz", en: "July" }, { tr: "Ağustos", en: "August" }, { tr: "Eylül", en: "September" },
        { tr: "Ekim", en: "October" }, { tr: "Kasım", en: "November" }, { tr: "Aralık", en: "December" }
    ]
    readonly property var haftaGunleri: [
        { tr: "Pt", en: "Mo" }, { tr: "Sa", en: "Tu" }, { tr: "Ça", en: "We" },
        { tr: "Pe", en: "Th" }, { tr: "Cu", en: "Fr" }, { tr: "Ct", en: "Sa" }, { tr: "Pz", en: "Su" }
    ]

    Rectangle {
        id: kutu
        anchors.fill: parent
        radius: 8
```

```
        color: "#0f1420"
        border.color: (alan.containsMouse || takvimPopup.visible) ? "#3b82f6" : "#1e2a3f"
        border.width: 1
        Behavior on border.color { ColorAnimation { duration: 120 } }

        Text {
            anchors.left: parent.left
            anchors.right: simgeMetni.left
            anchors.leftMargin: 10
            anchors.verticalCenter: parent.verticalCenter
            elide: Text.ElideRight
            text: root.secilenTarih.length > 0 ? root.secilenTarih : root.placeholderText
            color: root.secilenTarih.length > 0 ? "#dce8f5" : "#4b5563"
            font.family: "Segoe UI"
            font.pixelSize: 13
        }

        Text {
            id: simgeMetni
            anchors.right: parent.right
            anchors.rightMargin: 10
            anchors.verticalCenter: parent.verticalCenter
            text: "📅"
            font.pixelSize: 12
            color: "#6b7280"
        }

        MouseArea {
            id: alan
            anchors.fill: parent
            hoverEnabled: true
            cursorShape: Qt.PointingHandCursor
            onClicked: {
                if (root.secilenTarih.length === 10) {
                    var parcalar = root.secilenTarih.split(".")
                    root.gorunenAy = new Date(parseInt(parcalar[2], 10), parseInt(parcalar[1], 10) - 1, 1)
                } else {
                    root.gorunenAy = new Date()
                }
                takvimPopup.open()
            }
        }
    }

    Popup {
        id: takvimPopup
        y: kutu.height + 6
        width: root.gridGenisligi + 28
        padding: 14
        modal: true
        focus: true
        closePolicy: Popup.CloseOnEscape | Popup.CloseOnPressOutside

        background: Rectangle {
            color: "#0f1420"
            radius: 12
            border.color: "#1e2a3f"
            border.width: 1
        }

        Overlay.modal: Rectangle {
            color: "#a6000000"
        }

        contentItem: Column {
            spacing: 10

            Row {
                width: root.gridGenisligi
                height: 28

                Rectangle {
                    width: 28
                    height: 28
                    radius: 6
                    anchors.verticalCenter: parent.verticalCenter
                    color: oncekiAlan.containsMouse ? "#1e2a3f" : "transparent"
                    Behavior on color { ColorAnimation { duration: 120 } }

                    Text {
```

```
                        anchors.centerIn: parent
                        text: "‹"
                        color: "#9ca3af"
                        font.pixelSize: 16
                    }

                    MouseArea {
                        id: oncekiAlan
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onClicked: root.gorunenAy = new Date(root.gorunenAy.getFullYear(), root.gorunenAy.getMonth() - 1, 1)
                    }
                }

                Text {
                    width: parent.width - 56
                    anchors.verticalCenter: parent.verticalCenter
                    horizontalAlignment: Text.AlignHCenter
                    text: (Translations.turkish ? root.ayAdlari[root.gorunenAy.getMonth()].tr : root.ayAdlari[root.gorunenAy.getMonth()].en) + " " + root.gorunenAy.getFullYear()
                    color: "#dce8f5"
                    font.family: "Segoe UI"
                    font.pixelSize: 13
                    font.bold: true
                }

                Rectangle {
                    width: 28
                    height: 28
                    radius: 6
                    anchors.verticalCenter: parent.verticalCenter
                    color: sonrakiAlan.containsMouse ? "#1e2a3f" : "transparent"
                    Behavior on color { ColorAnimation { duration: 120 } }

                    Text {
                        anchors.centerIn: parent
                        text: "›"
                        color: "#9ca3af"
                        font.pixelSize: 16
                    }

                    MouseArea {
                        id: sonrakiAlan
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onClicked: root.gorunenAy = new Date(root.gorunenAy.getFullYear(), root.gorunenAy.getMonth() + 1, 1)
                    }
                }
            }

            Row {
                width: root.gridGenisligi
                spacing: root.hucreAraligi

                Repeater {
                    model: root.haftaGunleri

                    delegate: Text {
                        width: root.hucreBoyutu
                        horizontalAlignment: Text.AlignHCenter
                        text: Translations.turkish ? modelData.tr : modelData.en
                        color: "#6b7280"
                        font.family: "Segoe UI"
                        font.pixelSize: 10
                        font.bold: true
                    }
                }
            }

            Grid {
                width: root.gridGenisligi
                columns: 7
                rowSpacing: root.hucreAraligi
                columnSpacing: root.hucreAraligi

                Repeater {
                    model: root.gunHucreleri

```

```
                    delegate: Rectangle {
                        id: gunHucresi
                        width: root.hucreBoyutu
                        height: root.hucreBoyutu
                        radius: root.hucreBoyutu / 2
                        property bool buAyMi: modelData.getMonth() === root.gorunenAy.getMonth()
                        property bool seciliMi: root.secilenTarih === root.tarihStr(modelData)
                        property bool bugunMu: root.tarihStr(modelData) === root.tarihStr(new Date())
                        color: seciliMi ? "#3b82f6" : (gunAlani.containsMouse ? "#1e2a3f" : "transparent")
                        border.color: (bugunMu && !seciliMi) ? "#3b82f6" : "transparent"
                        border.width: 1

                        Text {
                            anchors.centerIn: parent
                            text: modelData.getDate()
                            color: gunHucresi.buAyMi ? (gunHucresi.seciliMi ? "#ffffff" : "#dce8f5") : "#374151"
                            font.family: "Segoe UI"
                            font.pixelSize: 12
                            font.bold: gunHucresi.seciliMi
                        }

                        MouseArea {
                            id: gunAlani
                            anchors.fill: parent
                            hoverEnabled: true
                            cursorShape: Qt.PointingHandCursor
                            onClicked: {
                                root.tarihSecildi(root.tarihStr(modelData))
                                takvimPopup.close()
                            }
                        }
                    }
                }
            }

            Rectangle {
                width: root.gridGenisligi
                height: 1
                color: "#1e2a3f"
            }

            Rectangle {
                width: root.gridGenisligi
                height: 30
                radius: 6
                color: temizleAlani.containsMouse ? "#1e2a3f" : "transparent"
                Behavior on color { ColorAnimation { duration: 120 } }

                Text {
                    anchors.centerIn: parent
                    text: root.txt("temizleButon")
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 12
                }

                MouseArea {
                    id: temizleAlani
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    onClicked: {
                        root.tarihSecildi("")
                        takvimPopup.close()
                    }
                }
            }
        }
    }
}
```

qml/pages/DashboardPage.qml

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtQuick.Layouts 6.7
import QtMultimedia
import sliper

Rectangle {
    color: "#0a0e17"

    signal cikisYapildi()

    readonly property var navOgeleri: [
        { etiketTr: "Ölçüm", etiketEn: "Measurement", ikon: "📊" },
        { etiketTr: "Sonuçlar", etiketEn: "Results", ikon: "📈" },
        { etiketTr: "Geçmiş", etiketEn: "History", ikon: "🕒" },
        { etiketTr: "Kalibrasyon", etiketEn: "Calibration", ikon: "⚙️" }
    ]

    readonly property var metinler: ({
        baglandi: { tr: "SLIPER-ESP32 Bağlı", en: "SLIPER-ESP32 Connected" },
        baglaniyor: { tr: "Bağlanıyor...", en: "Connecting..." },
        bagliDegil: { tr: "Bağlı Değil", en: "Not Connected" },
        batarya: { tr: "🔋 Batarya: ", en: "🔋 Battery: " },
        sesliKomut: { tr: "🎙 Sesli Komut: ", en: "🎙 Voice Command: " },
        yukleniyor: { tr: "Yükleniyor...", en: "Loading..." },
        acik: { tr: "Açık", en: "On" },
        kapali: { tr: "Kapalı", en: "Off" },
        cikisYap: { tr: "Çıkış Yap", en: "Log Out" }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    Row {
        anchors.fill: parent
        spacing: 0

        Rectangle {
            id: sidebar
            width: 230
            height: parent.height

            gradient: Gradient {
                GradientStop { position: 0.0; color: "#121b30" }
                GradientStop { position: 1.0; color: "#080b12" }
            }

            Image {
                anchors.top: parent.top
                anchors.horizontalCenter: parent.horizontalCenter
                anchors.topMargin: 24
                source: "../assets/logo.png"
                width: 144
                height: 144
                fillMode: Image.PreserveAspectFit
            }

            Rectangle {
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.leftMargin: 24
                anchors.rightMargin: 24
                anchors.topMargin: 172
                height: 1
                color: "#1f2c46"
            }

            Row {
                anchors.top: parent.top
                anchors.topMargin: 184
                anchors.horizontalCenter: parent.horizontalCenter
                spacing: 6

                Rectangle {
                    width: 32
```

```
                    height: 22
                    radius: 5
                    color: "#0f1420"
                    border.color: Translations.turkish ? "#3b82f6" : "#1e2a3f"
                    border.width: 1

                    Text {
                        anchors.centerIn: parent
                        text: "TR"
                        color: Translations.turkish ? "#ffffff" : "#6b7280"
                        font.pixelSize: 10
                        font.bold: Translations.turkish
                    }

                    MouseArea {
                        anchors.fill: parent
                        cursorShape: Qt.PointingHandCursor
                        onClicked: Translations.turkish = true
                    }
                }

                Rectangle {
                    width: 32
                    height: 22
                    radius: 5
                    color: "#0f1420"
                    border.color: !Translations.turkish ? "#3b82f6" : "#1e2a3f"
                    border.width: 1

                    Text {
                        anchors.centerIn: parent
                        text: "EN"
                        color: !Translations.turkish ? "#ffffff" : "#6b7280"
                        font.pixelSize: 10
                        font.bold: !Translations.turkish
                    }

                    MouseArea {
                        anchors.fill: parent
                        cursorShape: Qt.PointingHandCursor
                        onClicked: Translations.turkish = false
                    }
                }

                Rectangle {
                    id: cikisButonu
                    width: cikisMetni.implicitWidth + 16
                    height: 22
                    radius: 5
                    color: cikisAlani.pressed ? "#4c1414" : (cikisAlani.containsMouse ? "#3a1414" : "#1f1414")
                    border.color: "#7f1d1d"
                    border.width: 1
                    Behavior on color { ColorAnimation { duration: 120 } }

                    Text {
                        id: cikisMetni
                        anchors.centerIn: parent
                        text: txt("cikisYap")
                        color: "#f87171"
                        font.family: "Segoe UI"
                        font.pixelSize: 10
                        font.bold: true
                    }

                    MouseArea {
                        id: cikisAlani
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onClicked: cikisYapildi()
                    }
                }
            }

            Rectangle {
                id: baglantiDurumu
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.leftMargin: 24
```

```
                anchors.rightMargin: 24
                anchors.topMargin: 224
                height: 36
                radius: 6
                color: baglantiAlani.hoverli ? "#101a2c" : "transparent"
                Behavior on color { ColorAnimation { duration: 120 } }

                Row {
                    anchors.left: parent.left
                    anchors.leftMargin: 8
                    anchors.verticalCenter: parent.verticalCenter
                    spacing: 8

                    Rectangle {
                        width: 8
                        height: 8
                        radius: 4
                        anchors.verticalCenter: parent.verticalCenter
                        color: wifiManager.baglandi ? "#16a34a" : (wifiManager.baglaniyor ? "#f59e0b" : "#dc2626")
                    }

                    Text {
                        text: wifiManager.baglandi ? txt("baglandi") : (wifiManager.baglaniyor ? txt("baglaniyor") : txt("bagliDegil"))
                        color: "#9ca3af"
                        font.family: "Segoe UI"
                        font.pixelSize: 12
                        anchors.verticalCenter: parent.verticalCenter
                    }
                }

                MouseArea {
                    id: baglantiAlani
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    enabled: !wifiManager.baglandi && !wifiManager.baglaniyor
                    onClicked: wifiManager.baglan()
                }
            }

            Rectangle {
                id: bataryaGostergesi
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.leftMargin: 24
                anchors.rightMargin: 24
                anchors.topMargin: 236
                height: 22
                radius: 4
                visible: wifiManager.baglandi && sensorManager.bataryaVoltaj > 0.1
                color: {
                    if (sensorManager.bataryaVoltaj >= 14.5) return "#0f2417"
                    if (sensorManager.bataryaVoltaj >= 14.0) return "#2a2414"
                    return "#2a1414"
                }

                Text {
                    anchors.left: parent.left
                    anchors.leftMargin: 8
                    anchors.verticalCenter: parent.verticalCenter
                    text: txt("batarya") + sensorManager.bataryaVoltaj.toFixed(2) + " V"
                    color: {
                        if (sensorManager.bataryaVoltaj >= 14.5) return "#4ade80"
                        if (sensorManager.bataryaVoltaj >= 14.0) return "#e8a020"
                        return "#f87171"
                    }
                    font.family: "Segoe UI"
                    font.pixelSize: 11
                    font.bold: true
                }
            }

            Rectangle {
                id: sesKontrolu
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.leftMargin: 24
                anchors.rightMargin: 24
```

```
                anchors.topMargin: 264
                height: 30
                radius: 6
                color: sesKontroluAlani.hoverli ? "#101a2c" : "transparent"
                Behavior on color { ColorAnimation { duration: 120 } }

                Row {
                    anchors.left: parent.left
                    anchors.leftMargin: 8
                    anchors.verticalCenter: parent.verticalCenter
                    spacing: 8

                    Rectangle {
                        width: 8
                        height: 8
                        radius: 4
                        anchors.verticalCenter: parent.verticalCenter
                        color: !voiceCommandManager.modelHazir ? "#6b7280" : (voiceCommandManager.etkin ? "#16a34a" : "#dc2626")
                    }

                    Text {
                        text: txt("sesliKomut") + (!voiceCommandManager.modelHazir ? txt("yukleniyor") : (voiceCommandManager.etkin ? txt("acik") : txt("kapali")))
                        color: "#9ca3af"
                        font.family: "Segoe UI"
                        font.pixelSize: 12
                        anchors.verticalCenter: parent.verticalCenter
                    }
                }

                MouseArea {
                    id: sesKontroluAlani
                    property bool hoverli: false
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    enabled: voiceCommandManager.modelHazir
                    onEntered: hoverli = true
                    onExited: hoverli = false
                    onClicked: voiceCommandManager.etkin = !voiceCommandManager.etkin
                }
            }

            Rectangle {
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.leftMargin: 24
                anchors.rightMargin: 24
                anchors.topMargin: 302
                height: 1
                color: "#1f2c46"
            }

            Column {
                id: navColumn
                anchors.top: parent.top
                anchors.topMargin: 320
                width: parent.width
                spacing: 2

                Repeater {
                    model: navOgeleri

                    Rectangle {
                        id: navOgesi
                        width: parent.width
                        height: 46
                        property bool aktif: icerikStack.currentIndex === index
                        property bool hoverli: false
                        color: aktif ? "#182644" : (hoverli ? "#101a2c" : "transparent")
                        Behavior on color { ColorAnimation { duration: 120 } }

                        Rectangle {
                            width: 3
                            height: parent.height
                            color: "#3b82f6"
                            visible: navOgesi.aktif
                        }

```

```
                    Row {
                        anchors.left: parent.left
                        anchors.leftMargin: 24
                        anchors.verticalCenter: parent.verticalCenter
                        spacing: 12

                        Text {
                            text: modelData.ikon
                            font.pixelSize: 15
                            anchors.verticalCenter: parent.verticalCenter
                        }

                        Text {
                            text: Translations.turkish ? modelData.etiketTr : modelData.etiketEn
                            color: navOgesi.aktif ? "#6ea8ff" : (navOgesi.hoverli ? "#dce8f5" : "#9ca3af")
                            font.family: "Segoe UI"
                            font.pixelSize: 14
                            font.bold: navOgesi.aktif
                            anchors.verticalCenter: parent.verticalCenter
                            Behavior on color { ColorAnimation { duration: 120 } }
                        }
                    }

                    MouseArea {
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onEntered: navOgesi.hoverli = true
                        onExited: navOgesi.hoverli = false
                        onClicked: icerikStack.currentIndex = index
                    }
                }
            }
        }

        MediaPlayer {
            id: simulasyonOynatici
            source: "../assets/slipergif.mp4"
            videoOutput: simulasyonVideo
            loops: 1
        }

        Item {
            id: simulasyonAlani
            anchors.top: navColumn.bottom
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            anchors.margins: 16
            anchors.topMargin: 20
            clip: true

            VideoOutput {
                id: simulasyonVideo
                anchors.fill: parent
            }
        }
    }

    Rectangle {
        id: sidebarAyraci
        width: 3
        height: parent.height
        color: "#05070b"

        Rectangle {
            anchors.right: parent.right
            anchors.top: parent.top
            anchors.bottom: parent.bottom
            width: 1
            color: "#403b82f6"
        }
    }

    StackLayout {
        id: icerikStack
        width: parent.width - sidebar.width - sidebarAyraci.width
        height: parent.height
        currentIndex: 0

```

```
            OlcumPage {
                onOlcumTamamlandi: function(id) {
                    sonuclarSayfasi.olcumId = id
                    icerikStack.currentIndex = 1
                }
                onAgirlikEklendi: {
                    simulasyonOynatici.stop()
                    simulasyonOynatici.play()
                }
            }

            SonuclarPage {
                id: sonuclarSayfasi
            }

            GecmisPage {
                onTestSecildi: function(id) {
                    sonuclarSayfasi.olcumId = id
                    icerikStack.currentIndex = 1
                }
            }

            KalibrasyonPage {}
        }
    }
}
```

qml/pages/GecmisPage.qml

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import "../components"
import sliper

Rectangle {
    id: gecmisSayfasi
    color: "#0a0e17"

    signal testSecildi(int id)

    property var olcumListesi: []
    property string baslangicTarihi: ""
    property string bitisTarihi: ""

    readonly property var filtreAnahtarlari: ["tumTestler", "son7Gun", "son30Gun"]

    readonly property var metinler: ({
        aramaPlaceholder: { tr: "Müşteri veya reçete ara...", en: "Search by customer or recipe..." },
        tumTestler: { tr: "Tüm Testler", en: "All Tests" },
        son7Gun: { tr: "Son 7 Gün", en: "Last 7 Days" },
        son30Gun: { tr: "Son 30 Gün", en: "Last 30 Days" },
        ozelTarih: { tr: "Özel Tarih:", en: "Custom Date:" },
        temizle: { tr: "Temizle", en: "Clear" },
        musteriEtiket: { tr: "Müşteri: ", en: "Customer: " },
        receteEtiket: { tr: "  •  Reçete: ", en: "  •  Recipe: " },
        goruntule: { tr: "Görüntüle", en: "View" },
        kayitSilBaslik: { tr: "Kaydı Sil", en: "Delete Record" },
        kayitSilOnSoru: { tr: "'", en: "Are you sure you want to delete the measurement for '" },
        kayitSilSonSoru: { tr: "' ölçümünü silmek istediğinize emin misiniz? Bu işlem geri alınamaz.", en: "'? This action cannot be undone." },
        sifreEtiket: { tr: "ŞİFRE", en: "PASSWORD" },
        sifrenizGirin: { tr: "Şifrenizi girin", en: "Enter your password" },
        sifreHatali: { tr: "Şifre hatalı, tekrar deneyin.", en: "Incorrect password, please try again." },
        sil: { tr: "Sil", en: "Delete" },
        iptal: { tr: "İptal", en: "Cancel" },
        gelismisAyarlarBaslik: { tr: "Gelişmiş Ayarlar", en: "Advanced Settings" },
        onayla: { tr: "Onayla", en: "Confirm" },
        veriTabaniDisaAktar: { tr: "Veritabanını Dışa Aktar", en: "Export Database" },
        disaAktarildi: { tr: "✓ Dışa aktarıldı: ", en: "✓ Exported: " },
        disaAktarilamadi: { tr: "✕ Dışa aktarılamadı", en: "✕ Export failed" },
        icaAktarYoluEtiket: { tr: "İÇE AKTARILACAK .DB DOSYA YOLU", en: ".DB FILE PATH TO IMPORT" },
        icaAktarPlaceholder: { tr: "ör. C:/Users/.../SliperRaporlari/sliper_yedek_....db", en: "e.g. C:/Users/.../SliperRaporlari/sliper_yedek_....db" },
        veriTabaniIcaAktar: { tr: "Veritabanını İçe Aktar", en: "Import Database" },
        icaAktarildi: { tr: "✓ İçe aktarıldı, uygulamayı yeniden başlatmanız önerilir.", en: "✓ Imported, it is recommended to restart the application." },
        icaAktarilamadi: { tr: "✕ İçe aktarılamadı, dosya yolunu kontrol edin.", en: "✕ Import failed, please check the file path." },
        tumVeriyiSil: { tr: "Tüm Verileri Sil", en: "Delete All Data" },
        kapat: { tr: "Kapat", en: "Close" },
        tumVeriyiSilMesaj: { tr: "Tüm ölçüm ve stroke verileri kalıcı olarak silinecek. Sensör kalibrasyonları etkilenmez. Bu işlem GERİ ALINAMAZ, emin misiniz?", en: "All measurement and stroke data will be permanently deleted. Sensor calibrations will not be affected. This action CANNOT BE UNDONE, are you sure?" },
        evetHepsiniSil: { tr: "Evet, Hepsini Sil", en: "Yes, Delete All" },
        tumVerilerSilindi: { tr: "✓ Tüm veriler silindi.", en: "✓ All data deleted." }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    function listeyiYenile() {
        olcumListesi = database.tumOlcumleriGetir()
    }

    onVisibleChanged: {
        if (visible) {
            listeyiYenile()
        }
    }

    Component.onCompleted: listeyiYenile()

    function tarihiAyristir(tarihStr) {
        if (!tarihStr) return null
        var tarihKismi = tarihStr.split(" ")[0].split(".")
        if (tarihKismi.length !== 3) return null
```

```
        var gun = parseInt(tarihKismi[0], 10)
        var ay = parseInt(tarihKismi[1], 10)
        var yil = parseInt(tarihKismi[2], 10)
        if (isNaN(gun) || isNaN(ay) || isNaN(yil)) return null
        return new Date(yil, ay - 1, gun)
    }

    function gecerliOzelTarih(metin) {
        if (!metin || metin.length !== 10) return null
        return tarihiAyristir(metin)
    }

    function filtrelenmisListeyiHesapla() {
        var sonuc = []
        var aramaMetni = aramaKutusu.text.toLowerCase()
        var ozelBaslangic = gecerliOzelTarih(gecmisSayfasi.baslangicTarihi)
        var ozelBitis = gecerliOzelTarih(gecmisSayfasi.bitisTarihi)
        var simdi = new Date()

        for (var i = 0; i < olcumListesi.length; i++) {
            var kayit = olcumListesi[i]
            var tarihObj = tarihiAyristir(kayit.tarih)

            if (aramaMetni.length > 0) {
                var musteriEslesir = kayit.musteri.toLowerCase().indexOf(aramaMetni) !== -1
                var receteEslesir = kayit.recete.toLowerCase().indexOf(aramaMetni) !== -1
                if (!musteriEslesir && !receteEslesir) continue
            }

            if (ozelBaslangic || ozelBitis) {
                if (ozelBaslangic && tarihObj && tarihObj < ozelBaslangic) continue
                if (ozelBitis && tarihObj) {
                    var bitisSonu = new Date(ozelBitis.getFullYear(), ozelBitis.getMonth(), ozelBitis.getDate(), 23, 59, 59)
                    if (tarihObj > bitisSonu) continue
                }
            } else if (tarihObj) {
                if (filtreKutusu.currentIndex === 1) {
                    var esik7 = new Date(simdi.getTime() - 7 * 24 * 60 * 60 * 1000)
                    if (tarihObj < esik7) continue
                } else if (filtreKutusu.currentIndex === 2) {
                    var esik30 = new Date(simdi.getTime() - 30 * 24 * 60 * 60 * 1000)
                    if (tarihObj < esik30) continue
                }
            }

            sonuc.push(kayit)
        }

        return sonuc
    }

    property var filtrelenmisListe: filtrelenmisListeyiHesapla()

    Column {
        anchors.fill: parent
        anchors.margins: 20
        spacing: 16

        Row {
            width: parent.width
            spacing: 12

            TextField {
                id: aramaKutusu
                width: parent.width - filtreKutusu.width - gelismisAyarlarButonu.width - parent.spacing * 2
                height: 40
                placeholderText: txt("aramaPlaceholder")
                placeholderTextColor: "#4b5563"
                color: "#ffffff"
                font.pixelSize: 13
                leftPadding: 34
                verticalAlignment: TextInput.AlignVCenter
                background: Rectangle {
                    color: "#0f1420"
                    radius: 8
                    border.color: aramaKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                    border.width: 1
                }

                Text {
```

```
                    anchors.left: parent.left
                    anchors.leftMargin: 12
                    anchors.verticalCenter: parent.verticalCenter
                    text: "🔍"
                    color: "#4b5563"
                    font.pixelSize: 13
                }
            }

            ComboBox {
                id: filtreKutusu
                width: 160
                height: 40
                model: filtreAnahtarlari.map(function(anahtar) { return txt(anahtar) })

                background: Rectangle {
                    color: "#0f1420"
                    radius: 8
                    border.color: filtreKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                    border.width: 1
                }

                contentItem: Text {
                    text: filtreKutusu.displayText
                    color: "#9ca3af"
                    font.pixelSize: 13
                    leftPadding: 10
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Button {
                id: gelismisAyarlarButonu
                width: 44
                height: 40
                text: "⚙️"
                font.pixelSize: 15

                onClicked: {
                    gelismisAyarlarPopup.pinDogrulandi = false
                    pinKutusu.text = ""
                    gelismisAyarlarBildirim.text = ""
                    gelismisAyarlarPopup.open()
                }

                background: Rectangle {
                    radius: 8
                    color: parent.hovered ? "#162033" : "#0f1420"
                    border.color: "#1e2a3f"
                    border.width: 1
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#9ca3af"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }
        }

        Row {
            width: parent.width
            spacing: 10

            Text {
                text: txt("ozelTarih")
                color: "#9ca3af"
                font.family: "Segoe UI"
                font.pixelSize: 12
                anchors.verticalCenter: parent.verticalCenter
            }

            TarihSecici {
                id: baslangicSecici
                anchors.verticalCenter: parent.verticalCenter
                secilenTarih: gecmisSayfasi.baslangicTarihi
                onTarihSecildi: function(t) { gecmisSayfasi.baslangicTarihi = t }
```

```
            }

            Text {
                text: "–"
                color: "#4b5563"
                font.pixelSize: 13
                anchors.verticalCenter: parent.verticalCenter
            }

            TarihSecici {
                id: bitisSecici
                anchors.verticalCenter: parent.verticalCenter
                secilenTarih: gecmisSayfasi.bitisTarihi
                onTarihSecildi: function(t) { gecmisSayfasi.bitisTarihi = t }
            }

            Rectangle {
                width: temizleMetni.implicitWidth + 20
                height: 36
                radius: 8
                color: temizleMouse.containsMouse ? "#162033" : "#0f1420"
                border.color: "#1e2a3f"
                border.width: 1
                visible: gecmisSayfasi.baslangicTarihi.length > 0 || gecmisSayfasi.bitisTarihi.length > 0
                anchors.verticalCenter: parent.verticalCenter

                Text {
                    id: temizleMetni
                    anchors.centerIn: parent
                    text: txt("temizle")
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 12
                }

                MouseArea {
                    id: temizleMouse
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    onClicked: {
                        gecmisSayfasi.baslangicTarihi = ""
                        gecmisSayfasi.bitisTarihi = ""
                    }
                }
            }
        }

        ListView {
            width: parent.width
            height: parent.height - 96
            clip: true
            spacing: 10

            model: gecmisSayfasi.filtrelenmisListe

            delegate: Rectangle {
                id: gecmisKarti
                width: parent.width
                height: 76
                radius: 10
                color: "#0f1420"
                border.color: kartAlani.containsMouse ? "#3b82f6" : "#1e2a3f"
                border.width: 1
                Behavior on border.color { ColorAnimation { duration: 120 } }

                MouseArea {
                    id: kartAlani
                    anchors.fill: parent
                    hoverEnabled: true
                    acceptedButtons: Qt.NoButton
                }

                Rectangle {
                    id: rozet
                    width: 44
                    height: 44
                    radius: 10
                    anchors.left: parent.left
                    anchors.verticalCenter: parent.verticalCenter
```

```
    anchors.leftMargin: 16
    color: "#132335"

    Text {
        anchors.centerIn: parent
        text: modelData.musteri.charAt(0).toUpperCase()
        color: "#3b82f6"
        font.family: "Segoe UI"
        font.pixelSize: 18
        font.bold: true
    }
}

Column {
    anchors.left: rozet.right
    anchors.verticalCenter: parent.verticalCenter
    anchors.leftMargin: 14
    spacing: 4

    Row {
        spacing: 8

        Text {
            text: modelData.tarih
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 11
        }

        Rectangle {
            width: strokeMetni.implicitWidth + 12
            height: 16
            radius: 8
            color: "#2a2010"
            anchors.verticalCenter: parent.verticalCenter

            Text {
                id: strokeMetni
                anchors.centerIn: parent
                text: modelData.strokeSayisi + " stroke"
                color: "#e8a020"
                font.family: "Segoe UI"
                font.pixelSize: 10
                font.bold: true
            }
        }
    }

    Text {
        text: txt("musteriEtiket") + modelData.musteri + txt("receteEtiket") + modelData.recete
        color: "#dce8f5"
        font.family: "Segoe UI"
        font.pixelSize: 13
        font.bold: true
    }
}

Row {
    anchors.right: parent.right
    anchors.verticalCenter: parent.verticalCenter
    anchors.rightMargin: 16
    spacing: 8

    Rectangle {
        id: silButonu
        width: 34
        height: 34
        radius: 6
        color: silMouse.containsMouse ? "#3f1d24" : "#1a1420"
        border.color: silMouse.containsMouse ? "#ef4444" : "#2a2030"
        border.width: 1
        Behavior on color { ColorAnimation { duration: 120 } }

        Text {
            anchors.centerIn: parent
            text: "🗑"
            font.pixelSize: 14
        }

        MouseArea {
```

```
                            id: silMouse
                            anchors.fill: parent
                            hoverEnabled: true
                            cursorShape: Qt.PointingHandCursor
                            onClicked: {
                                silOnayPopup.hedefId = modelData.id
                                silOnayPopup.hedefMusteri = modelData.musteri
                                silOnayPopup.open()
                            }
                        }
                    }

                    Button {
                        anchors.verticalCenter: parent.verticalCenter
                        height: 34
                        text: txt("goruntule")
                        font.pixelSize: 12

                        onClicked: testSecildi(modelData.id)

                        background: Rectangle {
                            radius: 6
                            color: parent.hovered ? "#4f8cf7" : "#3b82f6"
                            Behavior on color { ColorAnimation { duration: 120 } }
                        }

                        contentItem: Text {
                            text: parent.text
                            color: "#ffffff"
                            font: parent.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                            leftPadding: 8
                            rightPadding: 8
                        }
                    }
                }
            }
        }
    }

    Popup {
        id: silOnayPopup
        anchors.centerIn: parent
        width: 380
        padding: 24
        modal: true
        focus: true
        closePolicy: Popup.CloseOnEscape

        property int hedefId: -1
        property string hedefMusteri: ""

        onOpened: {
            sifreGirisAlani.text = ""
            hataMetni.visible = false
            sifreGirisAlani.forceActiveFocus()
        }

        background: Rectangle {
            color: "#0f1420"
            radius: 14
            border.color: "#1e2a3f"
            border.width: 1
        }

        contentItem: Column {
            width: silOnayPopup.availableWidth
            spacing: 16

            Text {
                text: txt("kayitSilBaslik")
                color: "#ffffff"
                font.family: "Segoe UI"
                font.pixelSize: 17
                font.bold: true
            }

            Text {
                width: parent.width
```

```
                wrapMode: Text.WordWrap
                text: txt("kayitSilOnSoru") + silOnayPopup.hedefMusteri + txt("kayitSilSonSoru")
                color: "#9ca3af"
                font.family: "Segoe UI"
                font.pixelSize: 13
            }

            Column {
                width: parent.width
                spacing: 6

                Text {
                    text: txt("sifreEtiket")
                    color: "#6b7280"
                    font.family: "Segoe UI"
                    font.pixelSize: 10
                    font.letterSpacing: 1
                }

                TextField {
                    id: sifreGirisAlani
                    width: parent.width
                    height: 40
                    echoMode: TextInput.Password
                    placeholderText: txt("sifrenizGirin")
                    placeholderTextColor: "#4b5563"
                    color: "#ffffff"
                    font.pixelSize: 13
                    leftPadding: 12
                    verticalAlignment: TextInput.AlignVCenter
                    background: Rectangle {
                        color: "#0a0e17"
                        radius: 6
                        border.color: sifreGirisAlani.activeFocus ? "#3b82f6" : "#1e2a3f"
                        border.width: 1
                    }
                    Keys.onReturnPressed: silOnaylaButonu.clicked()
                }

                Text {
                    id: hataMetni
                    text: txt("sifreHatali")
                    color: "#ef4444"
                    font.family: "Segoe UI"
                    font.pixelSize: 11
                    visible: false
                }
            }

            Row {
                width: parent.width
                spacing: 10
                layoutDirection: Qt.RightToLeft

                Button {
                    id: silOnaylaButonu
                    width: (parent.width - 10) / 2
                    height: 40
                    text: txt("sil")
                    font.pixelSize: 13
                    font.bold: true

                    onClicked: {
                        if (backend.sifreDogrula(sifreGirisAlani.text)) {
                            database.olcumSil(silOnayPopup.hedefId)
                            gecmisSayfasi.listeyiYenile()
                            silOnayPopup.close()
                        } else {
                            hataMetni.visible = true
                            sifreGirisAlani.text = ""
                            sifreGirisAlani.forceActiveFocus()
                        }
                    }

                    background: Rectangle {
                        radius: 8
                        color: parent.pressed ? "#b91c1c" : (parent.hovered ? "#ef4444" : "#dc2626")
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

```

```
                    contentItem: Text {
                        text: parent.text
                        color: "#ffffff"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Button {
                    width: (parent.width - 10) / 2
                    height: 40
                    text: txt("iptal")
                    font.pixelSize: 13

                    onClicked: silOnayPopup.close()

                    background: Rectangle {
                        radius: 8
                        color: parent.hovered ? "#162033" : "#0f1420"
                        border.color: "#1e2a3f"
                        border.width: 1
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#9ca3af"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }

    // Orijinal SLIPER'daki "Expert Settings > Database Export / Import / Delete"
    // özelliklerine karşılık gelir (bkz. slipermanV13.pdf bölüm 6). Şifre
    // doğrulaması, uygulamada zaten var olan giriş şifresini (backend.sifreDogrula)
    // kullanır; orijinal cihazdaki ayrı "2603" servis şifresi yerine bu tercih
    // edildi çünkü uygulamada zaten tek bir şifre akışı var.
    Popup {
        id: gelismisAyarlarPopup
        anchors.centerIn: parent
        width: 420
        padding: 24
        modal: true
        focus: true
        closePolicy: Popup.CloseOnEscape

        property bool pinDogrulandi: false

        background: Rectangle {
            color: "#0f1420"
            radius: 14
            border.color: "#1e2a3f"
            border.width: 1
        }

        contentItem: Column {
            width: gelismisAyarlarPopup.availableWidth
            spacing: 16

            Text {
                text: txt("gelismisAyarlarBaslik")
                color: "#ffffff"
                font.family: "Segoe UI"
                font.pixelSize: 17
                font.bold: true
            }

            Column {
                width: parent.width
                spacing: 6
                visible: !gelismisAyarlarPopup.pinDogrulandi

                Text {
                    text: txt("sifreEtiket")
                    color: "#6b7280"
```

```
                        font.family: "Segoe UI"
                        font.pixelSize: 10
                        font.letterSpacing: 1
                    }

                    TextField {
                        id: pinKutusu
                        width: parent.width
                        height: 40
                        echoMode: TextInput.Password
                        placeholderText: txt("sifrenizGirin")
                        placeholderTextColor: "#4b5563"
                        color: "#ffffff"
                        font.pixelSize: 13
                        leftPadding: 12
                        verticalAlignment: TextInput.AlignVCenter
                        background: Rectangle {
                            color: "#0a0e17"
                            radius: 6
                            border.color: pinKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                            border.width: 1
                        }
                        Keys.onReturnPressed: pinDogrulaButonu.clicked()
                    }

                    Text {
                        id: pinHataMetni
                        text: txt("sifreHatali")
                        color: "#ef4444"
                        font.family: "Segoe UI"
                        font.pixelSize: 11
                        visible: false
                    }

                    Button {
                        id: pinDogrulaButonu
                        width: parent.width
                        height: 40
                        text: txt("onayla")
                        font.pixelSize: 13
                        font.bold: true

                        onClicked: {
                            if (backend.sifreDogrula(pinKutusu.text)) {
                                gelismisAyarlarPopup.pinDogrulandi = true
                                pinHataMetni.visible = false
                            } else {
                                pinHataMetni.visible = true
                                pinKutusu.text = ""
                                pinKutusu.forceActiveFocus()
                            }
                        }

                        background: Rectangle {
                            radius: 8
                            color: parent.hovered ? "#4f8cf7" : "#3b82f6"
                            Behavior on color { ColorAnimation { duration: 120 } }
                        }

                        contentItem: Text {
                            text: parent.text
                            color: "#ffffff"
                            font: parent.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                        }
                    }
                }

                Column {
                    width: parent.width
                    spacing: 10
                    visible: gelismisAyarlarPopup.pinDogrulandi

                    Button {
                        width: parent.width
                        height: 40
                        text: txt("veriTabaniDisaAktar")
                        font.pixelSize: 13

```

```
    onClicked: {
        var yol = database.veriTabaniDisaAktar()
        gelismisAyarlarBildirim.text = yol.length > 0
            ? txt("disaAktarildi") + yol
            : txt("disaAktarilamadi")
    }

    background: Rectangle {
        radius: 8
        color: parent.hovered ? "#162033" : "#0f1420"
        border.color: "#1e2a3f"
        border.width: 1
        Behavior on color { ColorAnimation { duration: 120 } }
    }

    contentItem: Text {
        text: parent.text
        color: "#dce8f5"
        font: parent.font
        horizontalAlignment: Text.AlignHCenter
        verticalAlignment: Text.AlignVCenter
    }
}

Column {
    width: parent.width
    spacing: 6

    Text {
        text: txt("icaAktarYoluEtiket")
        color: "#6b7280"
        font.family: "Segoe UI"
        font.pixelSize: 10
        font.letterSpacing: 1
    }

    TextField {
        id: icaAktarYoluKutusu
        width: parent.width
        height: 40
        placeholderText: txt("icaAktarPlaceholder")
        placeholderTextColor: "#4b5563"
        color: "#ffffff"
        font.pixelSize: 12
        leftPadding: 12
        verticalAlignment: TextInput.AlignVCenter
        background: Rectangle {
            color: "#0a0e17"
            radius: 6
            border.color: icaAktarYoluKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
            border.width: 1
        }
    }
}

Button {
    width: parent.width
    height: 40
    text: txt("veriTabaniIcaAktar")
    font.pixelSize: 13

    onClicked: {
        var basarili = database.veriTabaniIcaAktar(icaAktarYoluKutusu.text)
        gelismisAyarlarBildirim.text = basarili
            ? txt("icaAktarildi")
            : txt("icaAktarilamadi")
        if (basarili) {
            gecmisSayfasi.listeyiYenile()
        }
    }

    background: Rectangle {
        radius: 8
        color: parent.hovered ? "#162033" : "#0f1420"
        border.color: "#1e2a3f"
        border.width: 1
        Behavior on color { ColorAnimation { duration: 120 } }
    }

    contentItem: Text {
```

```
                        text: parent.text
                        color: "#dce8f5"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Button {
                    width: parent.width
                    height: 40
                    text: txt("tumVeriyiSil")
                    font.pixelSize: 13
                    font.bold: true

                    onClicked: tumVeriyiSilOnayPopup.open()

                    background: Rectangle {
                        radius: 8
                        color: parent.pressed ? "#b91c1c" : (parent.hovered ? "#ef4444" : "#dc2626")
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#ffffff"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Text {
                    id: gelismisAyarlarBildirim
                    width: parent.width
                    text: ""
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 10
                    wrapMode: Text.WrapAnywhere
                }

                Button {
                    width: parent.width
                    height: 36
                    text: txt("kapat")
                    font.pixelSize: 12

                    onClicked: gelismisAyarlarPopup.close()

                    background: Rectangle {
                        radius: 8
                        color: parent.hovered ? "#162033" : "transparent"
                        border.color: "#1e2a3f"
                        border.width: 1
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#9ca3af"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }

    // "Tüm Verileri Sil" için çift onay (orijinal cihazdaki "Confirm if you
    // are really sure" uyarısına karşılık gelir).
    Popup {
        id: tumVeriyiSilOnayPopup
        anchors.centerIn: parent
        width: 380
        padding: 24
        modal: true
        focus: true
        closePolicy: Popup.CloseOnEscape
```

```
        background: Rectangle {
            color: "#0f1420"
            radius: 14
            border.color: "#dc2626"
            border.width: 1
        }

        contentItem: Column {
            width: tumVeriyiSilOnayPopup.availableWidth
            spacing: 16

            Text {
                text: txt("tumVeriyiSil")
                color: "#ffffff"
                font.family: "Segoe UI"
                font.pixelSize: 17
                font.bold: true
            }

            Text {
                width: parent.width
                wrapMode: Text.WordWrap
                text: txt("tumVeriyiSilMesaj")
                color: "#9ca3af"
                font.family: "Segoe UI"
                font.pixelSize: 13
            }

            Row {
                width: parent.width
                spacing: 10
                layoutDirection: Qt.RightToLeft

                Button {
                    width: (parent.width - 10) / 2
                    height: 40
                    text: txt("evetHepsiniSil")
                    font.pixelSize: 13
                    font.bold: true

                    onClicked: {
                        database.tumVeriyiSil()
                        gecmisSayfasi.listeyiYenile()
                        tumVeriyiSilOnayPopup.close()
                        gelismisAyarlarBildirim.text = txt("tumVerilerSilindi")
                    }

                    background: Rectangle {
                        radius: 8
                        color: parent.pressed ? "#b91c1c" : (parent.hovered ? "#ef4444" : "#dc2626")
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#ffffff"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Button {
                    width: (parent.width - 10) / 2
                    height: 40
                    text: txt("iptal")
                    font.pixelSize: 13

                    onClicked: tumVeriyiSilOnayPopup.close()

                    background: Rectangle {
                        radius: 8
                        color: parent.hovered ? "#162033" : "#0f1420"
                        border.color: "#1e2a3f"
                        border.width: 1
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
```

```
                        text: parent.text
                        color: "#9ca3af"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }
}
```

qml/pages/KalibrasyonPage.qml

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtQuick.Layouts 6.7
import "../components"
import sliper

Rectangle {
    color: "#0a0e17"

    readonly property var metinler: ({
        baslikSensorSec: { tr: "KALİBRASYON — Sensör Seçin", en: "CALIBRATION — Select Sensor" },
        egimSensoru: { tr: "Eğim Sensörü", en: "Incline Sensor" },
        egimAciklama: { tr: "Yatay hizalama kalibrasyonu", en: "Level alignment calibration" },
        kalibreEdildiOnEk: { tr: "✓ Kalibre Edildi: ", en: "✓ Calibrated: " },
        loadCellAdi: { tr: "Load Cell", en: "Load Cell" },
        loadCellAciklama: { tr: "Basınç sensörü kalibrasyonu", en: "Load pressure sensor calibration" },
        mesafeSensoru: { tr: "Mesafe Sensörü", en: "Distance Sensor" },
        mesafeAciklama: { tr: "Konum ölçümü kalibrasyonu", en: "Position measurement calibration" },
        geriButon: { tr: "← Geri", en: "← Back" },
        egimBaslik: { tr: "Eğim Sensörü Kalibrasyonu", en: "Incline Sensor Calibration" },
        loadCellBaslik: { tr: "Load Cell Kalibrasyonu", en: "Load Cell Calibration" },
        mesafeBaslik: { tr: "Mesafe Sensörü Kalibrasyonu", en: "Distance Sensor Calibration" },
        sonKalibrasyonOnEk: { tr: "Son kalibrasyon: ", en: "Last calibration: " },
        cihaziYerlestir: { tr: "Cihazı düz ve sabit bir yüzeye yerleştirin", en: "Place the device on a flat, stable surface" },
        cihazBagliDegilKalibrasyon: { tr: "Cihaz bağlı değil — kalibrasyon kaydedilemez", en: "Device not connected — calibration cannot be saved" },
        sifirlaButon: { tr: "Sıfırla (Zero)", en: "Zero (Reset)" },
        kalibrasyonKaydedildiMesaj: { tr: "✓ Kalibrasyon kaydedildi", en: "✓ Calibration saved" },
        kalibrasyonKaydedilemediMesaj: { tr: "✕ Kalibrasyon kaydedilemedi, tekrar deneyin", en: "✕ Calibration could not be saved, try again" },
        kayitliOnEk: { tr: "Kayıtlı: ", en: "Saved: " },
        noktaSonEk: { tr: " nokta", en: " points" },
        kayitliKalibrasyonBulundu: { tr: "Bu sensör için kayıtlı bir kalibrasyon bulundu", en: "A saved calibration was found for this sensor" },
        mevcutGoruntule: { tr: "Mevcut Kalibrasyonu Görüntüle", en: "View Current Calibration" },
        kayitliNoktalariIncele: { tr: "Kayıtlı noktaları incele", en: "Review saved points" },
        yenidenKalibreEt: { tr: "Yeniden Kalibre Et", en: "Recalibrate" },
        sifirdanYeniOlcum: { tr: "Sıfırdan yeni ölçüm al", en: "Take a fresh measurement" },
        kayitliNoktalarBaslik: { tr: "KAYITLI KALİBRASYON NOKTALARI", en: "SAVED CALIBRATION POINTS" },
        duzenleButon: { tr: "Düzenle", en: "Edit" },
        silinecekOnay: { tr: "Mevcut kalibrasyon silinecek, emin misiniz?", en: "The current calibration will be deleted, are you sure?" },
        vazgecButon: { tr: "Vazgeç", en: "Cancel" },
        evetYenidenBaslaButon: { tr: "Evet, Yeniden Başla", en: "Yes, Start Over" },
        kalibrasyonTamamlandiBaslik: { tr: "Kalibrasyon Tamamlandı", en: "Calibration Complete" },
        toplamOnEk: { tr: "Toplam ", en: "Total " },
        noktaGirildiSonEk: { tr: " nokta girildi. Bu kalibrasyonu kaydetmek istiyor musunuz?", en: " points entered. Do you want to save this calibration?" },
        noktalariKontrolEt: { tr: "Noktaları Kontrol Et", en: "Review Points" },
        kaydetVeBitir: { tr: "Kaydet ve Bitir", en: "Save and Finish" },
        kacNoktaSoru: { tr: "Kaç noktalı kalibrasyon yapmak istersiniz?", en: "How many calibration points would you like to use?" },
        dahaFazlaNokta: { tr: "Daha fazla nokta, daha hassas kalibrasyon sağlar", en: "More points provide a more precise calibration" },
        hamDegerOnEk: { tr: "Ham değer: ", en: "Raw value: " },
        hamDegerSifirUyari: { tr: "Cihaz bağlı değil — ham değer 0 olarak kaydedilecek", en: "Device not connected — raw value will be saved as 0" },
        degisiklikleriKaydet: { tr: "Değişiklikleri Kaydet", en: "Save Changes" },
        kaydedilenNoktalarBaslik: { tr: "KAYDEDİLEN NOKTALAR", en: "SAVED POINTS" },
        duzeltmekIcinTikla: { tr: "Düzeltmek için bir noktaya tıkla", en: "Click a point to correct it" },
        kalibrasyonuDuzenle: { tr: "Kalibrasyonu Düzenle", en: "Edit Calibration" },
        noktaOnEk: { tr: "Nokta ", en: "Point " },
        duzenleniyorSonEk: { tr: " düzenleniyor", en: " being edited" },
        duzenlemekIcinTikla: { tr: "Düzenlemek istediğiniz noktaya sağdaki listeden tıklayın", en: "Click the point you want to edit from the list on the right" },
        hedefAgirlikYonerge: { tr: "Hedef ağırlığı gir, sonra o ağırlığı load cell'e koy", en: "Enter the target weight, then place that weight on the load cell" },
        hedefMesafeYonerge: { tr: "Hedef mesafeyi gir, sonra boruyu o mesafeye getir", en: "Enter the target distance, then move the tube to that distance" },
        noktayiGuncelle: { tr: "Noktayı Güncelle", en: "Update Point" },
        kalibrasyonuTamamla: { tr: "Kalibrasyonu Tamamla", en: "Complete Calibration" },
        sonNoktayiKaydet: { tr: "Son Noktayı Kaydet", en: "Save Last Point" },
        buNoktayiKaydet: { tr: "Bu Noktayı Kaydet", en: "Save This Point" }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

```

```
    property int seciliSensor: -1
    property int loadCellAdimi: 0

    property bool egimKayitliVar: false
    property string egimSonTarih: ""
    property bool egimBildirimGoster: false
    property bool egimBildirimBasarili: true
    property string egimBildirimMesaj: ""

    property bool loadCellKayitliVar: false
    property string loadCellSonTarih: ""
    property int loadCellNoktaSayisi: 0
    property var loadCellNoktalari: []
    onLoadCellNoktalariChanged: loadCellAdimi = loadCellNoktalari.length
    property int loadCellToplamNokta: 10
    property bool loadCellSayiSecildi: false
    property int loadCellDuzenlemeIndex: -1

    property bool loadCellAraEkranGoster: false
    property bool loadCellGoruntuleModu: false
    property bool loadCellTamDuzenlemeModu: false
    property bool loadCellYenidenOnayGoster: false
    property bool loadCellTamamlandiOnayGoster: false

    property bool loadCellBildirimGoster: false
    property bool loadCellBildirimBasarili: true
    property string loadCellBildirimMesaj: ""

    property string mesafeSonTarih: ""

    property bool mesafeOlcumKayitliVar: false
    property int mesafeOlcumNoktaSayisi: 0
    property var mesafeNoktalari: []
    property int mesafeToplamNokta: 10
    property bool mesafeSayiSecildi: false
    property int mesafeDuzenlemeIndex: -1

    property bool mesafeAraEkranGoster: false
    property bool mesafeGoruntuleModu: false
    property bool mesafeTamDuzenlemeModu: false
    property bool mesafeYenidenOnayGoster: false
    property bool mesafeTamamlandiOnayGoster: false

    property bool mesafeBildirimGoster: false
    property bool mesafeBildirimBasarili: true
    property string mesafeBildirimMesaj: ""

    function loadCellSiraliKopya(dizi) {
        var kopya = dizi.slice()
        kopya.sort(function(a, b) { return a.hamDeger - b.hamDeger })
        return kopya
    }

    function loadCellKaydetVeBildir() {
        var sirali = loadCellSiraliKopya(loadCellNoktalari)
        var basarili = database.loadCellNoktalariKaydet(sirali)
        loadCellBildirimBasarili = basarili
        loadCellBildirimMesaj = basarili ? txt("kalibrasyonKaydedildiMesaj") : txt("kalibrasyonKaydedilemediMesaj")
        loadCellBildirimGoster = true
        loadCellBildirimTimer.restart()
    }

    Timer {
        id: loadCellBildirimTimer
        interval: loadCellBildirimBasarili ? 1800 : 2500
        repeat: false
        onTriggered: {
            loadCellBildirimGoster = false
            if (loadCellBildirimBasarili) {
                seciliSensor = -1
            }
        }
    }

    function mesafeSiraliKopya(dizi) {
        var kopya = dizi.slice()
        kopya.sort(function(a, b) { return a.hamDeger - b.hamDeger })
        return kopya
    }

```

```
    function mesafeKaydetVeBildir() {
        var sirali = mesafeSiraliKopya(mesafeNoktalari)
        var basarili = database.mesafeNoktalariKaydet(sirali)
        if (basarili) {
            wifiManager.kalibrasyonYenidenYukle()
        }
        mesafeBildirimBasarili = basarili
        mesafeBildirimMesaj = basarili ? txt("kalibrasyonKaydedildiMesaj") : txt("kalibrasyonKaydedilemediMesaj")
        mesafeBildirimGoster = true
        mesafeBildirimTimer.restart()
    }

    Timer {
        id: mesafeBildirimTimer
        interval: mesafeBildirimBasarili ? 1800 : 2500
        repeat: false
        onTriggered: {
            mesafeBildirimGoster = false
            if (mesafeBildirimBasarili) {
                seciliSensor = -1
            }
        }
    }

    function egimSifirlaVeBildir() {
        var biasX = sensorManager.hamAccelX
        var biasY = sensorManager.hamAccelY
        var basarili = database.egimKalibrasyonuKaydet(biasX, biasY, 0.0, 1.0, 1.0, 1.0)
        if (basarili) {
            wifiManager.kalibrasyonYenidenYukle()
        }
        egimBildirimBasarili = basarili
        egimBildirimMesaj = basarili ? txt("kalibrasyonKaydedildiMesaj") : txt("kalibrasyonKaydedilemediMesaj")
        egimBildirimGoster = true
        egimBildirimTimer.restart()
    }

    Timer {
        id: egimBildirimTimer
        interval: egimBildirimBasarili ? 1800 : 2500
        repeat: false
        onTriggered: {
            egimBildirimGoster = false
            if (egimBildirimBasarili) {
                seciliSensor = -1
            }
        }
    }

    // Sensör kartlarının "Kalibre Edildi" rozeti ve tarihi kalıcı depodan (SQLite)
    // okunur; sadece o an kalibrasyon yapılınca değil, sayfa her açıldığında yeniden
    // yüklenmeli ki uygulama yeniden başlatıldığında da rozet doğru görünsün.
    function kalibrasyonDurumunuYukle() {
        var egimKal = database.egimKalibrasyonuGetir()
        egimKayitliVar = egimKal.mevcut
        egimSonTarih = egimKal.mevcut ? egimKal.tarih : ""

        var lcNoktalar = database.loadCellNoktalariGetir()
        loadCellKayitliVar = lcNoktalar.length > 0
        loadCellNoktaSayisi = lcNoktalar.length
        loadCellSonTarih = loadCellKayitliVar ? database.kalibrasyonTarihiGetir("loadcell") : ""

        var mNoktalar = database.mesafeNoktalariGetir()
        mesafeOlcumKayitliVar = mNoktalar.length > 0
        mesafeOlcumNoktaSayisi = mNoktalar.length
        mesafeSonTarih = mesafeOlcumKayitliVar ? database.kalibrasyonTarihiGetir("mesafe_olcum") : ""
    }

    Component.onCompleted: kalibrasyonDurumunuYukle()

    onVisibleChanged: {
        if (visible) {
            kalibrasyonDurumunuYukle()
        }
    }

    onSeciliSensorChanged: {
        if (seciliSensor === 0) {
            var egimKal = database.egimKalibrasyonuGetir()
            egimKayitliVar = egimKal.mevcut
```

```
            egimSonTarih = egimKal.mevcut ? egimKal.tarih : ""
            egimBildirimGoster = false
        } else if (seciliSensor === 1) {
            var noktalar = database.loadCellNoktalariGetir()
            loadCellKayitliVar = noktalar.length > 0
            loadCellNoktaSayisi = noktalar.length
            loadCellSonTarih = loadCellKayitliVar ? database.kalibrasyonTarihiGetir("loadcell") : ""
            loadCellNoktalari = []
            loadCellSayiSecildi = false
            loadCellDuzenlemeIndex = -1
            loadCellTamDuzenlemeModu = false
            loadCellGoruntuleModu = false
            loadCellYenidenOnayGoster = false
            loadCellTamamlandiOnayGoster = false
            loadCellBildirimGoster = false
            loadCellAraEkranGoster = loadCellKayitliVar
        } else if (seciliSensor === 2) {
            var mNoktalar = database.mesafeNoktalariGetir()
            mesafeOlcumKayitliVar = mNoktalar.length > 0
            mesafeOlcumNoktaSayisi = mNoktalar.length
            mesafeSonTarih = mesafeOlcumKayitliVar ? database.kalibrasyonTarihiGetir("mesafe_olcum") : ""
            mesafeNoktalari = []
            mesafeSayiSecildi = false
            mesafeDuzenlemeIndex = -1
            mesafeTamDuzenlemeModu = false
            mesafeGoruntuleModu = false
            mesafeYenidenOnayGoster = false
            mesafeTamamlandiOnayGoster = false
            mesafeBildirimGoster = false
            mesafeAraEkranGoster = mesafeOlcumKayitliVar
        }
    }

    StackLayout {
        anchors.fill: parent
        currentIndex: seciliSensor === -1 ? 0 : 1

        Item {
            Text {
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.margins: 20
                text: txt("baslikSensorSec")
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 12
                font.bold: true
                font.letterSpacing: 1
            }

            Row {
                anchors.centerIn: parent
                spacing: 24

                Rectangle {
                    id: egimKarti
                    width: 350
                    height: 450
                    radius: 12
                    color: "#0f1420"
                    border.color: egimKarti.hovered ? "#14b8a6" : "#1e2a3f"
                    border.width: 1
                    property bool hovered: false
                    scale: hovered ? 1.03 : 1.0
                    Behavior on border.color { ColorAnimation { duration: 120 } }
                    Behavior on scale { NumberAnimation { duration: 120; easing.type: Easing.OutQuad } }

                    Column {
                        anchors.centerIn: parent
                        spacing: 14

                        Item {
                            anchors.horizontalCenter: parent.horizontalCenter
                            width: 210
                            height: 150

                            Image {
                                anchors.centerIn: parent
                                width: parent.width
                                height: parent.height
```

```
                source: "../assets/egim_sensor.png"
                fillMode: Image.PreserveAspectFit
            }
        }

        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("egimSensoru"); color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 24; font.bold: true }
        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("egimAciklama"); color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 14 }

        Rectangle {
            anchors.horizontalCenter: parent.horizontalCenter
            visible: egimKayitliVar
            width: kaliMetni1.implicitWidth + 16
            height: 20
            radius: 10
            color: "#123321"

            Text {
                id: kaliMetni1
                anchors.centerIn: parent
                text: txt("kalibreEdildiOnEk") + egimSonTarih
                color: "#4ade80"
                font.pixelSize: 10
                font.bold: true
            }
        }
    }

    MouseArea {
        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onEntered: egimKarti.hovered = true
        onExited: egimKarti.hovered = false
        onClicked: seciliSensor = 0
    }
}

Rectangle {
    id: loadCellKarti
    width: 350
    height: 450
    radius: 12
    color: "#0f1420"
    border.color: loadCellKarti.hovered ? "#3b82f6" : "#1e2a3f"
    border.width: 1
    property bool hovered: false
    scale: hovered ? 1.03 : 1.0
    Behavior on border.color { ColorAnimation { duration: 120 } }
    Behavior on scale { NumberAnimation { duration: 120; easing.type: Easing.OutQuad } }

    Column {
        anchors.centerIn: parent
        spacing: 14

        Item {
            anchors.horizontalCenter: parent.horizontalCenter
            width: 210
            height: 150

            Image {
                anchors.centerIn: parent
                width: parent.width
                height: parent.height
                source: "../assets/loadcell.png"
                fillMode: Image.PreserveAspectFit
            }
        }

        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("loadCellAdi"); color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 24; font.bold: true }
        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("loadCellAciklama"); color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 14 }

        Rectangle {
            anchors.horizontalCenter: parent.horizontalCenter
            visible: loadCellKayitliVar
            width: kaliMetni2.implicitWidth + 16
            height: 20
```

```
                                radius: 10
                                color: "#123321"

                                Text {
                                    id: kaliMetni2
                                    anchors.centerIn: parent
                                    text: txt("kalibreEdildiOnEk") + loadCellSonTarih
                                    color: "#4ade80"
                                    font.pixelSize: 10
                                    font.bold: true
                                }
                            }
                        }

                        MouseArea {
                            anchors.fill: parent
                            hoverEnabled: true
                            cursorShape: Qt.PointingHandCursor
                            onEntered: loadCellKarti.hovered = true
                            onExited: loadCellKarti.hovered = false
                            onClicked: seciliSensor = 1
                        }
                    }

                    Rectangle {
                        id: mesafeKarti
                        width: 350
                        height: 450
                        radius: 12
                        color: "#0f1420"
                        border.color: mesafeKarti.hovered ? "#9333ea" : "#1e2a3f"
                        border.width: 1
                        property bool hovered: false
                        scale: hovered ? 1.03 : 1.0
                        Behavior on border.color { ColorAnimation { duration: 120 } }
                        Behavior on scale { NumberAnimation { duration: 120; easing.type: Easing.OutQuad } }

                        Column {
                            anchors.centerIn: parent
                            spacing: 14

                            Item {
                                anchors.horizontalCenter: parent.horizontalCenter
                                width: 210
                                height: 150

                                Image {
                                    anchors.centerIn: parent
                                    width: parent.width
                                    height: parent.height
                                    source: "../assets/mesafe_sensor.png"
                                    fillMode: Image.PreserveAspectFit
                                }
                            }

                            Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("mesafeSensoru"); color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 24; font.bold: true }
                            Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("mesafeAciklama"); color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 14 }

                            Rectangle {
                                anchors.horizontalCenter: parent.horizontalCenter
                                visible: mesafeOlcumKayitliVar
                                width: kaliMetni3.implicitWidth + 16
                                height: 20
                                radius: 10
                                color: "#123321"

                                Text {
                                    id: kaliMetni3
                                    anchors.centerIn: parent
                                    text: txt("kalibreEdildiOnEk") + mesafeSonTarih
                                    color: "#4ade80"
                                    font.pixelSize: 10
                                    font.bold: true
                                }
                            }
                        }

                        MouseArea {
```

```
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    onEntered: mesafeKarti.hovered = true
                    onExited: mesafeKarti.hovered = false
                    onClicked: seciliSensor = 2
                }
            }
        }
    }

    Item {
        Row {
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.margins: 20
            spacing: 12

            Button {
                text: txt("geriButon")
                font.pixelSize: 13

                background: Rectangle {
                    radius: 6
                    color: parent.hovered ? "#1e2a3f" : "#0f1420"
                    border.color: "#1e2a3f"
                    border.width: 1
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#dce8f5"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                    leftPadding: 10
                    rightPadding: 10
                }

                onClicked: seciliSensor = -1
            }

            Rectangle {
                width: 8
                height: 8
                radius: 4
                anchors.verticalCenter: parent.verticalCenter
                color: {
                    if (seciliSensor === 0) return "#14b8a6"
                    if (seciliSensor === 1) return "#3b82f6"
                    if (seciliSensor === 2) return "#9333ea"
                    return "#6b7280"
                }
            }

            Text {
                anchors.verticalCenter: parent.verticalCenter
                text: {
                    if (seciliSensor === 0) return txt("egimBaslik")
                    if (seciliSensor === 1) return txt("loadCellBaslik")
                    if (seciliSensor === 2) return txt("mesafeBaslik")
                    return ""
                }
                color: "#dce8f5"
                font.family: "Segoe UI"
                font.pixelSize: 16
                font.bold: true
            }
        }

        Item {
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            anchors.topMargin: 70
            visible: seciliSensor === 0

            Column {
```

```
                        anchors.centerIn: parent
                        spacing: 20
                        width: 360

                        Rectangle {
                            anchors.horizontalCenter: parent.horizontalCenter
                            visible: egimKayitliVar
                            width: egimSonKaliMetni.implicitWidth + 20
                            height: 26
                            radius: 13
                            color: "#0f1420"
                            border.color: "#1e2a3f"
                            border.width: 1

                            Text {
                                id: egimSonKaliMetni
                                anchors.centerIn: parent
                                text: txt("sonKalibrasyonOnEk") + egimSonTarih
                                color: "#6b7280"
                                font.pixelSize: 11
                            }
                        }

                        Text {
                            width: parent.width
                            horizontalAlignment: Text.AlignHCenter
                            wrapMode: Text.WordWrap
                            text: txt("cihaziYerlestir")
                            color: "#dce8f5"
                            font.family: "Segoe UI"
                            font.pixelSize: 14
                        }

                        SuTerazisi {
                            anchors.horizontalCenter: parent.horizontalCenter
                            egimX: sensorManager.egimX
                            egimY: sensorManager.egimY
                        }

                        Rectangle {
                            anchors.horizontalCenter: parent.horizontalCenter
                            width: 220
                            height: 72
                            radius: 12
                            color: "#0f2e2a"
                            border.color: "#14b8a6"
                            border.width: 1

                            Text {
                                anchors.centerIn: parent
                                text: "X: " + sensorManager.egimX.toFixed(1) + "°   Y: " + sensorManager.egimY.toFixed(1) + "°"
                                color: "#2dd4bf"
                                font.family: "Segoe UI"
                                font.pixelSize: 22
                                font.bold: true
                            }
                        }

                        Text {
                            width: parent.width
                            horizontalAlignment: Text.AlignHCenter
                            wrapMode: Text.WordWrap
                            visible: !wifiManager.baglandi
                            text: txt("cihazBagliDegilKalibrasyon")
                            color: "#f87171"
                            font.family: "Segoe UI"
                            font.pixelSize: 11
                        }

                        Button {
                            width: parent.width
                            height: 44
                            text: txt("sifirlaButon")
                            font.pixelSize: 14
                            font.bold: true
                            enabled: wifiManager.baglandi

                            onClicked: egimSifirlaVeBildir()

                            background: Rectangle {
```

```
                        radius: 8
                        color: parent.enabled ? (parent.hovered ? "#2dd4bf" : "#14b8a6") : "#1e2a3f"
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#ffffff"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }

            Rectangle {
                id: egimBildirimKutusu
                anchors.top: parent.top
                anchors.horizontalCenter: parent.horizontalCenter
                anchors.topMargin: 12
                z: 100
                width: egimBildirimMetni.implicitWidth + 24
                height: 32
                radius: 16
                color: egimBildirimBasarili ? "#123321" : "#3a1414"
                opacity: egimBildirimGoster ? 1 : 0
                visible: opacity > 0
                Behavior on opacity { NumberAnimation { duration: 200 } }

                Text {
                    id: egimBildirimMetni
                    anchors.centerIn: parent
                    text: egimBildirimMesaj
                    color: egimBildirimBasarili ? "#4ade80" : "#f87171"
                    font.family: "Segoe UI"
                    font.pixelSize: 12
                    font.bold: true
                }
            }
        }

        Item {
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            anchors.topMargin: 70
            visible: seciliSensor === 1

            Column {
                anchors.centerIn: parent
                spacing: 24
                width: 420
                visible: loadCellAraEkranGoster

                Rectangle {
                    anchors.horizontalCenter: parent.horizontalCenter
                    width: araEkranBadge.implicitWidth + 20
                    height: 26
                    radius: 13
                    color: "#0f1420"
                    border.color: "#1e2a3f"
                    border.width: 1

                    Text {
                        id: araEkranBadge
                        anchors.centerIn: parent
                        text: txt("kayitliOnEk") + loadCellNoktaSayisi + txt("noktaSonEk")
                        color: "#6b7280"
                        font.pixelSize: 11
                    }
                }

                Text {
                    width: parent.width
                    horizontalAlignment: Text.AlignHCenter
                    wrapMode: Text.WordWrap
                    text: txt("kayitliKalibrasyonBulundu")
                    color: "#dce8f5"
                    font.family: "Segoe UI"
```

```
                    font.pixelSize: 16
                    font.bold: true
                }

                Row {
                    anchors.horizontalCenter: parent.horizontalCenter
                    spacing: 20

                    Rectangle {
                        id: goruntuleKarti
                        width: 190
                        height: 180
                        radius: 14
                        color: "#0f1420"
                        border.color: goruntuleKarti.hovered ? "#3b82f6" : "#1e2a3f"
                        border.width: 1
                        property bool hovered: false
                        Behavior on border.color { ColorAnimation { duration: 120 } }

                        Column {
                            anchors.centerIn: parent
                            spacing: 12
                            width: parent.width - 28

                            Canvas {
                                anchors.horizontalCenter: parent.horizontalCenter
                                width: 32
                                height: 32
                                onPaint: {
                                    var ctx = getContext("2d")
                                    ctx.clearRect(0, 0, width, height)
                                    ctx.lineWidth = 1.8
                                    ctx.lineCap = "round"
                                    ctx.lineJoin = "round"
                                    ctx.strokeStyle = "#3b82f6"

                                    ctx.beginPath()
                                    ctx.moveTo(3, 16)
                                    ctx.bezierCurveTo(7.5, 8.5, 12.5, 6.8, 16, 6.8)
                                    ctx.bezierCurveTo(19.5, 6.8, 24.5, 8.5, 29, 16)
                                    ctx.bezierCurveTo(24.5, 23.5, 19.5, 25.2, 16, 25.2)
                                    ctx.bezierCurveTo(12.5, 25.2, 7.5, 23.5, 3, 16)
                                    ctx.stroke()

                                    ctx.beginPath()
                                    ctx.arc(16, 16, 4.4, 0, Math.PI * 2, false)
                                    ctx.stroke()
                                }
                            }

                            Text {
                                width: parent.width
                                horizontalAlignment: Text.AlignHCenter
                                wrapMode: Text.WordWrap
                                text: txt("mevcutGoruntule")
                                color: "#dce8f5"
                                font.family: "Segoe UI"
                                font.pixelSize: 13
                                font.bold: true
                            }

                            Text {
                                width: parent.width
                                horizontalAlignment: Text.AlignHCenter
                                wrapMode: Text.WordWrap
                                text: txt("kayitliNoktalariIncele")
                                color: "#6b7280"
                                font.family: "Segoe UI"
                                font.pixelSize: 11
                            }
                        }

                        MouseArea {
                            anchors.fill: parent
                            hoverEnabled: true
                            cursorShape: Qt.PointingHandCursor
                            onEntered: goruntuleKarti.hovered = true
                            onExited: goruntuleKarti.hovered = false
                            onClicked: {
                                loadCellNoktalari = database.loadCellNoktalariGetir()
```

```
            loadCellAraEkranGoster = false
            loadCellGoruntuleModu = true
            loadCellDuzenlemeIndex = -1
        }
    }
}

Rectangle {
    id: yenidenKarti
    width: 190
    height: 180
    radius: 14
    color: "#0f1420"
    border.color: yenidenKarti.hovered ? "#9ca3af" : "#1e2a3f"
    border.width: 1
    property bool hovered: false
    Behavior on border.color { ColorAnimation { duration: 120 } }

    Column {
        anchors.centerIn: parent
        spacing: 12
        width: parent.width - 28

        Canvas {
            anchors.horizontalCenter: parent.horizontalCenter
            width: 32
            height: 32
            onPaint: {
                var ctx = getContext("2d")
                ctx.clearRect(0, 0, width, height)
                ctx.strokeStyle = "#9ca3af"
                ctx.fillStyle = "#9ca3af"
                ctx.lineWidth = 3.4
                ctx.lineCap = "butt"

                function yayOku(startDeg, endDeg) {
                    var s = startDeg * Math.PI / 180
                    var e = endDeg * Math.PI / 180
                    var cx = 16, cy = 16, r = 10.5

                    ctx.beginPath()
                    ctx.arc(cx, cy, r, s, e, false)
                    ctx.stroke()

                    var ex = cx + r * Math.cos(e)
                    var ey = cy + r * Math.sin(e)
                    var tx = -Math.sin(e)
                    var ty = Math.cos(e)
                    var px = -ty
                    var py = tx
                    var geri = 6.2
                    var yaricap = 3.6
                    var bx = ex - tx * geri
                    var by = ey - ty * geri

                    ctx.beginPath()
                    ctx.moveTo(ex, ey)
                    ctx.lineTo(bx + px * yaricap, by + py * yaricap)
                    ctx.lineTo(bx - px * yaricap, by - py * yaricap)
                    ctx.closePath()
                    ctx.fill()
                }

                yayOku(195, 345)
                yayOku(15, 165)
            }
        }

        Text {
            width: parent.width
            horizontalAlignment: Text.AlignHCenter
            wrapMode: Text.WordWrap
            text: txt("yenidenKalibreEt")
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 13
            font.bold: true
        }

        Text {
```

```
                                        width: parent.width
                                        horizontalAlignment: Text.AlignHCenter
                                        wrapMode: Text.WordWrap
                                        text: txt("sifirdanYeniOlcum")
                                        color: "#6b7280"
                                        font.family: "Segoe UI"
                                        font.pixelSize: 11
                                    }
                                }

                                MouseArea {
                                    anchors.fill: parent
                                    hoverEnabled: true
                                    cursorShape: Qt.PointingHandCursor
                                    onEntered: yenidenKarti.hovered = true
                                    onExited: yenidenKarti.hovered = false
                                    onClicked: loadCellYenidenOnayGoster = true
                                }
                            }
                        }
                    }

                    Column {
                        anchors.centerIn: parent
                        spacing: 16
                        width: 340
                        visible: loadCellGoruntuleModu

                        Text {
                            text: txt("kayitliNoktalarBaslik")
                            color: "#6b7280"
                            font.family: "Segoe UI"
                            font.pixelSize: 11
                            font.bold: true
                            font.letterSpacing: 1
                        }

                        Rectangle {
                            width: parent.width
                            height: 320
                            radius: 14
                            color: "#0f1420"
                            border.color: "#1e2a3f"
                            border.width: 1

                            ListView {
                                anchors.fill: parent
                                anchors.margins: 16
                                clip: true
                                spacing: 8

                                model: loadCellNoktalari

                                delegate: Rectangle {
                                    width: ListView.view.width
                                    height: 48
                                    radius: 8
                                    color: "#0a0e17"
                                    border.color: "#1e2a3f"
                                    border.width: 1

                                    Row {
                                        anchors.fill: parent
                                        anchors.leftMargin: 12
                                        anchors.rightMargin: 12
                                        spacing: 8

                                        Rectangle {
                                            width: 22
                                            height: 22
                                            radius: 11
                                            anchors.verticalCenter: parent.verticalCenter
                                            color: "#182644"
                                            border.color: "#3b82f6"
                                            border.width: 1

                                            Text {
                                                anchors.centerIn: parent
                                                text: index + 1
                                                color: "#3b82f6"
```

```
                                font.pixelSize: 10
                                font.bold: true
                            }
                        }

                        Column {
                            anchors.verticalCenter: parent.verticalCenter
                            spacing: 1

                            Text {
                                text: modelData.hedefKg.toFixed(2) + " kg"
                                color: "#dce8f5"
                                font.family: "Segoe UI"
                                font.pixelSize: 13
                                font.bold: true
                            }

                            Text {
                                text: "ADC: " + modelData.hamDeger.toFixed(0)
                                color: "#6b7280"
                                font.family: "Segoe UI"
                                font.pixelSize: 10
                            }
                        }
                    }
                }
            }
        }

        Button {
            width: parent.width
            height: 46
            text: txt("duzenleButon")
            font.pixelSize: 14
            font.bold: true

            onClicked: {
                loadCellToplamNokta = loadCellNoktalari.length
                loadCellGoruntuleModu = false
                loadCellTamDuzenlemeModu = true
                loadCellSayiSecildi = true
                loadCellDuzenlemeIndex = -1
            }

            background: Rectangle {
                radius: 8
                color: parent.hovered ? "#4f8cf7" : "#3b82f6"
                Behavior on color { ColorAnimation { duration: 120 } }
            }

            contentItem: Text {
                text: parent.text
                color: "#ffffff"
                font: parent.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }
    }

    Rectangle {
        anchors.fill: parent
        color: "#000000b3"
        visible: loadCellYenidenOnayGoster
        z: 200

        MouseArea { anchors.fill: parent }

        Rectangle {
            anchors.centerIn: parent
            width: 360
            height: 200
            radius: 14
            color: "#0f1420"
            border.color: "#1e2a3f"
            border.width: 1

            Column {
                anchors.centerIn: parent
                spacing: 18
```

```
                        width: parent.width - 40

                        Text {
                            width: parent.width
                            horizontalAlignment: Text.AlignHCenter
                            wrapMode: Text.WordWrap
                            text: txt("silinecekOnay")
                            color: "#dce8f5"
                            font.family: "Segoe UI"
                            font.pixelSize: 14
                            font.bold: true
                        }

                        Row {
                            anchors.horizontalCenter: parent.horizontalCenter
                            spacing: 12

                            Button {
                                width: 150
                                height: 42
                                text: txt("vazgecButon")

                                onClicked: loadCellYenidenOnayGoster = false

                                background: Rectangle {
                                    radius: 8
                                    color: parent.hovered ? "#1e2a3f" : "transparent"
                                    border.color: "#1e2a3f"
                                    border.width: 1
                                    Behavior on color { ColorAnimation { duration: 120 } }
                                }

                                contentItem: Text {
                                    text: parent.text
                                    color: "#9ca3af"
                                    font: parent.font
                                    horizontalAlignment: Text.AlignHCenter
                                    verticalAlignment: Text.AlignVCenter
                                }
                            }

                            Button {
                                width: 150
                                height: 42
                                text: txt("evetYenidenBaslaButon")
                                font.bold: true

                                onClicked: {
                                    loadCellYenidenOnayGoster = false
                                    loadCellAraEkranGoster = false
                                    loadCellGoruntuleModu = false
                                    loadCellTamDuzenlemeModu = false
                                    loadCellNoktalari = []
                                    loadCellSayiSecildi = false
                                    loadCellDuzenlemeIndex = -1
                                }

                                background: Rectangle {
                                    radius: 8
                                    color: parent.hovered ? "#b91c1c" : "#991b1b"
                                    Behavior on color { ColorAnimation { duration: 120 } }
                                }

                                contentItem: Text {
                                    text: parent.text
                                    color: "#ffffff"
                                    font: parent.font
                                    horizontalAlignment: Text.AlignHCenter
                                    verticalAlignment: Text.AlignVCenter
                                }
                            }
                        }
                    }
                }
            }

            Rectangle {
                anchors.fill: parent
                color: "#000000b3"
                visible: loadCellTamamlandiOnayGoster
```

```
z: 200

MouseArea { anchors.fill: parent }

Rectangle {
    anchors.centerIn: parent
    width: 380
    height: 220
    radius: 14
    color: "#0f1420"
    border.color: "#16a34a"
    border.width: 1

    Column {
        anchors.centerIn: parent
        spacing: 18
        width: parent.width - 40

        Column {
            width: parent.width
            spacing: 6

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                text: txt("kalibrasyonTamamlandiBaslik")
                color: "#4ade80"
                font.family: "Segoe UI"
                font.pixelSize: 17
                font.bold: true
            }

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                wrapMode: Text.WordWrap
                text: txt("toplamOnEk") + loadCellToplamNokta + txt("noktaGirildiSonEk")
                color: "#9ca3af"
                font.family: "Segoe UI"
                font.pixelSize: 13
            }
        }

        Row {
            anchors.horizontalCenter: parent.horizontalCenter
            spacing: 12

            Button {
                width: 150
                height: 42
                text: txt("noktalariKontrolEt")

                onClicked: loadCellTamamlandiOnayGoster = false

                background: Rectangle {
                    radius: 8
                    color: parent.hovered ? "#1e2a3f" : "transparent"
                    border.color: "#1e2a3f"
                    border.width: 1
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#9ca3af"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Button {
                width: 150
                height: 42
                text: txt("kaydetVeBitir")
                font.bold: true

                onClicked: {
                    loadCellTamamlandiOnayGoster = false
                    loadCellKaydetVeBildir()
```

```
                            }

                            background: Rectangle {
                                radius: 8
                                color: parent.hovered ? "#22c55e" : "#16a34a"
                                Behavior on color { ColorAnimation { duration: 120 } }
                            }

                            contentItem: Text {
                                text: parent.text
                                color: "#ffffff"
                                font: parent.font
                                horizontalAlignment: Text.AlignHCenter
                                verticalAlignment: Text.AlignVCenter
                            }
                        }
                    }
                }
            }
        }

        Rectangle {
            id: loadCellBildirimKutusu
            anchors.top: parent.top
            anchors.horizontalCenter: parent.horizontalCenter
            anchors.topMargin: 12
            z: 100
            width: loadCellBildirimMetni.implicitWidth + 24
            height: 32
            radius: 16
            color: loadCellBildirimBasarili ? "#123321" : "#3a1414"
            opacity: loadCellBildirimGoster ? 1 : 0
            visible: opacity > 0
            Behavior on opacity { NumberAnimation { duration: 200 } }

            Text {
                id: loadCellBildirimMetni
                anchors.centerIn: parent
                text: loadCellBildirimMesaj
                color: loadCellBildirimBasarili ? "#4ade80" : "#f87171"
                font.family: "Segoe UI"
                font.pixelSize: 12
                font.bold: true
            }
        }

        Column {
            anchors.centerIn: parent
            spacing: 24
            width: 360
            visible: !loadCellSayiSecildi && !loadCellAraEkranGoster && !loadCellGoruntuleModu

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                wrapMode: Text.WordWrap
                text: txt("kacNoktaSoru")
                color: "#dce8f5"
                font.family: "Segoe UI"
                font.pixelSize: 16
                font.bold: true
            }

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                wrapMode: Text.WordWrap
                text: txt("dahaFazlaNokta")
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 12
            }

            Grid {
                anchors.horizontalCenter: parent.horizontalCenter
                columns: 4
                spacing: 12

                Repeater {
                    model: [4, 6, 8, 10]
```

```
            Rectangle {
                id: secKarti
                width: 72
                height: 72
                radius: 12
                color: secKarti.hovered ? "#182644" : "#0f1420"
                border.color: "#3b82f6"
                border.width: 1
                property bool hovered: false
                Behavior on color { ColorAnimation { duration: 120 } }

                Text {
                    anchors.centerIn: parent
                    text: modelData
                    color: "#dce8f5"
                    font.family: "Segoe UI"
                    font.pixelSize: 26
                    font.bold: true
                }

                MouseArea {
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    onEntered: secKarti.hovered = true
                    onExited: secKarti.hovered = false
                    onClicked: {
                        loadCellToplamNokta = modelData
                        loadCellNoktalari = []
                        loadCellDuzenlemeIndex = -1
                        loadCellSayiSecildi = true
                    }
                }
            }
        }
    }
}

Row {
    anchors.centerIn: parent
    spacing: 24
    visible: loadCellSayiSecildi

    Rectangle {
        width: 340
        height: 500
        radius: 14
        color: "#0f1420"
        border.color: "#1e2a3f"
        border.width: 1

        Column {
            anchors.centerIn: parent
            spacing: 16
            width: parent.width - 40

            Rectangle {
                anchors.horizontalCenter: parent.horizontalCenter
                visible: loadCellKayitliVar && !loadCellTamDuzenlemeModu
                width: sonKaliMetni2.implicitWidth + 20
                height: 24
                radius: 12
                color: "#0a0e17"
                border.color: "#1e2a3f"
                border.width: 1

                Text {
                    id: sonKaliMetni2
                    anchors.centerIn: parent
                    text: txt("kayitliOnEk") + loadCellNoktaSayisi + txt("noktaSonEk")
                    color: "#6b7280"
                    font.pixelSize: 11
                }
            }

            Text {
                anchors.horizontalCenter: parent.horizontalCenter
                text: loadCellTamDuzenlemeModu
                      ? txt("kalibrasyonuDuzenle")
```

```
                            : txt("noktaOnEk") + (loadCellNoktalari.length + 1) + " / " + loadCellToplamNokta
                    color: "#3b82f6"
                    font.family: "Segoe UI"
                    font.pixelSize: 13
                    font.bold: true
                    font.letterSpacing: 1
                }

                Rectangle {
                    width: parent.width
                    height: 6
                    radius: 3
                    color: "#1e2a3f"

                    Rectangle {
                        width: parent.width * (loadCellNoktalari.length / loadCellToplamNokta)
                        height: parent.height
                        radius: 3
                        color: "#3b82f6"
                        Behavior on width { NumberAnimation { duration: 150 } }
                    }
                }

                Text {
                    width: parent.width
                    horizontalAlignment: Text.AlignHCenter
                    wrapMode: Text.WordWrap
                    text: loadCellDuzenlemeIndex >= 0
                          ? txt("noktaOnEk") + (loadCellDuzenlemeIndex + 1) + txt("duzenleniyorSonEk")
                          : (loadCellTamDuzenlemeModu
                             ? txt("duzenlemekIcinTikla")
                             : txt("hedefAgirlikYonerge"))
                    color: loadCellDuzenlemeIndex >= 0 ? "#3b82f6" : "#9ca3af"
                    font.family: "Segoe UI"
                    font.bold: loadCellDuzenlemeIndex >= 0
                    font.pixelSize: 13
                }

                TextField {
                    id: hedefKgKutusu
                    anchors.horizontalCenter: parent.horizontalCenter
                    width: 180
                    height: 44
                    placeholderText: "kg"
                    placeholderTextColor: "#4b5563"
                    color: "#ffffff"
                    font.pixelSize: 16
                    horizontalAlignment: TextInput.AlignHCenter
                    validator: DoubleValidator { bottom: 0; top: 500; decimals: 2 }
                    background: Rectangle {
                        color: "#0a0e17"
                        radius: 10
                        border.color: hedefKgKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                        border.width: 1
                    }
                }

                Rectangle {
                    anchors.horizontalCenter: parent.horizontalCenter
                    width: parent.width
                    height: 64
                    radius: 12
                    color: "#132335"
                    border.color: wifiManager.baglandi ? "#3b82f6" : "#dc2626"
                    border.width: 1
                    Behavior on border.color { ColorAnimation { duration: 120 } }

                    Text {
                        anchors.centerIn: parent
                        text: txt("hamDegerOnEk") + sensorManager.hamAgirlik.toFixed(0)
                        color: wifiManager.baglandi ? "#3b82f6" : "#f87171"
                        font.family: "Segoe UI"
                        font.pixelSize: 20
                        font.bold: true
                    }
                }

                Text {
                    width: parent.width
                    horizontalAlignment: Text.AlignHCenter
```

```
    wrapMode: Text.WordWrap
    visible: !wifiManager.baglandi
    text: txt("hamDegerSifirUyari")
    color: "#f87171"
    font.family: "Segoe UI"
    font.pixelSize: 11
}

Button {
    id: noktaKaydetButonu
    width: parent.width
    height: 46
    property bool tumNoktalarGirildi: !loadCellTamDuzenlemeModu && loadCellDuzenlemeIndex < 0 && loadCellNoktalari.length >= loadCellToplamNokta
    text: loadCellDuzenlemeIndex >= 0
          ? txt("noktayiGuncelle")
          : (tumNoktalarGirildi
             ? txt("kalibrasyonuTamamla")
             : (loadCellNoktalari.length >= loadCellToplamNokta - 1 ? txt("sonNoktayiKaydet") : txt("buNoktayiKaydet")))
    font.pixelSize: 14
    font.bold: true
    enabled: tumNoktalarGirildi || (hedefKgKutusu.text.length > 0 && (loadCellDuzenlemeIndex >= 0 || loadCellNoktalari.length < loadCellToplamNokta))

    onClicked: {
        if (tumNoktalarGirildi) {
            loadCellTamamlandiOnayGoster = true
            return
        }

        var yeniNokta = { hedefKg: parseFloat(hedefKgKutusu.text), hamDeger: sensorManager.hamAgirlik }

        if (loadCellDuzenlemeIndex >= 0) {
            // Var olan bir noktayı yerinde güncelliyoruz, diğer noktalar etkilenmiyor
            var guncelDizi = loadCellNoktalari.slice()
            guncelDizi[loadCellDuzenlemeIndex] = yeniNokta
            loadCellNoktalari = guncelDizi
            loadCellDuzenlemeIndex = -1
            hedefKgKutusu.text = ""
            return
        }

        var yeniDizi = loadCellNoktalari.slice()
        yeniDizi.push(yeniNokta)
        loadCellNoktalari = yeniDizi
        hedefKgKutusu.text = ""

        if (loadCellNoktalari.length >= loadCellToplamNokta) {
            loadCellTamamlandiOnayGoster = true
        }
    }

    background: Rectangle {
        radius: 10
        color: parent.enabled ? (parent.hovered ? "#4f8cf7" : "#3b82f6") : "#1e2a3f"
        Behavior on color { ColorAnimation { duration: 120 } }
    }

    contentItem: Text {
        text: parent.text
        color: "#ffffff"
        font: parent.font
        horizontalAlignment: Text.AlignHCenter
        verticalAlignment: Text.AlignVCenter
    }
}

Button {
    width: parent.width
    height: 46
    visible: loadCellTamDuzenlemeModu
    text: txt("degisiklikleriKaydet")
    font.pixelSize: 14
    font.bold: true

    onClicked: loadCellKaydetVeBildir()

    background: Rectangle {
        radius: 10
```

```
                color: parent.hovered ? "#22c55e" : "#16a34a"
                Behavior on color { ColorAnimation { duration: 120 } }
            }

            contentItem: Text {
                text: parent.text
                color: "#ffffff"
                font: parent.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }

        Button {
            width: parent.width
            height: 38
            visible: loadCellDuzenlemeIndex >= 0
            text: txt("vazgecButon")
            font.pixelSize: 13

            onClicked: {
                loadCellDuzenlemeIndex = -1
                hedefKgKutusu.text = ""
            }

            background: Rectangle {
                radius: 8
                color: parent.hovered ? "#1e2a3f" : "transparent"
                border.color: "#1e2a3f"
                border.width: 1
                Behavior on color { ColorAnimation { duration: 120 } }
            }

            contentItem: Text {
                text: parent.text
                color: "#9ca3af"
                font: parent.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }
    }
}

Rectangle {
    width: 300
    height: 500
    radius: 14
    color: "#0f1420"
    border.color: "#1e2a3f"
    border.width: 1

    Column {
        anchors.top: parent.top
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.margins: 20
        spacing: 4

        Text {
            text: txt("kaydedilenNoktalarBaslik")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 11
            font.bold: true
            font.letterSpacing: 1
        }

        Text {
            text: txt("duzeltmekIcinTikla")
            color: "#4b5563"
            font.family: "Segoe UI"
            font.pixelSize: 10
        }
    }

    ListView {
        anchors.top: parent.top
        anchors.left: parent.left
        anchors.right: parent.right
```

```
anchors.bottom: parent.bottom
anchors.margins: 16
anchors.topMargin: 60
clip: true
spacing: 8

model: loadCellNoktalari

delegate: Rectangle {
    width: ListView.view.width
    height: 48
    radius: 8
    color: satirAlani.containsMouse ? "#182644" : "#0a0e17"
    border.color: index === loadCellDuzenlemeIndex ? "#3b82f6" : "#1e2a3f"
    border.width: index === loadCellDuzenlemeIndex ? 2 : 1
    Behavior on color { ColorAnimation { duration: 120 } }

    Row {
        anchors.fill: parent
        anchors.leftMargin: 12
        anchors.rightMargin: 12
        spacing: 8

        Rectangle {
            width: 22
            height: 22
            radius: 11
            anchors.verticalCenter: parent.verticalCenter
            color: "#182644"
            border.color: "#3b82f6"
            border.width: 1

            Text {
                anchors.centerIn: parent
                text: index + 1
                color: "#3b82f6"
                font.pixelSize: 10
                font.bold: true
            }
        }

        Column {
            anchors.verticalCenter: parent.verticalCenter
            spacing: 1

            Text {
                text: modelData.hedefKg.toFixed(2) + " kg"
                color: "#dce8f5"
                font.family: "Segoe UI"
                font.pixelSize: 13
                font.bold: true
            }

            Text {
                text: "ADC: " + modelData.hamDeger.toFixed(0)
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 10
            }
        }
    }

    Text {
        anchors.right: parent.right
        anchors.verticalCenter: parent.verticalCenter
        anchors.rightMargin: 12
        text: "✏️"
        color: "#6b7280"
        font.pixelSize: 13
        visible: satirAlani.containsMouse
    }

    MouseArea {
        id: satirAlani
        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onClicked: {
            hedefKgKutusu.text = modelData.hedefKg.toString()
            loadCellDuzenlemeIndex = index
```

```
                                    }
                                }
                            }
                        }
                    }
                }
            }

            Item {
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.bottom: parent.bottom
                anchors.topMargin: 70
                visible: seciliSensor === 2

                // ============================================================
                // OLCUM KALIBRASYONU — cok nokta (load cell birebir kopyasi)
                // ============================================================
                Item {
                    anchors.fill: parent

                    Column {
                        anchors.centerIn: parent
                        spacing: 24
                        width: 420
                        visible: mesafeAraEkranGoster

                        Rectangle {
                            anchors.horizontalCenter: parent.horizontalCenter
                            width: mesafeAraEkranBadge.implicitWidth + 20
                            height: 26
                            radius: 13
                            color: "#0f1420"
                            border.color: "#1e2a3f"
                            border.width: 1

                            Text {
                                id: mesafeAraEkranBadge
                                anchors.centerIn: parent
                                text: txt("kayitliOnEk") + mesafeOlcumNoktaSayisi + txt("noktaSonEk")
                                color: "#6b7280"
                                font.pixelSize: 11
                            }
                        }

                        Text {
                            width: parent.width
                            horizontalAlignment: Text.AlignHCenter
                            wrapMode: Text.WordWrap
                            text: txt("kayitliKalibrasyonBulundu")
                            color: "#dce8f5"
                            font.family: "Segoe UI"
                            font.pixelSize: 16
                            font.bold: true
                        }

                        Row {
                            anchors.horizontalCenter: parent.horizontalCenter
                            spacing: 20

                            Rectangle {
                                id: mesafeGoruntuleKarti
                                width: 190
                                height: 180
                                radius: 14
                                color: "#0f1420"
                                border.color: mesafeGoruntuleKarti.hovered ? "#9333ea" : "#1e2a3f"
                                border.width: 1
                                property bool hovered: false
                                Behavior on border.color { ColorAnimation { duration: 120 } }

                                Column {
                                    anchors.centerIn: parent
                                    spacing: 12
                                    width: parent.width - 28

                                    Canvas {
                                        anchors.horizontalCenter: parent.horizontalCenter
                                        width: 32
```

```
            height: 32
            onPaint: {
                var ctx = getContext("2d")
                ctx.clearRect(0, 0, width, height)
                ctx.lineWidth = 1.8
                ctx.lineCap = "round"
                ctx.lineJoin = "round"
                ctx.strokeStyle = "#9333ea"

                ctx.beginPath()
                ctx.moveTo(3, 16)
                ctx.bezierCurveTo(7.5, 8.5, 12.5, 6.8, 16, 6.8)
                ctx.bezierCurveTo(19.5, 6.8, 24.5, 8.5, 29, 16)
                ctx.bezierCurveTo(24.5, 23.5, 19.5, 25.2, 16, 25.2)
                ctx.bezierCurveTo(12.5, 25.2, 7.5, 23.5, 3, 16)
                ctx.stroke()

                ctx.beginPath()
                ctx.arc(16, 16, 4.4, 0, Math.PI * 2, false)
                ctx.stroke()
            }
        }

        Text {
            width: parent.width
            horizontalAlignment: Text.AlignHCenter
            wrapMode: Text.WordWrap
            text: txt("mevcutGoruntule")
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 13
            font.bold: true
        }

        Text {
            width: parent.width
            horizontalAlignment: Text.AlignHCenter
            wrapMode: Text.WordWrap
            text: txt("kayitliNoktalariIncele")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 11
        }
    }

    MouseArea {
        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onEntered: mesafeGoruntuleKarti.hovered = true
        onExited: mesafeGoruntuleKarti.hovered = false
        onClicked: {
            mesafeNoktalari = database.mesafeNoktalariGetir()
            mesafeAraEkranGoster = false
            mesafeGoruntuleModu = true
            mesafeDuzenlemeIndex = -1
        }
    }
}

Rectangle {
    id: mesafeYenidenKarti
    width: 190
    height: 180
    radius: 14
    color: "#0f1420"
    border.color: mesafeYenidenKarti.hovered ? "#9ca3af" : "#1e2a3f"
    border.width: 1
    property bool hovered: false
    Behavior on border.color { ColorAnimation { duration: 120 } }

    Column {
        anchors.centerIn: parent
        spacing: 12
        width: parent.width - 28

        Canvas {
            anchors.horizontalCenter: parent.horizontalCenter
            width: 32
            height: 32
```

```
                                        onPaint: {
                                            var ctx = getContext("2d")
                                            ctx.clearRect(0, 0, width, height)
                                            ctx.strokeStyle = "#9ca3af"
                                            ctx.fillStyle = "#9ca3af"
                                            ctx.lineWidth = 3.4
                                            ctx.lineCap = "butt"

                                            function yayOku(startDeg, endDeg) {
                                                var s = startDeg * Math.PI / 180
                                                var e = endDeg * Math.PI / 180
                                                var cx = 16, cy = 16, r = 10.5

                                                ctx.beginPath()
                                                ctx.arc(cx, cy, r, s, e, false)
                                                ctx.stroke()

                                                var ex = cx + r * Math.cos(e)
                                                var ey = cy + r * Math.sin(e)
                                                var tx = -Math.sin(e)
                                                var ty = Math.cos(e)
                                                var px = -ty
                                                var py = tx
                                                var geri = 6.2
                                                var yaricap = 3.6
                                                var bx = ex - tx * geri
                                                var by = ey - ty * geri

                                                ctx.beginPath()
                                                ctx.moveTo(ex, ey)
                                                ctx.lineTo(bx + px * yaricap, by + py * yaricap)
                                                ctx.lineTo(bx - px * yaricap, by - py * yaricap)
                                                ctx.closePath()
                                                ctx.fill()
                                            }

                                            yayOku(195, 345)
                                            yayOku(15, 165)
                                        }
                                    }

                                    Text {
                                        width: parent.width
                                        horizontalAlignment: Text.AlignHCenter
                                        wrapMode: Text.WordWrap
                                        text: txt("yenidenKalibreEt")
                                        color: "#dce8f5"
                                        font.family: "Segoe UI"
                                        font.pixelSize: 13
                                        font.bold: true
                                    }

                                    Text {
                                        width: parent.width
                                        horizontalAlignment: Text.AlignHCenter
                                        wrapMode: Text.WordWrap
                                        text: txt("sifirdanYeniOlcum")
                                        color: "#6b7280"
                                        font.family: "Segoe UI"
                                        font.pixelSize: 11
                                    }
                                }

                                MouseArea {
                                    anchors.fill: parent
                                    hoverEnabled: true
                                    cursorShape: Qt.PointingHandCursor
                                    onEntered: mesafeYenidenKarti.hovered = true
                                    onExited: mesafeYenidenKarti.hovered = false
                                    onClicked: mesafeYenidenOnayGoster = true
                                }
                            }
                        }
                    }

                    Column {
                        anchors.centerIn: parent
                        spacing: 16
                        width: 340
                        visible: mesafeGoruntuleModu
```

```
                        Text {
                            text: txt("kayitliNoktalarBaslik")
                            color: "#6b7280"
                            font.family: "Segoe UI"
                            font.pixelSize: 11
                            font.bold: true
                            font.letterSpacing: 1
                        }

                        Rectangle {
                            width: parent.width
                            height: 320
                            radius: 14
                            color: "#0f1420"
                            border.color: "#1e2a3f"
                            border.width: 1

                            ListView {
                                anchors.fill: parent
                                anchors.margins: 16
                                clip: true
                                spacing: 8

                                model: mesafeNoktalari

                                delegate: Rectangle {
                                    width: ListView.view.width
                                    height: 48
                                    radius: 8
                                    color: "#0a0e17"
                                    border.color: "#1e2a3f"
                                    border.width: 1

                                    Row {
                                        anchors.fill: parent
                                        anchors.leftMargin: 12
                                        anchors.rightMargin: 12
                                        spacing: 8

                                        Rectangle {
                                            width: 22
                                            height: 22
                                            radius: 11
                                            anchors.verticalCenter: parent.verticalCenter
                                            color: "#241a38"
                                            border.color: "#9333ea"
                                            border.width: 1

                                            Text {
                                                anchors.centerIn: parent
                                                text: index + 1
                                                color: "#c084fc"
                                                font.pixelSize: 10
                                                font.bold: true
                                            }
                                        }

                                        Column {
                                            anchors.verticalCenter: parent.verticalCenter
                                            spacing: 1

                                            Text {
                                                text: (modelData.hedefMm / 10).toFixed(1) + " cm"
                                                color: "#dce8f5"
                                                font.family: "Segoe UI"
                                                font.pixelSize: 13
                                                font.bold: true
                                            }

                                            Text {
                                                text: "ADC: " + modelData.hamDeger.toFixed(0)
                                                color: "#6b7280"
                                                font.family: "Segoe UI"
                                                font.pixelSize: 10
                                            }
                                        }
                                    }
                                }
                            }
```

```
        }

        Button {
            width: parent.width
            height: 46
            text: txt("duzenleButon")
            font.pixelSize: 14
            font.bold: true

            onClicked: {
                mesafeToplamNokta = mesafeNoktalari.length
                mesafeGoruntuleModu = false
                mesafeTamDuzenlemeModu = true
                mesafeSayiSecildi = true
                mesafeDuzenlemeIndex = -1
            }

            background: Rectangle {
                radius: 8
                color: parent.hovered ? "#a855f7" : "#9333ea"
                Behavior on color { ColorAnimation { duration: 120 } }
            }

            contentItem: Text {
                text: parent.text
                color: "#ffffff"
                font: parent.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }
    }

    Rectangle {
        anchors.fill: parent
        color: "#000000b3"
        visible: mesafeYenidenOnayGoster
        z: 200

        MouseArea { anchors.fill: parent }

        Rectangle {
            anchors.centerIn: parent
            width: 360
            height: 200
            radius: 14
            color: "#0f1420"
            border.color: "#1e2a3f"
            border.width: 1

            Column {
                anchors.centerIn: parent
                spacing: 18
                width: parent.width - 40

                Text {
                    width: parent.width
                    horizontalAlignment: Text.AlignHCenter
                    wrapMode: Text.WordWrap
                    text: txt("silinecekOnay")
                    color: "#dce8f5"
                    font.family: "Segoe UI"
                    font.pixelSize: 14
                    font.bold: true
                }

                Row {
                    anchors.horizontalCenter: parent.horizontalCenter
                    spacing: 12

                    Button {
                        width: 150
                        height: 42
                        text: txt("vazgecButon")

                        onClicked: mesafeYenidenOnayGoster = false

                        background: Rectangle {
                            radius: 8
                            color: parent.hovered ? "#1e2a3f" : "transparent"
```

```
                        border.color: "#1e2a3f"
                        border.width: 1
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#9ca3af"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Button {
                    width: 150
                    height: 42
                    text: txt("evetYenidenBaslaButon")
                    font.bold: true

                    onClicked: {
                        mesafeYenidenOnayGoster = false
                        mesafeAraEkranGoster = false
                        mesafeGoruntuleModu = false
                        mesafeTamDuzenlemeModu = false
                        mesafeNoktalari = []
                        mesafeSayiSecildi = false
                        mesafeDuzenlemeIndex = -1
                    }

                    background: Rectangle {
                        radius: 8
                        color: parent.hovered ? "#b91c1c" : "#991b1b"
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#ffffff"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }
}

Rectangle {
    anchors.fill: parent
    color: "#000000b3"
    visible: mesafeTamamlandiOnayGoster
    z: 200

    MouseArea { anchors.fill: parent }

    Rectangle {
        anchors.centerIn: parent
        width: 380
        height: 220
        radius: 14
        color: "#0f1420"
        border.color: "#16a34a"
        border.width: 1

        Column {
            anchors.centerIn: parent
            spacing: 18
            width: parent.width - 40

            Column {
                width: parent.width
                spacing: 6

                Text {
                    width: parent.width
                    horizontalAlignment: Text.AlignHCenter
                    text: txt("kalibrasyonTamamlandiBaslik")
                    color: "#4ade80"
```

```
                        font.family: "Segoe UI"
                        font.pixelSize: 17
                        font.bold: true
                    }

                    Text {
                        width: parent.width
                        horizontalAlignment: Text.AlignHCenter
                        wrapMode: Text.WordWrap
                        text: txt("toplamOnEk") + mesafeToplamNokta + txt("noktaGirildiSonEk")
                        color: "#9ca3af"
                        font.family: "Segoe UI"
                        font.pixelSize: 13
                    }
                }

                Row {
                    anchors.horizontalCenter: parent.horizontalCenter
                    spacing: 12

                    Button {
                        width: 150
                        height: 42
                        text: txt("noktalariKontrolEt")

                        onClicked: mesafeTamamlandiOnayGoster = false

                        background: Rectangle {
                            radius: 8
                            color: parent.hovered ? "#1e2a3f" : "transparent"
                            border.color: "#1e2a3f"
                            border.width: 1
                            Behavior on color { ColorAnimation { duration: 120 } }
                        }

                        contentItem: Text {
                            text: parent.text
                            color: "#9ca3af"
                            font: parent.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                        }
                    }

                    Button {
                        width: 150
                        height: 42
                        text: txt("kaydetVeBitir")
                        font.bold: true

                        onClicked: {
                            mesafeTamamlandiOnayGoster = false
                            mesafeKaydetVeBildir()
                        }

                        background: Rectangle {
                            radius: 8
                            color: parent.hovered ? "#22c55e" : "#16a34a"
                            Behavior on color { ColorAnimation { duration: 120 } }
                        }

                        contentItem: Text {
                            text: parent.text
                            color: "#ffffff"
                            font: parent.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                        }
                    }
                }
            }
        }
    }

    Column {
        anchors.centerIn: parent
        spacing: 24
        width: 360
        visible: !mesafeSayiSecildi && !mesafeAraEkranGoster && !mesafeGoruntuleModu

```

```
                        Text {
                            width: parent.width
                            horizontalAlignment: Text.AlignHCenter
                            wrapMode: Text.WordWrap
                            text: txt("kacNoktaSoru")
                            color: "#dce8f5"
                            font.family: "Segoe UI"
                            font.pixelSize: 16
                            font.bold: true
                        }

                        Text {
                            width: parent.width
                            horizontalAlignment: Text.AlignHCenter
                            wrapMode: Text.WordWrap
                            text: txt("dahaFazlaNokta")
                            color: "#6b7280"
                            font.family: "Segoe UI"
                            font.pixelSize: 12
                        }

                        Grid {
                            anchors.horizontalCenter: parent.horizontalCenter
                            columns: 4
                            spacing: 12

                            Repeater {
                                model: [4, 6, 8, 10]

                                Rectangle {
                                    id: mesafeSecKarti
                                    width: 72
                                    height: 72
                                    radius: 12
                                    color: mesafeSecKarti.hovered ? "#241a38" : "#0f1420"
                                    border.color: "#9333ea"
                                    border.width: 1
                                    property bool hovered: false
                                    Behavior on color { ColorAnimation { duration: 120 } }

                                    Text {
                                        anchors.centerIn: parent
                                        text: modelData
                                        color: "#dce8f5"
                                        font.family: "Segoe UI"
                                        font.pixelSize: 26
                                        font.bold: true
                                    }

                                    MouseArea {
                                        anchors.fill: parent
                                        hoverEnabled: true
                                        cursorShape: Qt.PointingHandCursor
                                        onEntered: mesafeSecKarti.hovered = true
                                        onExited: mesafeSecKarti.hovered = false
                                        onClicked: {
                                            mesafeToplamNokta = modelData
                                            mesafeNoktalari = []
                                            mesafeDuzenlemeIndex = -1
                                            mesafeSayiSecildi = true
                                        }
                                    }
                                }
                            }
                        }
                    }

                    Row {
                        anchors.centerIn: parent
                        spacing: 24
                        visible: mesafeSayiSecildi

                        Rectangle {
                            width: 340
                            height: 500
                            radius: 14
                            color: "#0f1420"
                            border.color: "#1e2a3f"
                            border.width: 1

```

```
                                Column {
                                    anchors.centerIn: parent
                                    spacing: 16
                                    width: parent.width - 40

                                    Rectangle {
                                        anchors.horizontalCenter: parent.horizontalCenter
                                        visible: mesafeOlcumKayitliVar && !mesafeTamDuzenlemeModu
                                        width: mesafeSonKaliMetni.implicitWidth + 20
                                        height: 24
                                        radius: 12
                                        color: "#0a0e17"
                                        border.color: "#1e2a3f"
                                        border.width: 1

                                        Text {
                                            id: mesafeSonKaliMetni
                                            anchors.centerIn: parent
                                            text: txt("kayitliOnEk") + mesafeOlcumNoktaSayisi + txt("noktaSonEk")
                                            color: "#6b7280"
                                            font.pixelSize: 11
                                        }
                                    }

                                    Text {
                                        anchors.horizontalCenter: parent.horizontalCenter
                                        text: mesafeTamDuzenlemeModu
                                              ? txt("kalibrasyonuDuzenle")
                                              : txt("noktaOnEk") + (mesafeNoktalari.length + 1) + " / " + mesafeToplamNokta
                                        color: "#9333ea"
                                        font.family: "Segoe UI"
                                        font.pixelSize: 13
                                        font.bold: true
                                        font.letterSpacing: 1
                                    }

                                    Rectangle {
                                        width: parent.width
                                        height: 6
                                        radius: 3
                                        color: "#1e2a3f"

                                        Rectangle {
                                            width: parent.width * (mesafeNoktalari.length / mesafeToplamNokta)
                                            height: parent.height
                                            radius: 3
                                            color: "#9333ea"
                                            Behavior on width { NumberAnimation { duration: 150 } }
                                        }
                                    }

                                    Text {
                                        width: parent.width
                                        horizontalAlignment: Text.AlignHCenter
                                        wrapMode: Text.WordWrap
                                        text: mesafeDuzenlemeIndex >= 0
                                              ? txt("noktaOnEk") + (mesafeDuzenlemeIndex + 1) + txt("duzenleniyorSonEk")
                                              : (mesafeTamDuzenlemeModu
                                                 ? txt("duzenlemekIcinTikla")
                                                 : txt("hedefMesafeYonerge"))
                                        color: mesafeDuzenlemeIndex >= 0 ? "#9333ea" : "#9ca3af"
                                        font.family: "Segoe UI"
                                        font.bold: mesafeDuzenlemeIndex >= 0
                                        font.pixelSize: 13
                                    }

                                    TextField {
                                        id: hedefCmKutusu
                                        anchors.horizontalCenter: parent.horizontalCenter
                                        width: 180
                                        height: 44
                                        placeholderText: "cm"
                                        placeholderTextColor: "#4b5563"
                                        color: "#ffffff"
                                        font.pixelSize: 16
                                        horizontalAlignment: TextInput.AlignHCenter
                                        validator: DoubleValidator { bottom: 0; top: 300; decimals: 1 }
                                        background: Rectangle {
                                            color: "#0a0e17"
                                            radius: 10
```

```
        border.color: hedefCmKutusu.activeFocus ? "#9333ea" : "#1e2a3f"
        border.width: 1
    }
}

Rectangle {
    anchors.horizontalCenter: parent.horizontalCenter
    width: parent.width
    height: 64
    radius: 12
    color: "#132335"
    border.color: wifiManager.baglandi ? "#9333ea" : "#dc2626"
    border.width: 1
    Behavior on border.color { ColorAnimation { duration: 120 } }

    Text {
        anchors.centerIn: parent
        text: txt("hamDegerOnEk") + sensorManager.hamMesafe.toFixed(0)
        color: wifiManager.baglandi ? "#c084fc" : "#f87171"
        font.family: "Segoe UI"
        font.pixelSize: 20
        font.bold: true
    }
}

Text {
    width: parent.width
    horizontalAlignment: Text.AlignHCenter
    wrapMode: Text.WordWrap
    visible: !wifiManager.baglandi
    text: txt("hamDegerSifirUyari")
    color: "#f87171"
    font.family: "Segoe UI"
    font.pixelSize: 11
}

Button {
    id: mesafeNoktaKaydetButonu
    width: parent.width
    height: 46
    property bool tumNoktalarGirildi: !mesafeTamDuzenlemeModu && mesafeDuzenlemeIndex < 0 && mesafeNoktalari.length >= mesafeToplamNokta
    text: mesafeDuzenlemeIndex >= 0
          ? txt("noktayiGuncelle")
          : (tumNoktalarGirildi
             ? txt("kalibrasyonuTamamla")
             : (mesafeNoktalari.length >= mesafeToplamNokta - 1 ? txt("sonNoktayiKaydet") : txt("buNoktayiKaydet")))
    font.pixelSize: 14
    font.bold: true
    enabled: tumNoktalarGirildi || (hedefCmKutusu.text.length > 0 && (mesafeDuzenlemeIndex >= 0 || mesafeNoktalari.length < mesafeToplamNokta))

    onClicked: {
        if (tumNoktalarGirildi) {
            mesafeTamamlandiOnayGoster = true
            return
        }

        var yeniNokta = { hedefMm: parseFloat(hedefCmKutusu.text) * 10, hamDeger: sensorManager.hamMesafe }


        if (mesafeDuzenlemeIndex >= 0) {
            // Var olan bir noktayı yerinde güncelliyoruz, diğer noktalar etkilenmiyor
            var guncelDizi = mesafeNoktalari.slice()
            guncelDizi[mesafeDuzenlemeIndex] = yeniNokta
            mesafeNoktalari = guncelDizi
            mesafeDuzenlemeIndex = -1
            hedefCmKutusu.text = ""
            return
        }

        var yeniDizi = mesafeNoktalari.slice()
        yeniDizi.push(yeniNokta)
        mesafeNoktalari = yeniDizi
        hedefCmKutusu.text = ""

        if (mesafeNoktalari.length >= mesafeToplamNokta) {
            mesafeTamamlandiOnayGoster = true
        }
```

```
            }

            background: Rectangle {
                radius: 10
                color: parent.enabled ? (parent.hovered ? "#a855f7" : "#9333ea") : "#1e2a3f"
                Behavior on color { ColorAnimation { duration: 120 } }
            }

            contentItem: Text {
                text: parent.text
                color: "#ffffff"
                font: parent.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }

        Button {
            width: parent.width
            height: 46
            visible: mesafeTamDuzenlemeModu
            text: txt("degisiklikleriKaydet")
            font.pixelSize: 14
            font.bold: true

            onClicked: mesafeKaydetVeBildir()

            background: Rectangle {
                radius: 10
                color: parent.hovered ? "#22c55e" : "#16a34a"
                Behavior on color { ColorAnimation { duration: 120 } }
            }

            contentItem: Text {
                text: parent.text
                color: "#ffffff"
                font: parent.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }

        Button {
            width: parent.width
            height: 38
            visible: mesafeDuzenlemeIndex >= 0
            text: txt("vazgecButon")
            font.pixelSize: 13

            onClicked: {
                mesafeDuzenlemeIndex = -1
                hedefCmKutusu.text = ""
            }

            background: Rectangle {
                radius: 8
                color: parent.hovered ? "#1e2a3f" : "transparent"
                border.color: "#1e2a3f"
                border.width: 1
                Behavior on color { ColorAnimation { duration: 120 } }
            }

            contentItem: Text {
                text: parent.text
                color: "#9ca3af"
                font: parent.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }
    }
}

Rectangle {
    width: 300
    height: 500
    radius: 14
    color: "#0f1420"
    border.color: "#1e2a3f"
    border.width: 1
```

```
                                Column {
                                    anchors.top: parent.top
                                    anchors.left: parent.left
                                    anchors.right: parent.right
                                    anchors.margins: 20
                                    spacing: 4

                                    Text {
                                        text: txt("kaydedilenNoktalarBaslik")
                                        color: "#6b7280"
                                        font.family: "Segoe UI"
                                        font.pixelSize: 11
                                        font.bold: true
                                        font.letterSpacing: 1
                                    }

                                    Text {
                                        text: txt("duzeltmekIcinTikla")
                                        color: "#4b5563"
                                        font.family: "Segoe UI"
                                        font.pixelSize: 10
                                    }
                                }

                                ListView {
                                    anchors.top: parent.top
                                    anchors.left: parent.left
                                    anchors.right: parent.right
                                    anchors.bottom: parent.bottom
                                    anchors.margins: 16
                                    anchors.topMargin: 60
                                    clip: true
                                    spacing: 8

                                    model: mesafeNoktalari

                                    delegate: Rectangle {
                                        width: ListView.view.width
                                        height: 48
                                        radius: 8
                                        color: mesafeSatirAlani.containsMouse ? "#241a38" : "#0a0e17"
                                        border.color: index === mesafeDuzenlemeIndex ? "#9333ea" : "#1e2a3f"
                                        border.width: index === mesafeDuzenlemeIndex ? 2 : 1
                                        Behavior on color { ColorAnimation { duration: 120 } }

                                        Row {
                                            anchors.fill: parent
                                            anchors.leftMargin: 12
                                            anchors.rightMargin: 12
                                            spacing: 8

                                            Rectangle {
                                                width: 22
                                                height: 22
                                                radius: 11
                                                anchors.verticalCenter: parent.verticalCenter
                                                color: "#241a38"
                                                border.color: "#9333ea"
                                                border.width: 1

                                                Text {
                                                    anchors.centerIn: parent
                                                    text: index + 1
                                                    color: "#c084fc"
                                                    font.pixelSize: 10
                                                    font.bold: true
                                                }
                                            }

                                            Column {
                                                anchors.verticalCenter: parent.verticalCenter
                                                spacing: 1

                                                Text {
                                                    text: (modelData.hedefMm / 10).toFixed(1) + " cm"
                                                    color: "#dce8f5"
                                                    font.family: "Segoe UI"
                                                    font.pixelSize: 13
                                                    font.bold: true
```

```
                                            }

                                            Text {
                                                text: "ADC: " + modelData.hamDeger.toFixed(0)
                                                color: "#6b7280"
                                                font.family: "Segoe UI"
                                                font.pixelSize: 10
                                            }
                                        }
                                    }

                                    Text {
                                        anchors.right: parent.right
                                        anchors.verticalCenter: parent.verticalCenter
                                        anchors.rightMargin: 12
                                        text: "✏️"
                                        color: "#6b7280"
                                        font.pixelSize: 13
                                        visible: mesafeSatirAlani.containsMouse
                                    }

                                    MouseArea {
                                        id: mesafeSatirAlani
                                        anchors.fill: parent
                                        hoverEnabled: true
                                        cursorShape: Qt.PointingHandCursor
                                        onClicked: {
                                            hedefCmKutusu.text = (modelData.hedefMm / 10).toString()
                                            mesafeDuzenlemeIndex = index
                                        }
                                    }
                                }
                            }
                        }
                    }
                }

                Rectangle {
                    id: mesafeBildirimKutusu
                    anchors.top: parent.top
                    anchors.horizontalCenter: parent.horizontalCenter
                    anchors.topMargin: 12
                    z: 100
                    width: mesafeBildirimMetni.implicitWidth + 24
                    height: 32
                    radius: 16
                    color: mesafeBildirimBasarili ? "#123321" : "#3a1414"
                    opacity: mesafeBildirimGoster ? 1 : 0
                    visible: opacity > 0
                    Behavior on opacity { NumberAnimation { duration: 200 } }

                    Text {
                        id: mesafeBildirimMetni
                        anchors.centerIn: parent
                        text: mesafeBildirimMesaj
                        color: mesafeBildirimBasarili ? "#4ade80" : "#f87171"
                        font.family: "Segoe UI"
                        font.pixelSize: 12
                        font.bold: true
                    }
                }
            }
        }
    }
}
```

qml/pages/LoginPage.qml

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import sliper

Rectangle {
    id: sayfa

    property bool sifreGorunur: false
    property bool beniHatirla: false

    signal girisBasarili()

    readonly property var metinler: ({
        girisBasligi: { tr: "Sisteme Giriş", en: "System Login" },
        girisAltBaslik: { tr: "Yetkili hesabınızla devam edin", en: "Continue with your authorized account" },
        kullaniciAdiEtiket: { tr: "KULLANICI ADI", en: "USERNAME" },
        kullaniciAdiPlaceholder: { tr: "kullanici_adi", en: "username" },
        sifreEtiket: { tr: "ŞİFRE", en: "PASSWORD" },
        girisYapButon: { tr: "Giriş Yap", en: "Log In" },
        telifHakki: { tr: "© 2026 Liya Test A.Ş. SLIPER Analiz Yazılımı", en: "© 2026 Liya Test A.Ş. SLIPER Analysis Software" }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    gradient: Gradient {
        GradientStop { position: 0.0; color: "#101c33" }
        GradientStop { position: 1.0; color: "#0a1220" }
    }

    Row {
        anchors.top: parent.top
        anchors.right: parent.right
        anchors.margins: 24
        spacing: 8

        Rectangle {
            width: 44
            height: 30
            radius: 6
            color: "#0f1420"
            border.color: Translations.turkish ? "#3b82f6" : "#1e2a3f"
            border.width: 1

            Text {
                anchors.centerIn: parent
                text: "TR"
                color: Translations.turkish ? "#ffffff" : "#6b7280"
                font.pixelSize: 12
                font.bold: Translations.turkish
            }

            MouseArea {
                anchors.fill: parent
                cursorShape: Qt.PointingHandCursor
                onClicked: Translations.turkish = true
            }
        }

        Rectangle {
            width: 44
            height: 30
            radius: 6
            color: "#0f1420"
            border.color: !Translations.turkish ? "#3b82f6" : "#1e2a3f"
            border.width: 1

            Text {
                anchors.centerIn: parent
                text: "EN"
                color: !Translations.turkish ? "#ffffff" : "#6b7280"
                font.pixelSize: 12
                font.bold: !Translations.turkish
            }

            MouseArea {
```

```
                anchors.fill: parent
                cursorShape: Qt.PointingHandCursor
                onClicked: Translations.turkish = false
            }
        }
    }

    Rectangle {
        anchors.centerIn: kart
        width: kart.width + 48
        height: kart.height + 48
        radius: 28
        color: "#18000000"
    }

    Rectangle {
        anchors.centerIn: kart
        width: kart.width + 32
        height: kart.height + 32
        radius: 24
        color: "#25000000"
    }

    Rectangle {
        anchors.centerIn: kart
        width: kart.width + 18
        height: kart.height + 18
        radius: 20
        color: "#35000000"
    }

    Rectangle {
        anchors.centerIn: kart
        width: kart.width + 8
        height: kart.height + 8
        radius: 18
        color: "#45000000"
    }

    Rectangle {
        id: kart
        anchors.centerIn: parent
        width: 400
        color: "#0f1420"
        radius: 16
        border.color: "#4b5563"
        border.width: 1
        height: govde.implicitHeight + 48

        Column {
            id: govde
            anchors.top: parent.top
            anchors.horizontalCenter: parent.horizontalCenter
            anchors.topMargin: 32
            width: parent.width - 48
            spacing: 20

            Image {
                anchors.horizontalCenter: parent.horizontalCenter
                source: "../assets/logo.png"
                width: 144
                height: 144
                fillMode: Image.PreserveAspectFit
            }

            Rectangle {
                width: parent.width
                height: 1
                color: "#1e2a3f"
            }

            Column {
                width: parent.width
                spacing: 4

                Text {
                    anchors.horizontalCenter: parent.horizontalCenter
                    text: txt("girisBasligi")
                    color: "#ffffff"
                    font.family: "Segoe UI"
```

```
                    font.pixelSize: 22
                    font.bold: true
                }

                Text {
                    anchors.horizontalCenter: parent.horizontalCenter
                    text: txt("girisAltBaslik")
                    color: "#6b7280"
                    font.family: "Segoe UI"
                    font.pixelSize: 12
                }
            }

            Column {
                width: parent.width
                spacing: 6

                Text {
                    text: txt("kullaniciAdiEtiket")
                    color: "#6b7280"
                    font.family: "Segoe UI"
                    font.pixelSize: 10
                    font.letterSpacing: 1
                }

                TextField {
                    id: kullaniciAdiKutusu
                    width: parent.width
                    height: 42
                    placeholderText: txt("kullaniciAdiPlaceholder")
                    placeholderTextColor: "#4b5563"
                    color: "#ffffff"
                    font.pixelSize: 13
                    leftPadding: 12
                    verticalAlignment: TextInput.AlignVCenter
                    background: Rectangle {
                        color: "#0a0e17"
                        radius: 6
                        border.color: kullaniciAdiKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                        border.width: 1
                    }
                }
            }

            Column {
                width: parent.width
                spacing: 6

                Text {
                    text: txt("sifreEtiket")
                    color: "#6b7280"
                    font.family: "Segoe UI"
                    font.pixelSize: 10
                    font.letterSpacing: 1
                }

                TextField {
                    id: sifreKutusu
                    width: parent.width
                    height: 42
                    placeholderText: "********"
                    placeholderTextColor: "#4b5563"
                    echoMode: sayfa.sifreGorunur ? TextInput.Normal : TextInput.Password
                    color: "#ffffff"
                    font.pixelSize: 13
                    leftPadding: 12
                    rightPadding: 48
                    verticalAlignment: TextInput.AlignVCenter
                    background: Rectangle {
                        color: "#0a0e17"
                        radius: 6
                        border.color: sifreKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                        border.width: 1
                    }

                    Rectangle {
                        id: sifreGosterButonu
                        anchors.right: parent.right
                        anchors.verticalCenter: parent.verticalCenter
                        anchors.rightMargin: 8
```

```
    width: 28
    height: 28
    radius: 6
    color: sifreGosterMouse.pressed ? "#162033" : (sifreGosterMouse.containsMouse ? "#111827" : "transparent")
    border.color: sayfa.sifreGorunur ? "#334155" : "transparent"
    border.width: 1

    property color ikonRengi: sifreGosterMouse.containsMouse || sayfa.sifreGorunur ? "#cbd5e1" : "#6b7280"

    Canvas {
        id: sifreIkonu
        anchors.centerIn: parent
        width: 18
        height: 18

        onPaint: {
            var ctx = getContext("2d")
            ctx.clearRect(0, 0, width, height)
            ctx.lineWidth = 1.7
            ctx.lineCap = "round"
            ctx.lineJoin = "round"
            ctx.strokeStyle = sifreGosterButonu.ikonRengi

            ctx.beginPath()
            ctx.moveTo(1.5, 9)
            ctx.bezierCurveTo(4.2, 4.8, 7.0, 3.8, 9, 3.8)
            ctx.bezierCurveTo(11.0, 3.8, 13.8, 4.8, 16.5, 9)
            ctx.bezierCurveTo(13.8, 13.2, 11.0, 14.2, 9, 14.2)
            ctx.bezierCurveTo(7.0, 14.2, 4.2, 13.2, 1.5, 9)
            ctx.stroke()

            ctx.beginPath()
            ctx.arc(9, 9, 2.5, 0, Math.PI * 2, false)
            ctx.stroke()

            if (sayfa.sifreGorunur) {
                ctx.beginPath()
                ctx.moveTo(3, 15)
                ctx.lineTo(15, 3)
                ctx.stroke()
            }
        }

        Connections {
            target: sayfa
            function onSifreGorunurChanged() {
                sifreIkonu.requestPaint()
            }
        }

        Connections {
            target: sifreGosterButonu
            function onIkonRengiChanged() {
                sifreIkonu.requestPaint()
            }
        }
    }

    MouseArea {
        id: sifreGosterMouse
        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onClicked: sayfa.sifreGorunur = !sayfa.sifreGorunur
    }
}

Text {
    anchors.right: parent.right
    anchors.verticalCenter: parent.verticalCenter
    anchors.rightMargin: 12
    text: sayfa.sifreGorunur ? "👁" : "👁"
    color: "#6b7280"
    font.pixelSize: 14
    visible: false

    MouseArea {
        anchors.fill: parent
        anchors.margins: -8
        cursorShape: Qt.PointingHandCursor
```

```
                            onClicked: sayfa.sifreGorunur = !sayfa.sifreGorunur
                        }
                    }
                }
            }

            Button {
                width: parent.width
                height: 44
                text: txt("girisYapButon")
                font.family: "Segoe UI"
                font.pixelSize: 14
                font.bold: true

                onClicked: {
                    var sonuc = backend.girisYap(kullaniciAdiKutusu.text, sifreKutusu.text)

                    if (sonuc) {
                        sayfa.girisBasarili()
                    } else {
                        console.log("Kullanici adi veya sifre hatali")
                    }
                }

                background: Rectangle {
                    radius: 8
                    color: parent.pressed ? "#2563eb" : (parent.hovered ? "#4f8cf7" : "#3b82f6")
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#ffffff"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Rectangle {
                width: parent.width
                height: 1
                color: "#1e2a3f"
            }

            Text {
                anchors.horizontalCenter: parent.horizontalCenter
                text: txt("telifHakki")
                color: "#4b5563"
                font.family: "Segoe UI"
                font.pixelSize: 10
            }
        }
    }
}
```

qml/pages/OlcumPage.qml

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtCharts 6.7
import sliper

Rectangle {
    color: "#0a0e17"

    readonly property var metinler: ({
        testBilgileri: { tr: "TEST BİLGİLERİ", en: "TEST INFORMATION" },
        musteri: { tr: "Müşteri", en: "Customer" },
        musteriPlaceholder: { tr: "Musteri adi girin...", en: "Enter customer name..." },
        betonRecetesi: { tr: "Beton Reçetesi", en: "Concrete Mix Design" },
        receteSeciniz: { tr: "Reçete Seçin...", en: "Select Mix Design..." },
        cimentoEtiket: { tr: "Çimento ", en: "Cement " },
        suCimentoEtiket: { tr: "  •  Su/Çimento ", en: "  •  Water/Cement " },
        eklenenAgirlik: { tr: "Eklenen Ağırlık (kg)", en: "Added Weight (kg)" },
        geriButon: { tr: "− Geri", en: "− Undo" },
        sifirlaButon: { tr: "↺ Sıfırla", en: "↺ Reset" },
        olcumuBaslatButon: { tr: "▶  Ölçümü Başlat", en: "▶  Start Measurement" },
        devamEtButon: { tr: "▶  Devam Et", en: "▶  Resume" },
        duraklatButon: { tr: "⏸  Duraklat", en: "⏸  Pause" },
        bitirButon: { tr: "■  Bitir", en: "■  Finish" },
        veriYok: { tr: "VERI YOK", en: "NO DATA" },
        duraklatildiDurum: { tr: "DURAKLATILDI", en: "PAUSED" },
        anlikDegerler: { tr: "ANLIK DEĞERLER", en: "CURRENT VALUES" },
        basincEtiket: { tr: "BASINÇ", en: "PRESSURE" },
        konumEtiket: { tr: "KONUM", en: "POSITION" },
        hizEtiket: { tr: "HIZ", en: "SPEED" },
        debiEtiket: { tr: "DEBİ", en: "FLOW RATE" },
        basincZaman: { tr: "Basınç - Zaman", en: "Pressure - Time" },
        konumZaman: { tr: "Konum - Zaman", en: "Position - Time" },
        hizZaman: { tr: "Hız - Zaman", en: "Speed - Time" },
        debiZaman: { tr: "Debi - Zaman", en: "Flow Rate - Time" },
        sesliDinleniyor: { tr: "🎙 Dinleniyor — \"Başlat / Durdur / Devam Et / Bitir / Ekle\"", en: "🎙 Listening — \"Start / Stop / Resume / Finish / Add\"" },
        sesliHazirlaniyor: { tr: "🎙 Sesli komut hazırlanıyor...", en: "🎙 Preparing voice commands..." },
        duyulanEtiket: { tr: "Duyulan: ", en: "Heard: " },
        bitirOnaySorusu: { tr: "Ölçümü bitirmek istediğinize emin misiniz?", en: "Are you sure you want to finish the measurement?" },
        kaydetVeBitir: { tr: "💾  Kaydet ve Bitir", en: "💾  Save and Finish" },
        testiSilVeBitir: { tr: "🗑  Testi Sil ve Bitir", en: "🗑  Delete Test and Finish" },
        vazgecTesteDevam: { tr: "Vazgeç, Teste Devam Et", en: "Cancel, Continue Testing" },
        uyariMusteriAdi: { tr: "Lütfen müşteri adını girin.", en: "Please enter the customer name." },
        uyariReceteSec: { tr: "Lütfen bir beton reçetesi seçin.", en: "Please select a concrete mix design." },
        uyariCihazBagliDegil: { tr: "Cihaz bağlı değil. Lütfen önce sol alttan SLIPER-ESP32'ye bağlanın.", en: "Device not connected. Please connect to SLIPER-ESP32 from the bottom left first." },
        uyariAktifOlcumYok: { tr: "Aktif bir ölçüm yok. Önce ölçümü başlatın.", en: "No active measurement. Please start a measurement first." }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }
    property real zamanSayaci: 0
    property int aktifOlcumId: -1
    property string uyariMesaji: ""
    property bool bitirIcinDuraklatildi: false

    property var basincGecmis: []
    property var konumGecmis: []
    property var hizGecmis: []
    property var debiGecmis: []

    readonly property real agirlikBirim: 1.6
    property int agirlikAdedi: 0
    readonly property real agirlikToplam: agirlikAdedi * agirlikBirim

    // TS EN 206 / TS 13515 normal beton dayanım sınıfları (C sınıfı) — çimento dozajı
    // ve azami su/çimento oranı, ilgili sınıfın minimum bağlayıcı gereksinimine göre.
    readonly property var receteListesi: [
        { sinifTr: "Reçete Seçin...", sinifEn: "Select Mix Design...", cimentoTr: "", cimentoEn: "", suCimentoTr: "", suCimentoEn: "", aciklamaTr: "", aciklamaEn: "" },
        { sinifTr: "C16/20", sinifEn: "C16/20", cimentoTr: "min. 260 kg/m³", cimentoEn: "min. 260 kg/m³", suCimentoTr: "maks. 0.65", suCimentoEn: "max. 0.65", aciklamaTr: "Hafif yükte yalın/donatılı beton", aciklamaEn: "Plain/reinforced concrete for light loads" },
        { sinifTr: "C20/25", sinifEn: "C20/25", cimentoTr: "min. 280 kg/m³", cimentoEn: "min. 280 kg/m³", suCimentoTr: "maks. 0.60", suCimentoEn: "max. 0.60", aciklamaTr: "Standart betonarme", aciklamaEn: "Standard reinforced concrete" },
        { sinifTr: "C25/30", sinifEn: "C25/30", cimentoTr: "min. 300 kg/m³", cimentoEn: "min. 300 kg/m³", suCimentoTr: "maks. 0.55",
```

```
suCimentoEn: "max. 0.55", aciklamaTr: "Standart betonarme (TS EN 206)", aciklamaEn: "Standard reinforced concrete (TS EN 206)" },
        { sinifTr: "C30/37", sinifEn: "C30/37", cimentoTr: "min. 320 kg/m³", cimentoEn: "min. 320 kg/m³", suCimentoTr: "maks. 0.50", suCimentoEn: "max. 0.50", aciklamaTr: "Standart betonarme", aciklamaEn: "Standard reinforced concrete" },
        { sinifTr: "C30/37 SCC", sinifEn: "C30/37 SCC", cimentoTr: "min. 380 kg/m³", cimentoEn: "min. 380 kg/m³", suCimentoTr: "maks. 0.45", suCimentoEn: "max. 0.45", aciklamaTr: "Kendiliğinden yerleşen beton (SF2)", aciklamaEn: "Self-compacting concrete (SF2)" },
        { sinifTr: "C35/45", sinifEn: "C35/45", cimentoTr: "min. 340 kg/m³", cimentoEn: "min. 340 kg/m³", suCimentoTr: "maks. 0.45", suCimentoEn: "max. 0.45", aciklamaTr: "Orta-yüksek dayanım", aciklamaEn: "Medium-high strength" },
        { sinifTr: "C40/50", sinifEn: "C40/50", cimentoTr: "min. 360 kg/m³", cimentoEn: "min. 360 kg/m³", suCimentoTr: "maks. 0.40", suCimentoEn: "max. 0.40", aciklamaTr: "Yüksek dayanım", aciklamaEn: "High strength" },
        { sinifTr: "C45/55", sinifEn: "C45/55", cimentoTr: "min. 380 kg/m³", cimentoEn: "min. 380 kg/m³", suCimentoTr: "maks. 0.38", suCimentoEn: "max. 0.38", aciklamaTr: "Yüksek dayanım", aciklamaEn: "High strength" },
        { sinifTr: "C50/60", sinifEn: "C50/60", cimentoTr: "min. 400 kg/m³", cimentoEn: "min. 400 kg/m³", suCimentoTr: "maks. 0.35", suCimentoEn: "max. 0.35", aciklamaTr: "Yüksek dayanım (TS EN 206 normal beton sınırı)", aciklamaEn: "High strength (TS EN 206 normal concrete limit)" }
    ]

    signal olcumTamamlandi(int id)
    signal agirlikEklendi()

    function resetMeasurementScreen() {
        aktifOlcumId = -1
        uyariMesaji = ""
        bitirIcinDuraklatildi = false
        musteriKutusu.text = ""
        receteKutusu.currentIndex = 0
        agirlikAdedi = 0
        calculator.sifirla()
        grafikleriSifirla()
    }

    function baslatOlcumu() {
        if (aktifOlcumId > 0) return

        if (musteriKutusu.text.trim().length === 0) {
            uyariMesaji = txt("uyariMusteriAdi")
            return
        }

        if (receteKutusu.currentIndex <= 0) {
            uyariMesaji = txt("uyariReceteSec")
            return
        }

        if (!sensorManager.veriGecerli) {
            console.warn("Gercek sensor verisi yok, olcum baslatilmadi.")
            uyariMesaji = txt("uyariCihazBagliDegil")
            return
        }

        uyariMesaji = ""
        grafikleriSifirla()

        calculator.sifirla()
        aktifOlcumId = database.olcumBaslat(
            musteriKutusu.text,
            receteKutusu.currentText,
            agirlikToplam
        )
        console.log("Aktif olcum id:", aktifOlcumId)
    }

    function bitirTalebiGoster() {
        if (aktifOlcumId > 0) {
            if (!calculator.duraklatildi) {
                calculator.duraklat()
                bitirIcinDuraklatildi = true
            }
            bitirOnayPopup.open()
        } else {
            uyariMesaji = txt("uyariAktifOlcumYok")
        }
    }

    function grafikleriSifirla() {
        zamanSayaci = 0
        basincSerisi.clear()
        konumSerisi.clear()
        hizSerisi.clear()
        debiSerisi.clear()
        basincGecmis = []
```

```
        konumGecmis = []
        hizGecmis = []
        debiGecmis = []
        xEkseni.min = 0
        xEkseni.max = 60
        yEkseni.min = 0
        yEkseni.max = 1100
        konumXEkseni.min = 0
        konumXEkseni.max = 60
        konumYEkseni.max = 500
        hizXEkseni.min = 0
        hizXEkseni.max = 60
        hizYEkseni.max = 4
        debiXEkseni.min = 0
        debiXEkseni.max = 60
        debiYEkseni.max = 180
    }

    function eksenTavaniHesapla(dizi, tavan, taban) {
        if (dizi.length === 0) return taban
        var maxDeger = Math.max.apply(null, dizi)
        var hedef = maxDeger * 1.25
        if (hedef < taban) hedef = taban
        if (hedef > tavan) hedef = tavan
        return hedef
    }

    // Basınç (mbar) atmosferik seviyede sabit bir taban etrafında dalgalanır,
    // bu yüzden sabit tavanlı ölçekleme yerine veriye göre kayan min/max kullanılır.
    function basincEkseniHesapla(dizi) {
        if (dizi.length === 0) return { min: 0, max: 1100 }
        var maxDeger = Math.max.apply(null, dizi)
        var minDeger = Math.min.apply(null, dizi)
        var pad = Math.max((maxDeger - minDeger) * 0.2, maxDeger * 0.02, 5)
        return { min: Math.max(0, minDeger - pad), max: maxDeger + pad }
    }

    Connections {
        target: sensorManager
        function onVeriGuncellendi() {
            if (!sensorManager.veriGecerli || aktifOlcumId <= 0 || calculator.duraklatildi) return

            zamanSayaci += 0.2

            basincSerisi.append(zamanSayaci, sensorManager.basinc)
            konumSerisi.append(zamanSayaci, sensorManager.konum)
            hizSerisi.append(zamanSayaci, sensorManager.hiz)
            debiSerisi.append(zamanSayaci, sensorManager.debi)

            basincGecmis.push(sensorManager.basinc)
            konumGecmis.push(sensorManager.konum)
            hizGecmis.push(sensorManager.hiz)
            debiGecmis.push(sensorManager.debi)

            if (zamanSayaci > 60) {
                basincSerisi.remove(0)
                konumSerisi.remove(0)
                hizSerisi.remove(0)
                debiSerisi.remove(0)

                basincGecmis.shift()
                konumGecmis.shift()
                hizGecmis.shift()
                debiGecmis.shift()

                xEkseni.min = zamanSayaci - 60
                xEkseni.max = zamanSayaci
                konumXEkseni.min = zamanSayaci - 60
                konumXEkseni.max = zamanSayaci
                hizXEkseni.min = zamanSayaci - 60
                hizXEkseni.max = zamanSayaci
                debiXEkseni.min = zamanSayaci - 60
                debiXEkseni.max = zamanSayaci
            }

            var basincAralik = basincEkseniHesapla(basincGecmis)
            yEkseni.min = basincAralik.min
            yEkseni.max = basincAralik.max
            konumYEkseni.max = eksenTavaniHesapla(konumGecmis, 500, 50)
            hizYEkseni.max = eksenTavaniHesapla(hizGecmis, 4, 0.5)
```

```
            debiYEkseni.max = eksenTavaniHesapla(debiGecmis, 180, 10)

            calculator.konumGuncelle(sensorManager.konum, sensorManager.hiz)
        }
    }

    // Sesli komutlar: "Başlat / Durdur / Bitir". Bu sayfa arka planda
    // (StackLayout icinde) da yasadigi icin, komutlar sadece Ölçüm sayfasi
    // ekranda goruntulenirken (visible) uygulanir.
    Connections {
        target: voiceCommandManager

        function onBaslatKomutu() {
            if (!visible) return
            baslatOlcumu()
        }

        function onDurdurKomutu() {
            if (!visible || aktifOlcumId <= 0) return
            if (!calculator.duraklatildi) calculator.duraklat()
        }

        function onDevamKomutu() {
            if (!visible || aktifOlcumId <= 0) return
            if (calculator.duraklatildi) calculator.devamEt()
        }

        function onBitirKomutu() {
            if (!visible) return
            bitirTalebiGoster()
        }

        function onEkleKomutu() {
            if (!visible) return
            agirlikAdedi += 1
            agirlikEklendi()
        }
    }

    Row {
        anchors.fill: parent
        spacing: 0

        Rectangle {
            id: testBilgileriPaneli
            width: 280
            height: parent.height
            color: "#0f1420"

            Text {
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.margins: 20
                text: txt("testBilgileri")
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 12
                font.bold: true
                font.letterSpacing: 1
            }

            Column {
                id: formAlanlari
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.margins: 20
                anchors.topMargin: 60
                spacing: 6

                Text {
                    text: txt("musteri")
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 12
                }

                TextField {
                    id: musteriKutusu
                    width: parent.width
```

```
                        height: 38
                        placeholderText: txt("musteriPlaceholder")
                        placeholderTextColor: "#4b5563"
                        color: "#ffffff"
                        font.pixelSize: 13
                        leftPadding: 10
                        verticalAlignment: TextInput.AlignVCenter
                        background: Rectangle {
                            color: "#0a0e17"
                            radius: 6
                            border.color: musteriKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                            border.width: 1
                        }
                    }

                    Text {
                        text: txt("betonRecetesi")
                        color: "#9ca3af"
                        font.family: "Segoe UI"
                        font.pixelSize: 12
                    }

                    ComboBox {
                        id: receteKutusu
                        width: parent.width
                        height: 38
                        model: receteListesi
                        textRole: Translations.turkish ? "sinifTr" : "sinifEn"

                        background: Rectangle {
                            color: "#0a0e17"
                            radius: 6
                            border.color: receteKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                            border.width: 1
                        }

                        contentItem: Text {
                            text: receteKutusu.displayText
                            color: "#9ca3af"
                            font.pixelSize: 13
                            leftPadding: 10
                            verticalAlignment: Text.AlignVCenter
                        }

                        delegate: ItemDelegate {
                            id: receteDelege
                            width: receteKutusu.width
                            height: modelData.cimentoTr.length > 0 ? 52 : 36
                            highlighted: receteKutusu.highlightedIndex === index

                            background: Rectangle {
                                color: receteDelege.highlighted ? "#182644" : "#0a0e17"
                            }

                            contentItem: Column {
                                anchors.verticalCenter: parent.verticalCenter
                                anchors.left: parent.left
                                anchors.leftMargin: 10
                                anchors.right: parent.right
                                anchors.rightMargin: 10
                                spacing: 2

                                Text {
                                    text: Translations.turkish ? modelData.sinifTr : modelData.sinifEn
                                    color: "#dce8f5"
                                    font.family: "Segoe UI"
                                    font.pixelSize: 13
                                    font.bold: true
                                }

                                Text {
                                    visible: modelData.cimentoTr.length > 0
                                    text: Translations.turkish
                                        ? (txt("cimentoEtiket") + modelData.cimentoTr + txt("suCimentoEtiket") + modelData.suCimentoTr)
                                        : (txt("cimentoEtiket") + modelData.cimentoEn + txt("suCimentoEtiket") + modelData.suCimentoEn)
                                    color: "#6b7280"
                                    font.family: "Segoe UI"
                                    font.pixelSize: 10
                                }
                            }
```

```
    }
}

Text {
    text: txt("eklenenAgirlik")
    color: "#9ca3af"
    font.family: "Segoe UI"
    font.pixelSize: 12
}

Column {
    width: parent.width
    spacing: 8

    Row {
        width: parent.width
        spacing: 8

        Button {
            id: agirlikEkleButonu
            width: (parent.width - 2 * parent.spacing) / 3
            height: 38
            text: "+ 1.6"
            font.family: "Segoe UI"
            font.pixelSize: 12

            onClicked: {
                agirlikAdedi += 1
                agirlikEklendi()
            }

            background: Rectangle {
                radius: 6
                color: agirlikEkleButonu.pressed ? "#1e3a8a" : (agirlikEkleButonu.hovered ? "#2563eb" : "#1d4ed8")
                border.color: "#3b82f6"
                border.width: 1
            }

            contentItem: Text {
                text: agirlikEkleButonu.text
                color: "#ffffff"
                font: agirlikEkleButonu.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }

        Button {
            id: agirlikAzaltButonu
            width: (parent.width - 2 * parent.spacing) / 3
            height: 38
            text: txt("geriButon")
            font.family: "Segoe UI"
            font.pixelSize: 12
            enabled: agirlikAdedi > 0

            onClicked: agirlikAdedi -= 1

            background: Rectangle {
                radius: 6
                color: agirlikAzaltButonu.pressed ? "#78350f" : (agirlikAzaltButonu.hovered ? "#a16207" : "#92400e")
                opacity: agirlikAzaltButonu.enabled ? 1.0 : 0.4
                border.color: "#d97706"
                border.width: 1
            }

            contentItem: Text {
                text: agirlikAzaltButonu.text
                color: "#ffffff"
                opacity: agirlikAzaltButonu.enabled ? 1.0 : 0.6
                font: agirlikAzaltButonu.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }

        Button {
            id: sifirlaButonu
            width: (parent.width - 2 * parent.spacing) / 3
            height: 38
```

```
                            text: txt("sifirlaButon")
                            font.family: "Segoe UI"
                            font.pixelSize: 12
                            enabled: agirlikAdedi > 0

                            onClicked: agirlikAdedi = 0

                            background: Rectangle {
                                radius: 6
                                color: sifirlaButonu.pressed ? "#7f1d1d" : (sifirlaButonu.hovered ? "#b91c1c" : "#991b1b")
                                opacity: sifirlaButonu.enabled ? 1.0 : 0.4
                                border.color: "#dc2626"
                                border.width: 1
                            }

                            contentItem: Text {
                                text: sifirlaButonu.text
                                color: "#ffffff"
                                opacity: sifirlaButonu.enabled ? 1.0 : 0.6
                                font: sifirlaButonu.font
                                horizontalAlignment: Text.AlignHCenter
                                verticalAlignment: Text.AlignVCenter
                            }
                        }
                    }

                    Rectangle {
                        width: parent.width
                        height: 34
                        radius: 6
                        color: "#0a0e17"
                        border.color: "#1e2a3f"
                        border.width: 1

                        Text {
                            anchors.centerIn: parent
                            text: agirlikAdedi + " x " + agirlikBirim.toFixed(1) + " = " + agirlikToplam.toFixed(1) + " kg"
                            color: agirlikAdedi > 0 ? "#dce8f5" : "#4b5563"
                            font.family: "Segoe UI"
                            font.pixelSize: 13
                            font.bold: true
                        }
                    }
                }

                Item { width: parent.width; height: 10 }

                Button {
                    id: baslatButonu
                    width: parent.width
                    height: 44
                    text: txt("olcumuBaslatButon")
                    font.family: "Segoe UI"
                    font.pixelSize: 14
                    font.bold: true
                    enabled: aktifOlcumId <= 0

                    onClicked: baslatOlcumu()

                    background: Rectangle {
                        radius: 8
                        color: !baslatButonu.enabled ? "#14532d" : (baslatButonu.pressed ? "#15803d" : (baslatButonu.hovered ? "#22c55e" : "#16a34a"))
                        opacity: baslatButonu.enabled ? 1.0 : 0.5
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: baslatButonu.text
                        color: "#ffffff"
                        opacity: baslatButonu.enabled ? 1.0 : 0.6
                        font: baslatButonu.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Text {
                    width: parent.width
                    visible: uyariMesaji.length > 0
```

```
                    text: uyariMesaji
                    color: "#f87171"
                    font.family: "Segoe UI"
                    font.pixelSize: 11
                    wrapMode: Text.WordWrap
                }

                Row {
                    width: parent.width
                    spacing: 10

                    Button {
                        id: duraklatButonu
                        width: (parent.width - parent.spacing) / 2
                        height: 40
                        text: calculator.duraklatildi ? txt("devamEtButon") : txt("duraklatButon")
                        font.family: "Segoe UI"
                        font.pixelSize: 13

                        onClicked: {
                            if (calculator.duraklatildi) {
                                calculator.devamEt()
                            } else {
                                calculator.duraklat()
                            }
                        }

                        background: Rectangle {
                            radius: 8
                            color: duraklatButonu.pressed ? "#78350f" : (duraklatButonu.hovered ? "#a16207" : "#92400e")
                            border.color: "#d97706"
                            border.width: 1
                        }

                        contentItem: Text {
                            text: duraklatButonu.text
                            color: "#ffffff"
                            font: duraklatButonu.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                        }
                    }

                    Button {
                        id: bitirButonu
                        width: (parent.width - parent.spacing) / 2
                        height: 40
                        text: txt("bitirButon")
                        font.family: "Segoe UI"
                        font.pixelSize: 13

                        onClicked: bitirTalebiGoster()

                        background: Rectangle {
                            radius: 8
                            color: bitirButonu.pressed ? "#7f1d1d" : (bitirButonu.hovered ? "#b91c1c" : "#991b1b")
                            border.color: "#dc2626"
                            border.width: 1
                        }

                        contentItem: Text {
                            text: bitirButonu.text
                            color: "#ffffff"
                            font: bitirButonu.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                        }
                    }
                }
            }

            Rectangle {
                id: durumKarti
                anchors.top: formAlanlari.bottom
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.topMargin: 20
                anchors.leftMargin: 20
                anchors.rightMargin: 20
                height: 52
```

```
                radius: 10
                color: "#0a0e17"
                border.color: "#1e2a3f"
                border.width: 1

                Row {
                    anchors.left: parent.left
                    anchors.verticalCenter: parent.verticalCenter
                    anchors.leftMargin: 14
                    spacing: 10

                    Rectangle {
                        id: trafikIsigi
                        width: 12
                        height: 12
                        radius: 6
                        anchors.verticalCenter: parent.verticalCenter
                        color: {
                            if (!sensorManager.veriGecerli) return "#dc2626"
                            if (calculator.duraklatildi) return "#6b7280"
                            if (calculator.durum === "YUKARIDA") return "#16a34a"
                            if (calculator.durum === "INIYOR") return "#f59e0b"
                            return "#dc2626"
                        }

                        Rectangle {
                            anchors.centerIn: parent
                            width: parent.width + 8
                            height: parent.height + 8
                            radius: width / 2
                            color: "transparent"
                            border.color: parent.color
                            border.width: 1
                            opacity: 0.35
                        }
                    }

                    Text {
                        anchors.verticalCenter: parent.verticalCenter
                        text: {
                            if (!sensorManager.veriGecerli) return txt("veriYok")
                            if (calculator.duraklatildi) return txt("duraklatildiDurum")
                            return calculator.durum
                        }
                        color: "#dce8f5"
                        font.family: "Segoe UI"
                        font.pixelSize: 13
                        font.bold: true
                    }
                }

                Row {
                    anchors.right: parent.right
                    anchors.verticalCenter: parent.verticalCenter
                    anchors.rightMargin: 14
                    spacing: 6

                    Text {
                        anchors.verticalCenter: parent.verticalCenter
                        text: "STROKE"
                        color: "#6b7280"
                        font.family: "Segoe UI"
                        font.pixelSize: 10
                        font.letterSpacing: 1
                    }

                    Text {
                        anchors.verticalCenter: parent.verticalCenter
                        text: calculator.strokeSayisi
                        color: "#e8a020"
                        font.family: "Segoe UI"
                        font.pixelSize: 16
                        font.bold: true
                    }
                }
            }

            Text {
                id: anlikDegerlerBaslik
                anchors.top: durumKarti.bottom
```

```
                    anchors.left: parent.left
                    anchors.topMargin: 24
                    anchors.leftMargin: 20
                    text: txt("anlikDegerler")
                    color: "#6b7280"
                    font.family: "Segoe UI"
                    font.pixelSize: 12
                    font.bold: true
                    font.letterSpacing: 1
                }

                Grid {
                    anchors.top: anlikDegerlerBaslik.bottom
                    anchors.left: parent.left
                    anchors.topMargin: 12
                    anchors.leftMargin: 20
                    columns: 2
                    spacing: 10

                    Rectangle {
                        id: basincKarti
                        width: 115
                        height: 90
                        radius: 10
                        color: "#0a0e17"
                        border.color: "#1e2a3f"
                        border.width: 1
                        clip: true

                        Rectangle { width: 4; height: parent.height; color: "#3b82f6" }

                        Column {
                            anchors.top: parent.top
                            anchors.right: parent.right
                            anchors.bottom: parent.bottom
                            anchors.left: parent.left
                            anchors.topMargin: 10
                            anchors.rightMargin: 10
                            anchors.bottomMargin: 10
                            anchors.leftMargin: 16
                            spacing: 6

                            Text {
                                text: txt("basincEtiket")
                                color: "#6b7280"
                                font.family: "Segoe UI"
                                font.pixelSize: 10
                                font.letterSpacing: 1
                            }

                            Text {
                                text: sensorManager.veriGecerli ? sensorManager.basinc.toFixed(1) : "--"
                                color: "#3b82f6"
                                font.family: "Segoe UI"
                                font.pixelSize: 22
                                font.bold: true
                            }

                            Text {
                                text: "mbar"
                                color: "#4b5563"
                                font.family: "Segoe UI"
                                font.pixelSize: 10
                            }
                        }
                    }

                    Rectangle {
                        width: 115
                        height: 90
                        radius: 10
                        color: "#0a0e17"
                        border.color: "#1e2a3f"
                        border.width: 1
                        clip: true

                        Rectangle { width: 4; height: parent.height; color: "#9333ea" }

                        Column {
                            anchors.top: parent.top
```

```
                    anchors.right: parent.right
                    anchors.bottom: parent.bottom
                    anchors.left: parent.left
                    anchors.topMargin: 10
                    anchors.rightMargin: 10
                    anchors.bottomMargin: 10
                    anchors.leftMargin: 16
                    spacing: 6

                    Text {
                        text: txt("konumEtiket")
                        color: "#6b7280"
                        font.family: "Segoe UI"
                        font.pixelSize: 10
                        font.letterSpacing: 1
                    }

                    Text {
                        text: sensorManager.veriGecerli ? sensorManager.konum.toFixed(1) : "--"
                        color: "#9333ea"
                        font.family: "Segoe UI"
                        font.pixelSize: 22
                        font.bold: true
                    }

                    Text {
                        text: "mm"
                        color: "#4b5563"
                        font.family: "Segoe UI"
                        font.pixelSize: 10
                    }
                }
            }

            Rectangle {
                width: 115
                height: 90
                radius: 10
                color: "#0a0e17"
                border.color: "#1e2a3f"
                border.width: 1
                clip: true

                Rectangle { width: 4; height: parent.height; color: "#f59e0b" }

                Column {
                    anchors.top: parent.top
                    anchors.right: parent.right
                    anchors.bottom: parent.bottom
                    anchors.left: parent.left
                    anchors.topMargin: 10
                    anchors.rightMargin: 10
                    anchors.bottomMargin: 10
                    anchors.leftMargin: 16
                    spacing: 6

                    Text {
                        text: txt("hizEtiket")
                        color: "#6b7280"
                        font.family: "Segoe UI"
                        font.pixelSize: 10
                        font.letterSpacing: 1
                    }

                    Text {
                        text: sensorManager.veriGecerli ? sensorManager.hiz.toFixed(1) : "--"
                        color: "#f59e0b"
                        font.family: "Segoe UI"
                        font.pixelSize: 22
                        font.bold: true
                    }

                    Text {
                        text: "m/s"
                        color: "#4b5563"
                        font.family: "Segoe UI"
                        font.pixelSize: 10
                    }
                }
            }
```

```

                Rectangle {
                    width: 115
                    height: 90
                    radius: 10
                    color: "#0a0e17"
                    border.color: "#1e2a3f"
                    border.width: 1
                    clip: true

                    Rectangle { width: 4; height: parent.height; color: "#16a34a" }

                    Column {
                        anchors.top: parent.top
                        anchors.right: parent.right
                        anchors.bottom: parent.bottom
                        anchors.left: parent.left
                        anchors.topMargin: 10
                        anchors.rightMargin: 10
                        anchors.bottomMargin: 10
                        anchors.leftMargin: 16
                        spacing: 6

                        Text {
                            text: txt("debiEtiket")
                            color: "#6b7280"
                            font.family: "Segoe UI"
                            font.pixelSize: 10
                            font.letterSpacing: 1
                        }

                        Text {
                            text: sensorManager.veriGecerli ? sensorManager.debi.toFixed(1) : "--"
                            color: "#16a34a"
                            font.family: "Segoe UI"
                            font.pixelSize: 22
                            font.bold: true
                        }

                        Text {
                            text: "m3/h"
                            color: "#4b5563"
                            font.family: "Segoe UI"
                            font.pixelSize: 10
                        }
                    }
                }
            }
        }

        Rectangle {
            id: panelAyraci
            width: 3
            height: parent.height
            color: "#05070b"

            Rectangle {
                anchors.right: parent.right
                anchors.top: parent.top
                anchors.bottom: parent.bottom
                width: 1
                color: "#403b82f6"
            }
        }

        Item {
            width: parent.width - testBilgileriPaneli.width - panelAyraci.width
            height: parent.height

            Grid {
                anchors.fill: parent
                anchors.margins: 16
                columns: 2
                spacing: 16

                Rectangle {
                    width: (parent.width - parent.spacing) / 2
                    height: (parent.height - parent.spacing) / 2
                    radius: 10
                    color: "#0f1420"
```

```
                    border.color: "#17263d"
                    border.width: 1

                    Item {
                        id: basincBaslik
                        anchors.top: parent.top
                        anchors.left: parent.left
                        anchors.right: parent.right
                        anchors.margins: 14
                        height: 20

                        Rectangle {
                            id: basincNoktasi
                            width: 8
                            height: 8
                            radius: 4
                            color: "#3b82f6"
                            anchors.left: parent.left
                            anchors.verticalCenter: parent.verticalCenter
                        }

                        Text {
                            anchors.left: basincNoktasi.right
                            anchors.leftMargin: 8
                            anchors.verticalCenter: parent.verticalCenter
                            text: txt("basincZaman")
                            color: "#9ca3af"
                            font.family: "Segoe UI"
                            font.pixelSize: 13
                            font.bold: true
                        }

                        Text {
                            anchors.right: parent.right
                            anchors.verticalCenter: parent.verticalCenter
                            text: sensorManager.veriGecerli ? sensorManager.basinc.toFixed(1) + " mbar" : "-- mbar"
                            color: "#3b82f6"
                            font.family: "Segoe UI"
                            font.pixelSize: 13
                            font.bold: true
                        }
                    }

                    ChartView {
                        anchors.top: basincBaslik.bottom
                        anchors.left: parent.left
                        anchors.right: parent.right
                        anchors.bottom: parent.bottom
                        anchors.topMargin: 4
                        anchors.bottomMargin: 4
                        backgroundColor: "transparent"
                        legend.visible: false
                        antialiasing: true

                        ValueAxis {
                            id: xEkseni
                            min: 0
                            max: 60
                            gridLineColor: "#182131"
                            labelsColor: "#4b5563"
                            labelsFont.pixelSize: 9
                            lineVisible: false
                            minorGridVisible: false
                        }

                        ValueAxis {
                            id: yEkseni
                            min: 0
                            max: 1100
                            gridLineColor: "#182131"
                            labelsColor: "#4b5563"
                            labelsFont.pixelSize: 9
                            lineVisible: false
                            minorGridVisible: false
                        }

                        LineSeries {
                            id: basincSerisi
                            axisX: xEkseni
                            axisY: yEkseni
```

```
                        color: "#3b82f6"
                        width: 2
                    }
                }
            }

            Rectangle {
                width: (parent.width - parent.spacing) / 2
                height: (parent.height - parent.spacing) / 2
                radius: 10
                color: "#0f1420"
                border.color: "#241a38"
                border.width: 1

                Item {
                    id: konumBaslik
                    anchors.top: parent.top
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.margins: 14
                    height: 20

                    Rectangle {
                        id: konumNoktasi
                        width: 8
                        height: 8
                        radius: 4
                        color: "#9333ea"
                        anchors.left: parent.left
                        anchors.verticalCenter: parent.verticalCenter
                    }

                    Text {
                        anchors.left: konumNoktasi.right
                        anchors.leftMargin: 8
                        anchors.verticalCenter: parent.verticalCenter
                        text: txt("konumZaman")
                        color: "#9ca3af"
                        font.family: "Segoe UI"
                        font.pixelSize: 13
                        font.bold: true
                    }

                    Text {
                        anchors.right: parent.right
                        anchors.verticalCenter: parent.verticalCenter
                        text: sensorManager.veriGecerli ? sensorManager.konum.toFixed(1) + " mm" : "-- mm"
                        color: "#9333ea"
                        font.family: "Segoe UI"
                        font.pixelSize: 13
                        font.bold: true
                    }
                }

                ChartView {
                    anchors.top: konumBaslik.bottom
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.bottom: parent.bottom
                    anchors.topMargin: 4
                    anchors.bottomMargin: 4
                    backgroundColor: "transparent"
                    legend.visible: false
                    antialiasing: true

                    ValueAxis {
                        id: konumXEkseni
                        min: 0
                        max: 60
                        gridLineColor: "#182131"
                        labelsColor: "#4b5563"
                        labelsFont.pixelSize: 9
                        lineVisible: false
                        minorGridVisible: false
                    }

                    ValueAxis {
                        id: konumYEkseni
                        min: 0
                        max: 500
```

```
                            gridLineColor: "#182131"
                            labelsColor: "#4b5563"
                            labelsFont.pixelSize: 9
                            lineVisible: false
                            minorGridVisible: false
                        }

                        LineSeries {
                            id: konumSerisi
                            axisX: konumXEkseni
                            axisY: konumYEkseni
                            color: "#9333ea"
                            width: 2
                        }

                        Connections {
                            target: calculator
                            function onStrokeSayisiChanged() {
                                if (aktifOlcumId > 0 && sensorManager.veriGecerli) {
                                    // gecerli parametresi artik Calculator'un hiz>0 kontrolunden geliyor
                                    // (orijinal SLIPER'daki "invalid stroke" mantigi), sabit "true" degil.
                                    database.strokeKaydet(aktifOlcumId, sensorManager.basinc, sensorManager.konum, sensorManager.debi, calculator.sonStrokeGecerli)
                                }
                            }
                        }
                    }
                }

                Rectangle {
                    width: (parent.width - parent.spacing) / 2
                    height: (parent.height - parent.spacing) / 2
                    radius: 10
                    color: "#0f1420"
                    border.color: "#3a2a14"
                    border.width: 1

                    Item {
                        id: hizBaslik
                        anchors.top: parent.top
                        anchors.left: parent.left
                        anchors.right: parent.right
                        anchors.margins: 14
                        height: 20

                        Rectangle {
                            id: hizNoktasi
                            width: 8
                            height: 8
                            radius: 4
                            color: "#f59e0b"
                            anchors.left: parent.left
                            anchors.verticalCenter: parent.verticalCenter
                        }

                        Text {
                            anchors.left: hizNoktasi.right
                            anchors.leftMargin: 8
                            anchors.verticalCenter: parent.verticalCenter
                            text: txt("hizZaman")
                            color: "#9ca3af"
                            font.family: "Segoe UI"
                            font.pixelSize: 13
                            font.bold: true
                        }

                        Text {
                            anchors.right: parent.right
                            anchors.verticalCenter: parent.verticalCenter
                            text: sensorManager.veriGecerli ? sensorManager.hiz.toFixed(1) + " m/s" : "-- m/s"
                            color: "#f59e0b"
                            font.family: "Segoe UI"
                            font.pixelSize: 13
                            font.bold: true
                        }
                    }

                    ChartView {
                        anchors.top: hizBaslik.bottom
                        anchors.left: parent.left
```

```
                anchors.right: parent.right
                anchors.bottom: parent.bottom
                anchors.topMargin: 4
                anchors.bottomMargin: 4
                backgroundColor: "transparent"
                legend.visible: false
                antialiasing: true

                ValueAxis {
                    id: hizXEkseni
                    min: 0
                    max: 60
                    gridLineColor: "#182131"
                    labelsColor: "#4b5563"
                    labelsFont.pixelSize: 9
                    lineVisible: false
                    minorGridVisible: false
                }

                ValueAxis {
                    id: hizYEkseni
                    min: 0
                    max: 4
                    gridLineColor: "#182131"
                    labelsColor: "#4b5563"
                    labelsFont.pixelSize: 9
                    lineVisible: false
                    minorGridVisible: false
                }

                LineSeries {
                    id: hizSerisi
                    axisX: hizXEkseni
                    axisY: hizYEkseni
                    color: "#f59e0b"
                    width: 2
                }
            }
        }

        Rectangle {
            width: (parent.width - parent.spacing) / 2
            height: (parent.height - parent.spacing) / 2
            radius: 10
            color: "#0f1420"
            border.color: "#163321"
            border.width: 1

            Item {
                id: debiBaslik
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.margins: 14
                height: 20

                Rectangle {
                    id: debiNoktasi
                    width: 8
                    height: 8
                    radius: 4
                    color: "#16a34a"
                    anchors.left: parent.left
                    anchors.verticalCenter: parent.verticalCenter
                }

                Text {
                    anchors.left: debiNoktasi.right
                    anchors.leftMargin: 8
                    anchors.verticalCenter: parent.verticalCenter
                    text: txt("debiZaman")
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 13
                    font.bold: true
                }

                Text {
                    anchors.right: parent.right
                    anchors.verticalCenter: parent.verticalCenter
```

```
                            text: sensorManager.veriGecerli ? sensorManager.debi.toFixed(1) + " m3/h" : "-- m3/h"
                            color: "#16a34a"
                            font.family: "Segoe UI"
                            font.pixelSize: 13
                            font.bold: true
                        }
                    }

                    ChartView {
                        anchors.top: debiBaslik.bottom
                        anchors.left: parent.left
                        anchors.right: parent.right
                        anchors.bottom: parent.bottom
                        anchors.topMargin: 4
                        anchors.bottomMargin: 4
                        backgroundColor: "transparent"
                        legend.visible: false
                        antialiasing: true

                        ValueAxis {
                            id: debiXEkseni
                            min: 0
                            max: 60
                            gridLineColor: "#182131"
                            labelsColor: "#4b5563"
                            labelsFont.pixelSize: 9
                            lineVisible: false
                            minorGridVisible: false
                        }

                        ValueAxis {
                            id: debiYEkseni
                            min: 0
                            max: 180
                            gridLineColor: "#182131"
                            labelsColor: "#4b5563"
                            labelsFont.pixelSize: 9
                            lineVisible: false
                            minorGridVisible: false
                        }

                        LineSeries {
                            id: debiSerisi
                            axisX: debiXEkseni
                            axisY: debiYEkseni
                            color: "#16a34a"
                            width: 2
                        }
                    }
                }
            }
        }
    }

    // Sesli komut geri bildirimi: eller kirliyken dokunmadan "Başlat /
    // Durdur / Bitir" denildiginde ne duyuldugunu gostererek kullaniciya
    // guven verir.
    Rectangle {
        id: sesliKomutRozeti
        visible: voiceCommandManager.etkin
        anchors.top: parent.top
        anchors.right: parent.right
        anchors.margins: 16
        width: sesliKomutIcerik.implicitWidth + 24
        height: sesliKomutIcerik.implicitHeight + 16
        radius: 10
        color: "#0a0e17"
        border.color: voiceCommandManager.dinliyor ? "#16a34a" : "#1e2a3f"
        border.width: 1
        z: 10

        Column {
            id: sesliKomutIcerik
            anchors.centerIn: parent
            spacing: 4

            Row {
                spacing: 6
                Rectangle {
                    width: 8
```

```
                    height: 8
                    radius: 4
                    anchors.verticalCenter: parent.verticalCenter
                    color: voiceCommandManager.dinliyor ? "#16a34a" : "#6b7280"
                }
                Text {
                    text: voiceCommandManager.dinliyor ? txt("sesliDinleniyor") : txt("sesliHazirlaniyor")
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 11
                }
            }

            Text {
                visible: voiceCommandManager.anlikMetin.length > 0
                text: txt("duyulanEtiket") + voiceCommandManager.anlikMetin
                color: "#4b5563"
                font.family: "Segoe UI"
                font.pixelSize: 10
                font.italic: true
            }
        }
    }

    Popup {
        id: bitirOnayPopup
        modal: true
        focus: true
        anchors.centerIn: parent
        width: 340
        padding: 24
        closePolicy: Popup.CloseOnEscape | Popup.CloseOnPressOutside

        background: Rectangle {
            color: "#0f1420"
            radius: 12
            border.color: "#1e2a3f"
            border.width: 1
        }

        Overlay.modal: Rectangle {
            color: "#a6000000"
        }

        contentItem: Column {
            spacing: 20

            Text {
                width: 292
                text: txt("bitirOnaySorusu")
                color: "#dce8f5"
                font.family: "Segoe UI"
                font.pixelSize: 14
                font.bold: true
                wrapMode: Text.WordWrap
            }

            Column {
                width: 292
                spacing: 10

                Button {
                    id: kaydetBitirButonu
                    width: parent.width
                    height: 40
                    text: txt("kaydetVeBitir")
                    font.family: "Segoe UI"
                    font.pixelSize: 13

                    onClicked: {
                        bitirOnayPopup.close()
                        var bitenId = aktifOlcumId
                        console.log("Olcum kaydedilip sonlandirildi, id:", bitenId)
                        resetMeasurementScreen()
                        olcumTamamlandi(bitenId)
                    }

                    background: Rectangle {
                        radius: 8
                        color: kaydetBitirButonu.pressed ? "#14532d" : (kaydetBitirButonu.hovered ? "#15803d" : "#16a34a")
```

```
                        border.color: "#22c55e"
                        border.width: 1
                    }

                    contentItem: Text {
                        text: kaydetBitirButonu.text
                        color: "#ffffff"
                        font: kaydetBitirButonu.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Button {
                    id: silBitirButonu
                    width: parent.width
                    height: 40
                    text: txt("testiSilVeBitir")
                    font.family: "Segoe UI"
                    font.pixelSize: 13

                    onClicked: {
                        bitirOnayPopup.close()
                        var silinecekId = aktifOlcumId
                        var basarili = database.olcumSil(silinecekId)
                        console.log("Olcum silinip sonlandirildi, id:", silinecekId, "basarili:", basarili)
                        resetMeasurementScreen()
                    }

                    background: Rectangle {
                        radius: 8
                        color: silBitirButonu.pressed ? "#7f1d1d" : (silBitirButonu.hovered ? "#b91c1c" : "#991b1b")
                        border.color: "#dc2626"
                        border.width: 1
                    }

                    contentItem: Text {
                        text: silBitirButonu.text
                        color: "#ffffff"
                        font: silBitirButonu.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Button {
                    id: vazgecButonu
                    width: parent.width
                    height: 40
                    text: txt("vazgecTesteDevam")
                    font.family: "Segoe UI"
                    font.pixelSize: 13

                    onClicked: {
                        bitirOnayPopup.close()
                        if (bitirIcinDuraklatildi) {
                            calculator.devamEt()
                            bitirIcinDuraklatildi = false
                        }
                    }

                    background: Rectangle {
                        radius: 8
                        color: vazgecButonu.pressed ? "#1e2a3f" : (vazgecButonu.hovered ? "#243349" : "#182131")
                        border.color: "#2c3b52"
                        border.width: 1
                    }

                    contentItem: Text {
                        text: vazgecButonu.text
                        color: "#dce8f5"
                        font: vazgecButonu.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }
}
```

qml/pages/SonuclarPage.qml

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtCharts 6.7
import sliper

Rectangle {
    color: "#0a0e17"

    property int olcumId: -1
    property real hesaplananTau0: 0
    property real hesaplananMu: 0
    property int playbackAdimi: -1
    property bool oynatiliyor: false
    property int playbackHizi: 1
    property string musteriAdi: ""
    property string receteAdi: ""
    property real agirlikDegeri: 0
    property string olcumTarihi: ""

    readonly property var metinler: ({
        sonucOzeti: { tr: "SONUÇ ÖZETİ", en: "RESULT SUMMARY" },
        akmaGerilmesi: { tr: "AKMA GERİLMESİ (τ₀)", en: "YIELD STRESS (τ₀)" },
        plastikViskozite: { tr: "PLASTİK VİSKOZİTE (μ)", en: "PLASTIC VISCOSITY (μ)" },
        uyumKalitesi: { tr: "UYUM KALİTESİ (R²)", en: "FIT QUALITY (R²)" },
        boruHattiTahmini: { tr: "BORU HATTI TAHMİNİ", en: "PIPELINE ESTIMATE" },
        hedefBoruCapi: { tr: "Hedef Boru Çapı (mm)", en: "Target Pipe Diameter (mm)" },
        boruUzunlugu: { tr: "Boru Uzunluğu (m) - karşılaştırma referansı", en: "Pipe Length (m) - comparison reference" },
        hedefDebi: { tr: "Hedef Debi (m³/h)", en: "Target Flow Rate (m³/h)" },
        hataPayi: { tr: "Hata Payı:", en: "Margin of Error:" },
        tahminiHesapla: { tr: "Tahmini Hesapla", en: "Calculate Estimate" },
        gecerliDegerGirin: { tr: "Geçerli değer girin", en: "Enter a valid value" },
        hesaplanamadi: { tr: "Hesaplanamadı", en: "Could not calculate" },
        pompalanabilir: { tr: "POMPALANABİLİR", en: "PUMPABLE" },
        pompalamaSorunu: { tr: "POMPALAMA SORUNU", en: "PUMPING ISSUE" },
        durumHerIkisiYuksek: { tr: "Hem akma gerilmesi hem plastik viskozite yüksek. Su/çimento oranını artırın ve agrega gradasyonunu gözden geçirin.", en: "Both yield stress and plastic viscosity are high. Increase the water/cement ratio and review the aggregate gradation." },
        durumTau0Yuksek: { tr: "Akma gerilmesi yüksek. Su/çimento oranını artırmayı veya süperakışkanlaştırıcı dozajını yükseltmeyi değerlendirin.", en: "Yield stress is high. Consider increasing the water/cement ratio or raising the superplasticizer dosage." },
        durumMuYuksek: { tr: "Plastik viskozite yüksek. İnce agrega oranını azaltmayı veya su azaltıcı katkı eklemeyi değerlendirin.", en: "Plastic viscosity is high. Consider reducing the fine aggregate ratio or adding a water-reducing admixture." },
        durumYetersizVeri: { tr: "Yeterli stroke verisi bulunamadı.", en: "Not enough stroke data found." },
        pdfRaporOnizle: { tr: "📄 PDF Rapor Önizle", en: "📄 Preview PDF Report" },
        excelCsvOnizle: { tr: "📊 Excel (CSV) Önizle", en: "📊 Preview Excel (CSV)" },
        excelXmlDisaAktar: { tr: "📑 Excel (XML) Dışa Aktar", en: "📑 Export Excel (XML)" },
        disaAktarildi: { tr: "✓ Dışa aktarıldı: ", en: "✓ Exported: " },
        disaAktarilamadi: { tr: "✕ Dışa aktarılamadı", en: "✕ Export failed" },
        oynatmaPlayback: { tr: "OYNATMA (PLAYBACK)", en: "PLAYBACK" },
        strokeEtiket: { tr: "Stroke", en: "Stroke" },
        oynatilacakStrokeYok: { tr: "Oynatilacak stroke yok", en: "No stroke to play back" },
        pqDagilimGrafigi: { tr: "P-Q Dağılım Grafiği", en: "P-Q Distribution Chart" },
        olcumNoktalari: { tr: "Ölçüm Noktaları", en: "Measurement Points" },
        regresyonDogrusu: { tr: "Regresyon Doğrusu", en: "Regression Line" },
        oynatmaEtiket: { tr: "Oynatma", en: "Playback" },
        strokeTablosu: { tr: "Stroke Tablosu", en: "Stroke Table" },
        strokeBaslik: { tr: "STROKE", en: "STROKE" },
        durumBaslik: { tr: "DURUM", en: "STATUS" },
        gecerliEtiket: { tr: "Geçerli", en: "Valid" },
        hataliEtiket: { tr: "Hatalı", en: "Invalid" },
        pdfRaporOnizlemeBaslik: { tr: "PDF Rapor Önizleme", en: "PDF Report Preview" },
        indirPdf: { tr: "💾 İndir (PDF)", en: "💾 Download (PDF)" },
        pdfKaydedildi: { tr: "PDF kaydedildi: ", en: "PDF saved: " },
        pdfOlusturulamadi: { tr: "PDF oluşturulamadı.", en: "Could not generate PDF." },
        kapat: { tr: "Kapat", en: "Close" },
        excelCsvOnizlemeBaslik: { tr: "Excel (CSV) Önizleme", en: "Excel (CSV) Preview" },
        olcumIdEtiket: { tr: "Ölçüm ID: ", en: "Measurement ID: " },
        tarihEtiket: { tr: "Tarih: ", en: "Date: " },
        musteriEtiket: { tr: "Müşteri: ", en: "Customer: " },
        receteEtiket: { tr: "Reçete: ", en: "Recipe: " },
        agirlikEtiket: { tr: "Ağırlık (kg): ", en: "Weight (kg): " },
        basincBaslik: { tr: "BASINÇ (mbar)", en: "PRESSURE (mbar)" },
        konumBaslik: { tr: "KONUM (mm)", en: "POSITION (mm)" },
        debiBuyukBaslik: { tr: "DEBİ (m³/h)", en: "FLOW (m³/h)" },
        gecerliBuyukBaslik: { tr: "GEÇERLİ", en: "VALID" },
        evet: { tr: "Evet", en: "Yes" },
        hayir: { tr: "Hayır", en: "No" },
        indirCsv: { tr: "💾 İndir (CSV)", en: "💾 Download (CSV)" },
        csvKaydedildi: { tr: "CSV kaydedildi: ", en: "CSV saved: " },
```

```
        csvOlusturulamadi: { tr: "CSV oluşturulamadı.", en: "Could not generate CSV." },
        bilinmiyor: { tr: "Bilinmiyor", en: "Unknown" }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    onOlcumIdChanged: verileriYukle()

    function verileriYukle() {
        if (olcumId <= 0) {
            return
        }

        playbackAdimi = -1
        oynatiliyor = false
        playbackTimer.stop()

        var bilgi = database.olcumBilgisiGetir(olcumId)
        musteriAdi = bilgi.bulundu ? bilgi.musteri : txt("bilinmiyor")
        receteAdi = bilgi.bulundu ? bilgi.recete : txt("bilinmiyor")
        agirlikDegeri = bilgi.bulundu ? bilgi.agirlik : 0
        olcumTarihi = bilgi.bulundu ? bilgi.tarih : ""

        var sonuc = database.binghamHesapla(olcumId)

        hesaplananTau0 = sonuc.tau0
        hesaplananMu = sonuc.mu

        tau0Metni.text = sonuc.tau0.toFixed(2) + " mbar"
        muMetni.text = sonuc.mu.toFixed(2) + " mbar·h/m³"
        r2Metni.text = sonuc.r2.toFixed(2)

        durumKutusu.durumIyiMi = sonuc.yeterliVeri && sonuc.tau0 < 5 && sonuc.mu < 10
        durumKutusu.tau0Yuksek = sonuc.tau0 >= 5
        durumKutusu.muYuksek = sonuc.mu >= 10

        pqSerisi.clear()
        regresyonCizgisi.clear()
        oynatmaSerisi.clear()
        strokeModeli.clear()

        var strokeListesi = database.strokeVerileriGetir(olcumId)
        var pMaks = 0
        var qMaks = 0
        for (var i = 0; i < strokeListesi.length; i++) {
            var s = strokeListesi[i]
            pqSerisi.append(s.debi, s.basinc)
            strokeModeli.append({ stroke: s.stroke, p: s.basinc, q: s.debi, gecerli: s.gecerli })
            if (s.basinc > pMaks) pMaks = s.basinc
            if (s.debi > qMaks) qMaks = s.debi
        }

        // Eksenler gerçek ölçüm verisine göre ayarlanır; sabit aralık kullanılırsa
        // basınç/debi bu aralığı aştığında noktalar (ve oynatma imleci) grafik
        // dışına taşıp görünmez olur.
        qMaks = Math.max(qMaks, 5)
        regresyonCizgisi.append(0, sonuc.tau0)
        regresyonCizgisi.append(qMaks, sonuc.tau0 + sonuc.mu * qMaks)

        pMaks = Math.max(pMaks, sonuc.tau0 + sonuc.mu * qMaks, 10)
        pEkseni.max = pMaks * 1.15
        qEkseni.max = qMaks * 1.15
    }

    Timer {
        id: playbackTimer
        interval: 1000 / playbackHizi
        repeat: true
        running: false

        onTriggered: {
            if (playbackAdimi < strokeModeli.count - 1) {
                playbackAdimi++
                var satir = strokeModeli.get(playbackAdimi)
                oynatmaSerisi.clear()
                oynatmaSerisi.append(satir.q, satir.p)
            } else {
                oynatiliyor = false
```

```
                playbackTimer.stop()
            }
        }
    }

    Row {
        anchors.fill: parent
        spacing: 0

        Rectangle {
            id: ozetPaneli
            width: 280
            height: parent.height
            color: "#0f1420"

            Text {
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.margins: 20
                text: txt("sonucOzeti")
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 12
                font.bold: true
                font.letterSpacing: 1
            }

            Flickable {
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.bottom: parent.bottom
                anchors.margins: 20
                anchors.topMargin: 60
                contentWidth: width
                contentHeight: sonucKartlari.height
                clip: true

                Column {
                    id: sonucKartlari
                    width: parent.width
                    spacing: 10

                    Rectangle {
                        width: parent.width
                        height: 70
                        radius: 10
                        color: "#0a0e17"
                        border.color: "#17263d"
                        border.width: 1
                        clip: true

                        Rectangle { width: 4; height: parent.height; color: "#3b82f6" }

                        Column {
                            anchors.left: parent.left
                            anchors.verticalCenter: parent.verticalCenter
                            anchors.leftMargin: 20
                            spacing: 2

                            Text {
                                text: txt("akmaGerilmesi")
                                color: "#6b7280"
                                font.family: "Segoe UI"
                                font.pixelSize: 10
                                font.letterSpacing: 1
                            }

                            Text {
                                id: tau0Metni
                                text: "–"
                                color: "#3b82f6"
                                font.family: "Segoe UI"
                                font.pixelSize: 20
                                font.bold: true
                            }
                        }
                    }

                    Rectangle {
```

```
                    width: parent.width
                    height: 70
                    radius: 10
                    color: "#0a0e17"
                    border.color: "#241a38"
                    border.width: 1
                    clip: true

                    Rectangle { width: 4; height: parent.height; color: "#9333ea" }

                    Column {
                        anchors.left: parent.left
                        anchors.verticalCenter: parent.verticalCenter
                        anchors.leftMargin: 20
                        spacing: 2

                        Text {
                            text: txt("plastikViskozite")
                            color: "#6b7280"
                            font.family: "Segoe UI"
                            font.pixelSize: 10
                            font.letterSpacing: 1
                        }

                        Text {
                            id: muMetni
                            text: "—"
                            color: "#9333ea"
                            font.family: "Segoe UI"
                            font.pixelSize: 20
                            font.bold: true
                        }
                    }
                }

                Rectangle {
                    width: parent.width
                    height: 70
                    radius: 10
                    color: "#0a0e17"
                    border.color: "#163321"
                    border.width: 1
                    clip: true

                    Rectangle { width: 4; height: parent.height; color: "#16a34a" }

                    Column {
                        anchors.left: parent.left
                        anchors.verticalCenter: parent.verticalCenter
                        anchors.leftMargin: 20
                        spacing: 2

                        Text {
                            text: txt("uyumKalitesi")
                            color: "#6b7280"
                            font.family: "Segoe UI"
                            font.pixelSize: 10
                            font.letterSpacing: 1
                        }

                        Text {
                            id: r2Metni
                            text: "—"
                            color: "#16a34a"
                            font.family: "Segoe UI"
                            font.pixelSize: 20
                            font.bold: true
                        }
                    }
                }

                Rectangle {
                    width: parent.width
                    radius: 10
                    color: "#0a0e17"
                    border.color: "#1e2a3f"
                    border.width: 1
                    height: tahminIcerik.implicitHeight + 24

                    Column {
```

```
                id: tahminIcerik
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.margins: 12
                spacing: 8

                Text {
                    text: txt("boruHattiTahmini")
                    color: "#6b7280"
                    font.family: "Segoe UI"
                    font.pixelSize: 10
                    font.letterSpacing: 1
                }

                TextField {
                    id: capKutusu
                    width: parent.width
                    height: 34
                    placeholderText: txt("hedefBoruCapi")
                    placeholderTextColor: "#4b5563"
                    color: "#ffffff"
                    font.pixelSize: 12
                    leftPadding: 10
                    text: "125"
                    verticalAlignment: TextInput.AlignVCenter
                    validator: DoubleValidator { bottom: 10; top: 500; decimals: 0 }
                    background: Rectangle {
                        color: "#060d17"
                        radius: 6
                        border.color: capKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                        border.width: 1
                    }
                }

                TextField {
                    id: uzunlukKutusu
                    width: parent.width
                    height: 34
                    placeholderText: txt("boruUzunlugu")
                    placeholderTextColor: "#4b5563"
                    color: "#ffffff"
                    font.pixelSize: 12
                    leftPadding: 10
                    verticalAlignment: TextInput.AlignVCenter
                    validator: DoubleValidator { bottom: 0; top: 2000; decimals: 0 }
                    background: Rectangle {
                        color: "#060d17"
                        radius: 6
                        border.color: uzunlukKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                        border.width: 1
                    }
                }

                TextField {
                    id: debiKutusu
                    width: parent.width
                    height: 34
                    placeholderText: txt("hedefDebi")
                    placeholderTextColor: "#4b5563"
                    color: "#ffffff"
                    font.pixelSize: 12
                    leftPadding: 10
                    verticalAlignment: TextInput.AlignVCenter
                    validator: DoubleValidator { bottom: 0; top: 200; decimals: 1 }
                    background: Rectangle {
                        color: "#060d17"
                        radius: 6
                        border.color: debiKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                        border.width: 1
                    }
                }

                Row {
                    width: parent.width
                    spacing: 8

                    Text {
                        text: txt("hataPayi")
                        color: "#9ca3af"
```

```
    font.family: "Segoe UI"
    font.pixelSize: 11
    anchors.verticalCenter: parent.verticalCenter
}

ComboBox {
    id: hataPayiSecici
    width: 90
    height: 28
    model: ["0", "10", "20"]
    currentIndex: 1

    background: Rectangle {
        color: "#060d17"
        radius: 6
        border.color: hataPayiSecici.activeFocus ? "#3b82f6" : "#1e2a3f"
        border.width: 1
    }

    contentItem: Text {
        text: hataPayiSecici.displayText
        color: "#dce8f5"
        font.family: "Segoe UI"
        font.pixelSize: 11
        leftPadding: 10
        verticalAlignment: Text.AlignVCenter
    }

    indicator: Text {
        x: hataPayiSecici.width - width - 8
        anchors.verticalCenter: parent.verticalCenter
        text: "▾"
        color: "#6b7280"
        font.pixelSize: 11
    }

    popup: Popup {
        y: hataPayiSecici.height + 2
        width: hataPayiSecici.width
        implicitHeight: contentItem.implicitHeight
        padding: 1

        background: Rectangle {
            color: "#0a0e17"
            radius: 6
            border.color: "#1e2a3f"
            border.width: 1
        }

        contentItem: ListView {
            implicitHeight: contentHeight
            model: hataPayiSecici.popup.visible ? hataPayiSecici.delegateModel : null
            currentIndex: hataPayiSecici.highlightedIndex
            clip: true
        }
    }

    delegate: ItemDelegate {
        id: hataPayiDelege
        width: hataPayiSecici.width
        height: 28
        highlighted: hataPayiSecici.highlightedIndex === index

        background: Rectangle {
            color: hataPayiDelege.highlighted ? "#182644" : "#0a0e17"
        }

        contentItem: Text {
            text: modelData
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 11
            leftPadding: 10
            verticalAlignment: Text.AlignVCenter
        }
    }
}

Text {
    text: "%"
```

```
        color: "#9ca3af"
        font.family: "Segoe UI"
        font.pixelSize: 11
        anchors.verticalCenter: parent.verticalCenter
    }
}

Button {
    width: parent.width
    height: 36
    text: txt("tahminiHesapla")
    font.pixelSize: 12
    font.bold: true

    background: Rectangle {
        radius: 6
        color: parent.hovered ? "#4f8cf7" : "#3b82f6"
        Behavior on color { ColorAnimation { duration: 120 } }
    }

    contentItem: Text {
        text: parent.text
        color: "#ffffff"
        font: parent.font
        horizontalAlignment: Text.AlignHCenter
        verticalAlignment: Text.AlignVCenter
    }

    onClicked: {
        var D = parseFloat(capKutusu.text)
        var L = parseFloat(uzunlukKutusu.text)
        var Q = parseFloat(debiKutusu.text)
        var tolerans = parseFloat(hataPayiSecici.currentText)

        if (isNaN(D) || isNaN(L) || isNaN(Q) || D <= 0 || L <= 0) {
            tahminSonucu.text = txt("gecerliDegerGirin")
            tahminKarsilastirma.text = ""
            return
        }

        var sonuc = calculator.boruHattiTahminHesapla(
            hesaplananTau0, hesaplananMu, Q, D, L, tolerans)

        if (!sonuc.gecerliGiris) {
            tahminSonucu.text = txt("hesaplanamadi")
            tahminKarsilastirma.text = ""
            return
        }

        tahminSonucu.text = sonuc.basincBar.toFixed(2) + " bar"
        if (tolerans > 0) {
            tahminSonucu.text += "  (±%1: %2 – %3 bar)".arg(tolerans.toFixed(0))
                .arg(sonuc.altSinirBar.toFixed(2)).arg(sonuc.ustSinirBar.toFixed(2))
        }

        // Aynı D/Q icin farkli boru uzunluklarinda karsilastirma
        // (orijinal SLIPER'daki "birden fazla boru uzunlugu" karsilastirmasi)
        var uzunluklar = [L * 0.5, L, L * 2]
        var satirlar = []
        for (var i = 0; i < uzunluklar.length; i++) {
            var s = calculator.boruHattiTahminHesapla(
                hesaplananTau0, hesaplananMu, Q, D, uzunluklar[i], 0)
            satirlar.push(uzunluklar[i].toFixed(0) + " m: " + s.basincBar.toFixed(2) + " bar")
        }
        tahminKarsilastirma.text = satirlar.join("   |   ")
    }
}

Text {
    id: tahminSonucu
    width: parent.width
    text: "—"
    color: "#e8a020"
    font.family: "Segoe UI"
    font.pixelSize: 18
    font.bold: true
    wrapMode: Text.WordWrap
    horizontalAlignment: Text.AlignHCenter
}

```

```
        Text {
            id: tahminKarsilastirma
            width: parent.width
            text: ""
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 10
            wrapMode: Text.WordWrap
            horizontalAlignment: Text.AlignHCenter
        }
    }
}

Rectangle {
    id: durumKutusu
    width: parent.width
    height: durumIcerik.implicitHeight + 24
    radius: 10
    color: durumIyiMi ? "#0f2417" : "#2a1414"
    border.color: durumIyiMi ? "#16a34a" : "#dc2626"
    border.width: 1

    property bool durumIyiMi: true
    property bool tau0Yuksek: false
    property bool muYuksek: false

    Column {
        id: durumIcerik
        anchors.top: parent.top
        anchors.horizontalCenter: parent.horizontalCenter
        anchors.topMargin: 12
        spacing: 8
        width: parent.width - 24

        Row {
            anchors.horizontalCenter: parent.horizontalCenter
            spacing: 10

            Rectangle {
                width: 24
                height: 24
                radius: 12
                anchors.verticalCenter: parent.verticalCenter
                color: durumKutusu.durumIyiMi ? "#16a34a" : "#dc2626"

                Text {
                    anchors.centerIn: parent
                    text: durumKutusu.durumIyiMi ? "✓" : "✗"
                    color: "#ffffff"
                    font.pixelSize: 13
                    font.bold: true
                }
            }

            Text {
                anchors.verticalCenter: parent.verticalCenter
                text: durumKutusu.durumIyiMi ? txt("pompalanabilir") : txt("pompalamaSorunu")
                color: durumKutusu.durumIyiMi ? "#4ade80" : "#f87171"
                font.family: "Segoe UI"
                font.pixelSize: 15
                font.bold: true
            }
        }

        Text {
            width: parent.width
            horizontalAlignment: Text.AlignHCenter
            visible: !durumKutusu.durumIyiMi
            text: {
                if (durumKutusu.tau0Yuksek && durumKutusu.muYuksek) {
                    return txt("durumHerIkisiYuksek")
                } else if (durumKutusu.tau0Yuksek) {
                    return txt("durumTau0Yuksek")
                } else if (durumKutusu.muYuksek) {
                    return txt("durumMuYuksek")
                }
                return txt("durumYetersizVeri")
            }
            color: "#9ca3af"
            font.family: "Segoe UI"
```

```
                        font.pixelSize: 11
                        wrapMode: Text.WordWrap
                    }
                }
            }

            Button {
                width: parent.width
                height: 40
                text: txt("pdfRaporOnizle")
                font.pixelSize: 13
                font.bold: true
                enabled: olcumId > 0

                onClicked: {
                    pdfOnizlemePopup.onizlemeHtml = reportManager.pdfOnizlemeHtml(
                        olcumId,
                        musteriAdi,
                        receteAdi,
                        hesaplananTau0,
                        hesaplananMu,
                        parseFloat(r2Metni.text)
                    )
                    pdfOnizlemePopup.open()
                }

                background: Rectangle {
                    radius: 8
                    color: parent.enabled ? (parent.hovered ? "#4f8cf7" : "#3b82f6") : "#1e2a3f"
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#ffffff"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Button {
                width: parent.width
                height: 40
                text: txt("excelCsvOnizle")
                font.pixelSize: 13
                font.bold: true
                enabled: olcumId > 0

                onClicked: {
                    csvOnizlemePopup.satirlar = database.strokeVerileriGetir(olcumId)
                    csvOnizlemePopup.open()
                }

                background: Rectangle {
                    radius: 8
                    color: parent.enabled ? (parent.hovered ? "#22c55e" : "#16a34a") : "#1e2a3f"
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#ffffff"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Button {
                width: parent.width
                height: 40
                text: txt("excelXmlDisaAktar")
                font.pixelSize: 13
                font.bold: true
                enabled: olcumId > 0

                onClicked: {
                    var yol = database.xmlDisaAktar(olcumId)
                    xmlDisaAktarBildirim.text = yol.length > 0
```

```
                            ? txt("disaAktarildi") + yol
                            : txt("disaAktarilamadi")
                        xmlDisaAktarBildirim.visible = true
                        xmlDisaAktarTimer.restart()
                    }

                    background: Rectangle {
                        radius: 8
                        color: parent.enabled ? (parent.hovered ? "#4f8cf7" : "#3b82f6") : "#1e2a3f"
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#ffffff"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Text {
                    id: xmlDisaAktarBildirim
                    width: parent.width
                    visible: false
                    text: ""
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 10
                    wrapMode: Text.WrapAnywhere
                    horizontalAlignment: Text.AlignHCenter

                    Timer {
                        id: xmlDisaAktarTimer
                        interval: 4000
                        onTriggered: xmlDisaAktarBildirim.visible = false
                    }
                }

                Rectangle {
                    width: parent.width
                    radius: 10
                    color: "#0a0e17"
                    border.color: "#1e2a3f"
                    border.width: 1
                    height: playbackIcerik.implicitHeight + 24
                    visible: olcumId > 0

                    Column {
                        id: playbackIcerik
                        anchors.top: parent.top
                        anchors.left: parent.left
                        anchors.right: parent.right
                        anchors.margins: 12
                        spacing: 8

                        Text {
                            text: txt("oynatmaPlayback")
                            color: "#6b7280"
                            font.family: "Segoe UI"
                            font.pixelSize: 10
                            font.letterSpacing: 1
                        }

                        Text {
                            text: strokeModeli.count > 0
                                  ? txt("strokeEtiket") + " " + (playbackAdimi + 1) + " / " + strokeModeli.count
                                  : txt("oynatilacakStrokeYok")
                            color: "#dce8f5"
                            font.family: "Segoe UI"
                            font.pixelSize: 12
                        }

                        Flow {
                            width: parent.width
                            spacing: 5

                            Button {
                                width: 40
                                height: 34
```

```
                                text: oynatiliyor ? "⏸" : "▶"
                                font.pixelSize: 14
                                enabled: strokeModeli.count > 0

                                onClicked: {
                                    if (oynatiliyor) {
                                        oynatiliyor = false
                                        playbackTimer.stop()
                                    } else {
                                        if (playbackAdimi >= strokeModeli.count - 1) {
                                            playbackAdimi = -1
                                        }
                                        oynatiliyor = true
                                        playbackTimer.start()
                                    }
                                }

                                background: Rectangle {
                                    radius: 6
                                    color: parent.enabled ? (parent.hovered ? "#4f8cf7" : "#3b82f6") : "#1e2a3f"
                                    Behavior on color { ColorAnimation { duration: 120 } }
                                }

                                contentItem: Text {
                                    text: parent.text
                                    color: "#ffffff"
                                    font: parent.font
                                    horizontalAlignment: Text.AlignHCenter
                                    verticalAlignment: Text.AlignVCenter
                                }
                            }

                            Repeater {
                                model: [1, 2, 5, 10]

                                Button {
                                    width: 38
                                    height: 34
                                    text: modelData + "x"
                                    font.pixelSize: 12

                                    onClicked: playbackHizi = modelData

                                    background: Rectangle {
                                        radius: 6
                                        color: playbackHizi === modelData ? "#3b82f6" : "#0f1420"
                                        border.color: "#1e2a3f"
                                        border.width: 1
                                    }

                                    contentItem: Text {
                                        text: parent.text
                                        color: playbackHizi === modelData ? "#ffffff" : "#9ca3af"
                                        font: parent.font
                                        horizontalAlignment: Text.AlignHCenter
                                        verticalAlignment: Text.AlignVCenter
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }

    Rectangle {
        id: panelAyraci
        width: 3
        height: parent.height
        color: "#05070b"

        Rectangle {
            anchors.right: parent.right
            anchors.top: parent.top
            anchors.bottom: parent.bottom
            width: 1
            color: "#403b82f6"
        }
    }
```

```
        Item {
            width: parent.width - ozetPaneli.width - panelAyraci.width
            height: parent.height

            Rectangle {
                id: grafikKutusu
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.margins: 16
                height: parent.height * 0.55
                radius: 10
                color: "#0f1420"
                border.color: "#1e2a3f"
                border.width: 1

                Item {
                    id: grafikBaslik
                    anchors.top: parent.top
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.margins: 14
                    height: 20

                    Text {
                        anchors.left: parent.left
                        anchors.verticalCenter: parent.verticalCenter
                        text: txt("pqDagilimGrafigi")
                        color: "#9ca3af"
                        font.family: "Segoe UI"
                        font.pixelSize: 13
                        font.bold: true
                    }

                    Row {
                        anchors.right: parent.right
                        anchors.verticalCenter: parent.verticalCenter
                        spacing: 16

                        Row {
                            spacing: 6
                            anchors.verticalCenter: parent.verticalCenter

                            Rectangle {
                                width: 8
                                height: 8
                                radius: 4
                                color: "#3b82f6"
                                anchors.verticalCenter: parent.verticalCenter
                            }

                            Text {
                                text: txt("olcumNoktalari")
                                color: "#6b7280"
                                font.family: "Segoe UI"
                                font.pixelSize: 11
                                anchors.verticalCenter: parent.verticalCenter
                            }
                        }

                        Row {
                            spacing: 6
                            anchors.verticalCenter: parent.verticalCenter

                            Rectangle {
                                width: 12
                                height: 2
                                color: "#e8a020"
                                anchors.verticalCenter: parent.verticalCenter
                            }

                            Text {
                                text: txt("regresyonDogrusu")
                                color: "#6b7280"
                                font.family: "Segoe UI"
                                font.pixelSize: 11
                                anchors.verticalCenter: parent.verticalCenter
                            }
                        }
```

```
                        Row {
                            spacing: 6
                            anchors.verticalCenter: parent.verticalCenter

                            Rectangle {
                                width: 8
                                height: 8
                                radius: 4
                                color: "#e8a020"
                                anchors.verticalCenter: parent.verticalCenter
                            }

                            Text {
                                text: txt("oynatmaEtiket")
                                color: "#6b7280"
                                font.family: "Segoe UI"
                                font.pixelSize: 11
                                anchors.verticalCenter: parent.verticalCenter
                            }
                        }
                    }
                }

                ChartView {
                    anchors.top: grafikBaslik.bottom
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.bottom: parent.bottom
                    anchors.topMargin: 4
                    anchors.margins: 8
                    backgroundColor: "transparent"
                    legend.visible: false
                    antialiasing: true

                    ValueAxis {
                        id: qEkseni
                        min: 0
                        max: 20
                        titleText: Translations.turkish ? "Debi (Q) m³/h" : "Flow Rate (Q) m³/h"
                        gridLineColor: "#182131"
                        labelsColor: "#4b5563"
                        labelsFont.pixelSize: 9
                        lineVisible: false
                        minorGridVisible: false
                    }

                    ValueAxis {
                        id: pEkseni
                        min: 0
                        max: 100
                        titleText: Translations.turkish ? "Basınç (P) mbar" : "Pressure (P) mbar"
                        gridLineColor: "#182131"
                        labelsColor: "#4b5563"
                        labelsFont.pixelSize: 9
                        lineVisible: false
                        minorGridVisible: false
                    }

                    ScatterSeries {
                        id: pqSerisi
                        axisX: qEkseni
                        axisY: pEkseni
                        color: "#3b82f6"
                        markerSize: 10
                    }

                    LineSeries {
                        id: regresyonCizgisi
                        axisX: qEkseni
                        axisY: pEkseni
                        color: "#e8a020"
                        width: 2
                    }

                    ScatterSeries {
                        id: oynatmaSerisi
                        axisX: qEkseni
                        axisY: pEkseni
                        color: "#e8a020"
```

```
                        markerSize: 16
                    }
                }
            }

            Rectangle {
                anchors.top: grafikKutusu.bottom
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.bottom: parent.bottom
                anchors.margins: 16
                anchors.topMargin: 8
                radius: 10
                color: "#0f1420"
                border.color: "#1e2a3f"
                border.width: 1

                Text {
                    anchors.top: parent.top
                    anchors.left: parent.left
                    anchors.margins: 14
                    text: txt("strokeTablosu")
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 13
                    font.bold: true
                }

                Row {
                    id: tabloBasligi
                    anchors.top: parent.top
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.topMargin: 36
                    anchors.leftMargin: 12
                    anchors.rightMargin: 12

                    Text { width: parent.width * 0.25; text: txt("strokeBaslik"); color: "#6b7280"; font.pixelSize: 11; font.bold: true; font.letterSpacing: 1 }
                    Text { width: parent.width * 0.25; text: "P (mbar)"; color: "#6b7280"; font.pixelSize: 11; font.bold: true; font.letterSpacing: 1 }
                    Text { width: parent.width * 0.25; text: "Q (m³/h)"; color: "#6b7280"; font.pixelSize: 11; font.bold: true; font.letterSpacing: 1 }
                    Text { width: parent.width * 0.25; text: txt("durumBaslik"); color: "#6b7280"; font.pixelSize: 11; font.bold: true; font.letterSpacing: 1 }
                }

                Rectangle {
                    anchors.top: tabloBasligi.bottom
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.topMargin: 8
                    anchors.leftMargin: 12
                    anchors.rightMargin: 12
                    height: 1
                    color: "#1e2a3f"
                }

                ListView {
                    anchors.top: tabloBasligi.bottom
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.bottom: parent.bottom
                    anchors.margins: 12
                    anchors.topMargin: 14
                    clip: true

                    model: ListModel {
                        id: strokeModeli
                    }

                    delegate: Rectangle {
                        width: parent.width
                        height: 32
                        radius: 6
                        color: index === playbackAdimi ? "#1e2a3f" : (index % 2 === 0 ? "transparent" : "#0a0e17")

                        Row {
                            anchors.fill: parent
                            anchors.leftMargin: 6
```

```
                            Text { width: parent.width * 0.25; anchors.verticalCenter: parent.verticalCenter; text: stroke; color: "#dce8f5"; font.pixelSize: 12 }
                            Text { width: parent.width * 0.25; anchors.verticalCenter: parent.verticalCenter; text: p.toFixed(1); color: "#dce8f5"; font.pixelSize: 12 }
                            Text { width: parent.width * 0.25; anchors.verticalCenter: parent.verticalCenter; text: q.toFixed(2); color: "#dce8f5"; font.pixelSize: 12 }

                            Item {
                                width: parent.width * 0.25
                                height: parent.height

                                Rectangle {
                                    anchors.verticalCenter: parent.verticalCenter
                                    width: durumMetni.implicitWidth + 16
                                    height: 20
                                    radius: 10
                                    color: gecerli ? "#123321" : "#2a1414"

                                    Text {
                                        id: durumMetni
                                        anchors.centerIn: parent
                                        text: gecerli ? txt("gecerliEtiket") : txt("hataliEtiket")
                                        color: gecerli ? "#4ade80" : "#f87171"
                                        font.pixelSize: 11
                                        font.bold: true
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }

    Rectangle {
        id: bildirimKutusu
        property bool hataMi: false

        function goster(mesaj, hata) {
            bildirimMetni.text = mesaj
            hataMi = hata
            opacity = 1
            bildirimTimer.restart()
        }

        visible: opacity > 0
        opacity: 0
        z: 100
        anchors.bottom: parent.bottom
        anchors.horizontalCenter: parent.horizontalCenter
        anchors.bottomMargin: 24
        width: bildirimMetni.implicitWidth + 40
        height: 44
        radius: 10
        color: hataMi ? "#7f1d1d" : "#14532d"
        border.color: hataMi ? "#dc2626" : "#22c55e"
        border.width: 1

        Behavior on opacity { NumberAnimation { duration: 200 } }

        Text {
            id: bildirimMetni
            anchors.centerIn: parent
            text: ""
            color: "#ffffff"
            font.family: "Segoe UI"
            font.pixelSize: 13
            font.bold: true
        }

        Timer {
            id: bildirimTimer
            interval: 3000
            onTriggered: bildirimKutusu.opacity = 0
        }
    }

    Popup {
```

```
        id: pdfOnizlemePopup
        modal: true
        focus: true
        anchors.centerIn: parent
        width: 640
        height: Math.min(720, parent.height - 40)
        padding: 0
        closePolicy: Popup.CloseOnEscape

        property string onizlemeHtml: ""

        background: Rectangle {
            color: "#0f1420"
            radius: 12
            border.color: "#1e2a3f"
            border.width: 1
        }

        Overlay.modal: Rectangle {
            color: "#a6000000"
        }

        contentItem: Column {
            width: pdfOnizlemePopup.width
            spacing: 0

            Rectangle {
                width: parent.width
                height: 52
                color: "#0a0e17"

                Text {
                    anchors.left: parent.left
                    anchors.leftMargin: 20
                    anchors.verticalCenter: parent.verticalCenter
                    text: txt("pdfRaporOnizlemeBaslik")
                    color: "#dce8f5"
                    font.family: "Segoe UI"
                    font.pixelSize: 14
                    font.bold: true
                }
            }

            Rectangle {
                width: parent.width
                height: pdfOnizlemePopup.height - 52 - 64
                color: "#f3f4f6"

                Flickable {
                    anchors.fill: parent
                    anchors.margins: 20
                    contentWidth: width
                    contentHeight: onizlemeMetni.implicitHeight
                    clip: true
                    boundsBehavior: Flickable.StopAtBounds

                    TextEdit {
                        id: onizlemeMetni
                        width: parent.width
                        readOnly: true
                        selectByMouse: true
                        textFormat: TextEdit.RichText
                        wrapMode: Text.WordWrap
                        text: pdfOnizlemePopup.onizlemeHtml
                        color: "#111827"
                        font.pixelSize: 13
                    }
                }
            }

            Rectangle {
                width: parent.width
                height: 64
                color: "#0a0e17"

                Row {
                    anchors.centerIn: parent
                    spacing: 12

                    Button {
```

```
                        width: 140
                        height: 38
                        text: txt("indirPdf")
                        font.pixelSize: 13
                        font.bold: true

                        onClicked: {
                            var yol = reportManager.pdfOlustur(
                                olcumId,
                                musteriAdi,
                                receteAdi,
                                hesaplananTau0,
                                hesaplananMu,
                                parseFloat(r2Metni.text)
                            )
                            pdfOnizlemePopup.close()
                            if (yol.length > 0) {
                                bildirimKutusu.goster(txt("pdfKaydedildi") + yol, false)
                            } else {
                                bildirimKutusu.goster(txt("pdfOlusturulamadi"), true)
                            }
                        }

                        background: Rectangle {
                            radius: 8
                            color: parent.hovered ? "#4f8cf7" : "#3b82f6"
                            Behavior on color { ColorAnimation { duration: 120 } }
                        }

                        contentItem: Text {
                            text: parent.text
                            color: "#ffffff"
                            font: parent.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                        }
                    }

                    Button {
                        width: 100
                        height: 38
                        text: txt("kapat")
                        font.pixelSize: 13

                        onClicked: pdfOnizlemePopup.close()

                        background: Rectangle {
                            radius: 8
                            color: parent.hovered ? "#1e2a3f" : "transparent"
                            border.color: "#1e2a3f"
                            border.width: 1
                            Behavior on color { ColorAnimation { duration: 120 } }
                        }

                        contentItem: Text {
                            text: parent.text
                            color: "#9ca3af"
                            font: parent.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                        }
                    }
                }
            }
        }
    }

    Popup {
        id: csvOnizlemePopup
        modal: true
        focus: true
        anchors.centerIn: parent
        width: 640
        height: Math.min(680, parent.height - 40)
        padding: 0
        closePolicy: Popup.CloseOnEscape

        property var satirlar: []

        background: Rectangle {
```

```
            color: "#0f1420"
            radius: 12
            border.color: "#1e2a3f"
            border.width: 1
        }

        Overlay.modal: Rectangle {
            color: "#a6000000"
        }

        contentItem: Column {
            width: csvOnizlemePopup.width
            spacing: 0

            Rectangle {
                width: parent.width
                height: 52
                color: "#0a0e17"

                Text {
                    anchors.left: parent.left
                    anchors.leftMargin: 20
                    anchors.verticalCenter: parent.verticalCenter
                    text: txt("excelCsvOnizlemeBaslik")
                    color: "#dce8f5"
                    font.family: "Segoe UI"
                    font.pixelSize: 14
                    font.bold: true
                }
            }

            Item {
                width: parent.width
                height: csvBilgiSutunu.implicitHeight + 32

                Column {
                    id: csvBilgiSutunu
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.top: parent.top
                    anchors.margins: 20
                    spacing: 4

                    Text { text: txt("olcumIdEtiket") + olcumId; color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 12 }
                    Text { text: txt("tarihEtiket") + olcumTarihi; color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 12 }
                    Text { text: txt("musteriEtiket") + musteriAdi; color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 12 }
                    Text { text: txt("receteEtiket") + receteAdi; color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 12 }
                    Text { text: txt("agirlikEtiket") + agirlikDegeri.toFixed(1); color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 12 }
                }
            }

            Rectangle {
                width: parent.width
                height: csvOnizlemePopup.height - 52 - 130 - 64
                color: "#0a0e17"
                border.color: "#1e2a3f"
                border.width: 1

                Row {
                    id: csvTabloBasligi
                    anchors.top: parent.top
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.margins: 12

                    Text { width: parent.width * 0.2; text: txt("strokeBaslik"); color: "#6b7280"; font.pixelSize: 11; font.bold: true }
                    Text { width: parent.width * 0.2; text: txt("basincBaslik"); color: "#6b7280"; font.pixelSize: 11; font.bold: true }
                    Text { width: parent.width * 0.2; text: txt("konumBaslik"); color: "#6b7280"; font.pixelSize: 11; font.bold: true }
                    Text { width: parent.width * 0.2; text: txt("debiBuyukBaslik"); color: "#6b7280"; font.pixelSize: 11; font.bold: true }
                    Text { width: parent.width * 0.2; text: txt("gecerliBuyukBaslik"); color: "#6b7280"; font.pixelSize: 11; font.bold: true }
                }

                ListView {
                    anchors.top: csvTabloBasligi.bottom
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.bottom: parent.bottom
```

```
                    anchors.margins: 12
                    anchors.topMargin: 8
                    clip: true

                    model: csvOnizlemePopup.satirlar

                    delegate: Rectangle {
                        width: ListView.view.width
                        height: 26
                        color: index % 2 === 0 ? "transparent" : "#0f1420"

                        Row {
                            anchors.fill: parent

                            Text { width: parent.width * 0.2; anchors.verticalCenter: parent.verticalCenter; text: modelData.stroke; color: "#dce8f5"; font.pixelSize: 12 }
                            Text { width: parent.width * 0.2; anchors.verticalCenter: parent.verticalCenter; text: modelData.basinc.toFixed(1); color: "#dce8f5"; font.pixelSize: 12 }
                            Text { width: parent.width * 0.2; anchors.verticalCenter: parent.verticalCenter; text: modelData.konum.toFixed(1); color: "#dce8f5"; font.pixelSize: 12 }
                            Text { width: parent.width * 0.2; anchors.verticalCenter: parent.verticalCenter; text: modelData.debi.toFixed(2); color: "#dce8f5"; font.pixelSize: 12 }
                            Text { width: parent.width * 0.2; anchors.verticalCenter: parent.verticalCenter; text: modelData.gecerli ? txt("evet") : txt("hayir"); color: modelData.gecerli ? "#4ade80" : "#f87171"; font.pixelSize: 12 }
                        }
                    }
                }
            }

            Rectangle {
                width: parent.width
                height: 64
                color: "#0a0e17"

                Row {
                    anchors.centerIn: parent
                    spacing: 12

                    Button {
                        width: 140
                        height: 38
                        text: txt("indirCsv")
                        font.pixelSize: 13
                        font.bold: true

                        onClicked: {
                            var yol = database.csvDisaAktar(olcumId)
                            csvOnizlemePopup.close()
                            if (yol.length > 0) {
                                bildirimKutusu.goster(txt("csvKaydedildi") + yol, false)
                            } else {
                                bildirimKutusu.goster(txt("csvOlusturulamadi"), true)
                            }
                        }

                        background: Rectangle {
                            radius: 8
                            color: parent.hovered ? "#22c55e" : "#16a34a"
                            Behavior on color { ColorAnimation { duration: 120 } }
                        }

                        contentItem: Text {
                            text: parent.text
                            color: "#ffffff"
                            font: parent.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                        }
                    }

                    Button {
                        width: 100
                        height: 38
                        text: txt("kapat")
                        font.pixelSize: 13

                        onClicked: csvOnizlemePopup.close()

                        background: Rectangle {
                            radius: 8
```

```
                                color: parent.hovered ? "#1e2a3f" : "transparent"
                                border.color: "#1e2a3f"
                                border.width: 1
                                Behavior on color { ColorAnimation { duration: 120 } }
                            }

                            contentItem: Text {
                                text: parent.text
                                color: "#9ca3af"
                                font: parent.font
                                horizontalAlignment: Text.AlignHCenter
                                verticalAlignment: Text.AlignVCenter
                            }
                        }
                    }
                }
            }
        }
}
```

src/Backend.cpp

```cpp
#include "Backend.h"
#include <QDebug>

Backend::Backend(QObject *parent)
    : QObject(parent)
{
}

QString Backend::durum() const
{
    return "Baglanti bekleniyor...";
}

bool Backend::girisYap(const QString &kullaniciAdi, const QString &sifre)
{
    if (kullaniciAdi == "admin" && sifre == "admin123") {
        qDebug() << "Giris Basarili:" << kullaniciAdi;
        return true;
    }

    qDebug() << "Giris Basarisiz:" << kullaniciAdi;
    return false;
}

bool Backend::sifreDogrula(const QString &sifre)
{
    return sifre == "admin123";
}
```

src/Backend.h

```
#pragma once
#include <QObject>

class Backend : public QObject
{
    Q_OBJECT
    Q_PROPERTY(QString durum READ durum CONSTANT)

public:
    explicit Backend(QObject *parent = nullptr);
    QString durum() const;
    Q_INVOKABLE bool girisYap(const QString &kullaniciAdi, const QString &sifre);
    Q_INVOKABLE bool sifreDogrula(const QString &sifre);
};
```

src/Calculator.cpp

```
#include "Calculator.h"
#include <cmath>

Calculator::Calculator(QObject *parent)
    : QObject(parent)
{
}

QString Calculator::durum() const
{
    return m_durum;
}

int Calculator::strokeSayisi() const
{
    return m_strokeSayisi;
}

bool Calculator::duraklatildi() const
{
    return m_duraklatildi;
}

bool Calculator::sonStrokeGecerli() const
{
    return m_sonStrokeGecerli;
}

void Calculator::konumGuncelle(double konum, double hiz)
{
    if (m_duraklatildi) {
        return;
    }

    m_sonHiz = hiz;

    QString yeniDurum;

    if (konum > 450) {
        yeniDurum = "YUKARIDA";
    } else if (konum > 20) {
        yeniDurum = "INIYOR";
    } else {
        yeniDurum = "TAMAMLANDI";
    }

    if (yeniDurum == "YUKARIDA") {
        m_yukaridaGorulduMu = true;
    }

    if (yeniDurum == "TAMAMLANDI" && m_durum != "TAMAMLANDI" && m_yukaridaGorulduMu) {
        // Orijinal SLIPER'daki "invalid stroke" mantığı: hızı negatif veya
        // sıfır olan stroke'lar geçersiz sayılır (boru gerçekte hareket
        // etmemiş veya sensör verisi tutarsızdır).
        m_sonStrokeGecerli = hiz > 0.0;
        emit sonStrokeGecerliChanged();

        m_strokeSayisi++;
        m_yukaridaGorulduMu = false;
        emit strokeSayisiChanged();
    }

    if (yeniDurum != m_durum) {
        m_durum = yeniDurum;
        emit durumChanged();
    }
}

void Calculator::duraklat()
{
    if (m_duraklatildi) {
        return;
    }

    m_duraklatildi = true;
    emit duraklatildiChanged();
}
```

```
void Calculator::devamEt()
{
    if (!m_duraklatildi) {
        return;
    }

    m_duraklatildi = false;
    emit duraklatildiChanged();
}

void Calculator::sifirla()
{
    m_strokeSayisi = 0;
    m_durum = "BEKLENIYOR";
    m_duraklatildi = false;
    m_yukaridaGorulduMu = false;
    m_sonStrokeGecerli = true;
    m_sonHiz = 0.0;
    emit strokeSayisiChanged();
    emit durumChanged();
    emit duraklatildiChanged();
    emit sonStrokeGecerliChanged();
}

QVariantMap Calculator::boruHattiTahminHesapla(double tau0Mbar, double muMbarHm3,
                                                double debiM3h, double boruCapiMm,
                                                double boruUzunluguM,
                                                double hataPayiYuzde) const
{
    QVariantMap sonuc;
    sonuc["gecerliGiris"] = false;
    sonuc["basincBar"] = 0.0;
    sonuc["altSinirBar"] = 0.0;
    sonuc["ustSinirBar"] = 0.0;

    if (boruCapiMm <= 0.0 || boruUzunluguM <= 0.0 || debiM3h < 0.0) {
        return sonuc;
    }

    const double D_ref = REF_BORU_CAPI_MM / 1000.0;   // m
    const double L_ref = REF_BORU_UZUNLUGU_M;         // m
    const double A_ref = M_PI * (D_ref / 2.0) * (D_ref / 2.0); // m^2

    // Referans borudaki (tau0, mu) katsayılarından "gerçek" Bingham
    // rheolojik parametrelerini (tau0': Pa, mu': Pa.s) geri hesapla.
    // Bkz. Calculator.h başındaki formül açıklaması.
    const double tau0Prime = (100.0 * tau0Mbar * D_ref) / (4.0 * L_ref);
    const double muPrime = (100.0 * muMbarHm3 * A_ref * 3600.0 * D_ref * D_ref) / (32.0 * L_ref);

    const double D_t = boruCapiMm / 1000.0; // m
    const double L_t = boruUzunluguM;       // m
    const double A_t = M_PI * (D_t / 2.0) * (D_t / 2.0); // m^2
    const double v_t = (debiM3h / 3600.0) / A_t;         // m/s

    const double basincPa = L_t * (4.0 * tau0Prime / D_t + 32.0 * muPrime * v_t / (D_t * D_t));
    const double basincBar = basincPa / 1.0e5;

    const double tolerans = qBound(0.0, hataPayiYuzde, 100.0) / 100.0;

    sonuc["gecerliGiris"] = true;
    sonuc["basincBar"] = basincBar;
    sonuc["altSinirBar"] = basincBar * (1.0 - tolerans);
    sonuc["ustSinirBar"] = basincBar * (1.0 + tolerans);
    return sonuc;
}
```

src/Calculator.h

```cpp
#pragma once
#include <QObject>
#include <QVariantMap>

class Calculator : public QObject
{
    Q_OBJECT
    Q_PROPERTY(QString durum READ durum NOTIFY durumChanged)
    Q_PROPERTY(int strokeSayisi READ strokeSayisi NOTIFY strokeSayisiChanged)
    Q_PROPERTY(bool duraklatildi READ duraklatildi NOTIFY duraklatildiChanged)
    Q_PROPERTY(bool sonStrokeGecerli READ sonStrokeGecerli NOTIFY sonStrokeGecerliChanged)

public:
    explicit Calculator(QObject *parent = nullptr);
    QString durum() const;
    int strokeSayisi() const;
    bool duraklatildi() const;
    bool sonStrokeGecerli() const;

    // konum: mesafe sensöründen gelen anlık konum (mm)
    // hiz: sensorManager.hiz üzerinden hesaplanan anlık hız (m/s)
    // Stroke tamamlandığında (YUKARIDA -> INIYOR -> TAMAMLANDI geçişinde),
    // orijinal SLIPER cihazındaki "invalid stroke" mantığına paralel olarak
    // hız negatif veya sıfırsa (yani boru gerçekte hareket etmediyse ya da
    // yön/konum sensörü tutarsız veri verdiyse) stroke geçersiz sayılır.
    Q_INVOKABLE void konumGuncelle(double konum, double hiz);
    Q_INVOKABLE void duraklat();
    Q_INVOKABLE void devamEt();
    Q_INVOKABLE void sifirla();

    // Bingham modelinden (tau0, mu) elde edilen değerleri, hedef pompa
    // hattının çapı ve uzunluğuna göre ölçekleyerek beklenen basınç kaybını
    // hesaplar. Referans değerler SLIPER'ın kendi ölçüm borusuna aittir
    // (boru çapı 126 mm, dolum yüksekliği/ölçüm uzunluğu 500 mm -
    // bkz. Datasheet_SLIPER.pdf / slipermanV13.pdf "Technical specifications").
    //
    // NOT: Orijinal Schleibinger SLIPER yazılımının tam hesaplama modeli
    // kamuya açık değildir. Burada kullanılan yaklaşım, Bingham akışkanlar
    // için bilinen genelleştirilmiş boru basınç kaybı formülüne dayanır:
    //   dP/dL = 4*tau0/D + 32*mu*v/D^2
    // (v: ortalama akış hızı = Q/A). Referans borudaki ölçümden elde edilen
    // tau0 ve mu, bu formülle referans geometriye göre "geri hesaplanır",
    // sonra hedef boru geometrisine uygulanır. Bu, sahadaki gerçek pompa
    // basıncıyla karşılaştırılarak doğrulanmalıdır - kesin mühendislik
    // referansı olarak kullanılmadan önce gerçek verilerle kalibre edilmeli.
    Q_INVOKABLE QVariantMap boruHattiTahminHesapla(double tau0Mbar, double muMbarHm3,
                                                    double debiM3h, double boruCapiMm,
                                                    double boruUzunluguM,
                                                    double hataPayiYuzde) const;

signals:
    void durumChanged();
    void strokeSayisiChanged();
    void duraklatildiChanged();
    void sonStrokeGecerliChanged();

private:
    QString m_durum = "BEKLENIYOR";
    int m_strokeSayisi = 0;
    bool m_duraklatildi = false;
    bool m_yukaridaGorulduMu = false;
    bool m_sonStrokeGecerli = true;
    double m_sonHiz = 0.0;

    // Referans geometri: SLIPER'ın kendi ölçüm borusu (WifiManager::BORU_CAPI_M ile aynı)
    static constexpr double REF_BORU_CAPI_MM = 126.0;
    static constexpr double REF_BORU_UZUNLUGU_M = 0.5;
};
```

src/Database.cpp

```
#include "Database.h"
#include <QSqlQuery>
#include <QSqlError>
#include <QDebug>
#include <QDateTime>
#include <QVector>
#include <QFile>
#include <QTextStream>
#include <QStandardPaths>
#include <QDir>

Database::Database(QObject *parent)
    : QObject(parent)
{
    const QString klasor = QStandardPaths::writableLocation(QStandardPaths::AppLocalDataLocation) + "/database";
    QDir().mkpath(klasor);

    m_veriTabaniDosyaYolu = klasor + "/sliper.db";

    m_db = QSqlDatabase::addDatabase("QSQLITE");
    m_db.setDatabaseName(m_veriTabaniDosyaYolu);

    if (!m_db.open()) {
        qWarning() << "Veritabani acilamadi:" << m_db.lastError().text();
    } else {
        qDebug() << "Veritabani basariyla acildi.";
        tablolariOlustur();
    }
}

void Database::tablolariOlustur()
{
    QSqlQuery sorgu;

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS Olcumler ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT, "
        "tarih TEXT, "
        "musteri TEXT, "
        "recete TEXT, "
        "agirlik REAL"
        ")"
    );

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS Strokelar ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT, "
        "olcumId INTEGER, "
        "basinc REAL, "
        "konum REAL, "
        "debi REAL, "
        "gecerli INTEGER, "
        "FOREIGN KEY(olcumId) REFERENCES Olcumler(id)"
        ")"
    );

    QSqlQuery kolonKontrol("PRAGMA table_info(Strokelar)");
    bool debiKolonuVar = false;
    while (kolonKontrol.next()) {
        if (kolonKontrol.value("name").toString() == "debi") {
            debiKolonuVar = true;
            break;
        }
    }

    if (!debiKolonuVar) {
        QSqlQuery migrasyon;
        migrasyon.exec("ALTER TABLE Strokelar ADD COLUMN debi REAL DEFAULT 0");
    }

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS Kalibrasyonlar ("
        "sensor TEXT PRIMARY KEY, "
        "deger1 REAL, "
        "deger2 REAL, "
        "tarih TEXT"
        ")"
```

```
    );

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS LoadCellNoktalari ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT, "
        "hedefKg REAL, "
        "hamDeger REAL"
        ")"
    );

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS MesafeNoktalari ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT, "
        "hedefMm REAL, "
        "hamDeger REAL"
        ")"
    );

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS EgimKalibrasyon ("
        "id INTEGER PRIMARY KEY CHECK (id = 1), "
        "biasX REAL, biasY REAL, biasZ REAL, "
        "gainX REAL, gainY REAL, gainZ REAL, "
        "tarih TEXT"
        ")"
    );

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS KalibrasyonTarihleri ("
        "sensor TEXT PRIMARY KEY, "
        "tarih TEXT"
        ")"
    );

    qDebug() << "Tablolar hazir.";
}

int Database::olcumBaslat(const QString &musteri, const QString &recete, double agirlik)
{
    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO Olcumler (tarih, musteri, recete, agirlik) "
        "VALUES (:tarih, :musteri, :recete, :agirlik)"
    );

    sorgu.bindValue(":tarih", QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm"));
    sorgu.bindValue(":musteri", musteri);
    sorgu.bindValue(":recete", recete);
    sorgu.bindValue(":agirlik", agirlik);

    if (!sorgu.exec()) {
        qWarning() << "Olcum baslatilamadi:" << sorgu.lastError().text();
        return -1;
    }

    int yeniId = sorgu.lastInsertId().toInt();
    qDebug() << "Yeni olcum baslatildi, id:" << yeniId;
    return yeniId;
}

void Database::strokeKaydet(int olcumId, double basinc, double konum, double debi, bool gecerli)
{
    if (olcumId <= 0) {
        qWarning() << "Gecersiz olcumId, stroke kaydedilemedi.";
        return;
    }

    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO Strokelar (olcumId, basinc, konum, debi, gecerli) "
        "VALUES (:olcumId, :basinc, :konum, :debi, :gecerli)"
    );

    sorgu.bindValue(":olcumId", olcumId);
    sorgu.bindValue(":basinc", basinc);
    sorgu.bindValue(":konum", konum);
    sorgu.bindValue(":debi", debi);
    sorgu.bindValue(":gecerli", gecerli ? 1 : 0);

    if (!sorgu.exec()) {
```

```
        qWarning() << "Stroke kaydedilemedi:" << sorgu.lastError().text();
    } else {
        qDebug() << "Stroke kaydedildi, olcumId:" << olcumId;
    }
}

bool Database::olcumSil(int olcumId)
{
    if (olcumId <= 0) {
        qWarning() << "Gecersiz olcumId, olcum silinemedi.";
        return false;
    }

    if (!m_db.transaction()) {
        qWarning() << "Olcum silme icin transaction baslatilamadi:" << m_db.lastError().text();
        return false;
    }

    QSqlQuery strokeSil;
    strokeSil.prepare("DELETE FROM Strokelar WHERE olcumId = :olcumId");
    strokeSil.bindValue(":olcumId", olcumId);
    if (!strokeSil.exec()) {
        qWarning() << "Strokelar silinemedi:" << strokeSil.lastError().text();
        m_db.rollback();
        return false;
    }

    QSqlQuery olcumSil;
    olcumSil.prepare("DELETE FROM Olcumler WHERE id = :id");
    olcumSil.bindValue(":id", olcumId);
    if (!olcumSil.exec()) {
        qWarning() << "Olcum silinemedi:" << olcumSil.lastError().text();
        m_db.rollback();
        return false;
    }

    if (!m_db.commit()) {
        qWarning() << "Olcum silme commit edilemedi:" << m_db.lastError().text();
        return false;
    }

    qDebug() << "Olcum silindi, id:" << olcumId;
    return true;
}

QVariantList Database::tumOlcumleriGetir()
{
    QVariantList sonuclar;

    QSqlQuery sorgu(
        "SELECT o.id, o.tarih, o.musteri, o.recete, o.agirlik, "
        "COUNT(s.id) AS strokeSayisi "
        "FROM Olcumler o "
        "LEFT JOIN Strokelar s ON s.olcumId = o.id "
        "GROUP BY o.id "
        "ORDER BY o.id DESC"
    );

    while (sorgu.next()) {
        QVariantMap satir;
        satir["id"] = sorgu.value("id");
        satir["tarih"] = sorgu.value("tarih");
        satir["musteri"] = sorgu.value("musteri");
        satir["recete"] = sorgu.value("recete");
        satir["agirlik"] = sorgu.value("agirlik");
        satir["strokeSayisi"] = sorgu.value("strokeSayisi");
        sonuclar.append(satir);
    }

    return sonuclar;
}

QVariantList Database::strokeVerileriGetir(int olcumId)
{
    QVariantList sonuclar;

    QSqlQuery sorgu;
    sorgu.prepare(
        "SELECT basinc, konum, debi, gecerli FROM Strokelar "
        "WHERE olcumId = :olcumId ORDER BY id ASC"
```

```
    );
    sorgu.bindValue(":olcumId", olcumId);

    if (!sorgu.exec()) {
        qWarning() << "Stroke verileri alinamadi:" << sorgu.lastError().text();
        return sonuclar;
    }

    int sira = 1;
    while (sorgu.next()) {
        QVariantMap satir;
        satir["stroke"] = sira++;
        satir["basinc"] = sorgu.value("basinc");
        satir["konum"] = sorgu.value("konum");
        satir["debi"] = sorgu.value("debi");
        satir["gecerli"] = sorgu.value("gecerli").toBool();
        sonuclar.append(satir);
    }

    return sonuclar;
}

QVariantMap Database::binghamHesapla(int olcumId)
{
    QVariantMap sonuc;

    QSqlQuery sorgu;
    sorgu.prepare(
        "SELECT basinc, debi FROM Strokelar "
        "WHERE olcumId = :olcumId AND gecerli = 1"
    );
    sorgu.bindValue(":olcumId", olcumId);

    QVector<double> pDegerleri;
    QVector<double> qDegerleri;

    if (sorgu.exec()) {
        while (sorgu.next()) {
            pDegerleri.append(sorgu.value("basinc").toDouble());
            qDegerleri.append(sorgu.value("debi").toDouble());
        }
    }

    const int n = pDegerleri.size();

    if (n < 2) {
        sonuc["tau0"] = 0.0;
        sonuc["mu"] = 0.0;
        sonuc["r2"] = 0.0;
        sonuc["yeterliVeri"] = false;
        return sonuc;
    }

    double qToplam = 0, pToplam = 0, qpToplam = 0, qKareToplam = 0;
    for (int i = 0; i < n; ++i) {
        qToplam += qDegerleri[i];
        pToplam += pDegerleri[i];
        qpToplam += qDegerleri[i] * pDegerleri[i];
        qKareToplam += qDegerleri[i] * qDegerleri[i];
    }

    const double qOrt = qToplam / n;
    const double pOrt = pToplam / n;

    const double payda = qKareToplam - n * qOrt * qOrt;
    const double mu = (payda != 0.0) ? (qpToplam - n * qOrt * pOrt) / payda : 0.0;
    const double tau0 = pOrt - mu * qOrt;

    double ssTot = 0, ssRes = 0;
    for (int i = 0; i < n; ++i) {
        const double tahmin = tau0 + mu * qDegerleri[i];
        ssRes += (pDegerleri[i] - tahmin) * (pDegerleri[i] - tahmin);
        ssTot += (pDegerleri[i] - pOrt) * (pDegerleri[i] - pOrt);
    }

    const double r2 = (ssTot != 0.0) ? (1.0 - ssRes / ssTot) : 0.0;

    sonuc["tau0"] = tau0;
    sonuc["mu"] = mu;
    sonuc["r2"] = r2;
```

```
    sonuc["yeterliVeri"] = true;
    return sonuc;
}

bool Database::kalibrasyonKaydet(const QString &sensor, double deger1, double deger2)
{
    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO Kalibrasyonlar (sensor, deger1, deger2, tarih) "
        "VALUES (:sensor, :deger1, :deger2, :tarih) "
        "ON CONFLICT(sensor) DO UPDATE SET "
        "deger1 = :deger1, deger2 = :deger2, tarih = :tarih"
    );

    sorgu.bindValue(":sensor", sensor);
    sorgu.bindValue(":deger1", deger1);
    sorgu.bindValue(":deger2", deger2);
    sorgu.bindValue(":tarih", QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm"));

    if (!sorgu.exec()) {
        qWarning() << "Kalibrasyon kaydedilemedi:" << sorgu.lastError().text();
        return false;
    }

    qDebug() << "Kalibrasyon kaydedildi:" << sensor;
    return true;
}

QVariantMap Database::kalibrasyonGetir(const QString &sensor)
{
    QVariantMap sonuc;

    QSqlQuery sorgu;
    sorgu.prepare("SELECT deger1, deger2, tarih FROM Kalibrasyonlar WHERE sensor = :sensor");
    sorgu.bindValue(":sensor", sensor);

    if (sorgu.exec() && sorgu.next()) {
        sonuc["deger1"] = sorgu.value("deger1");
        sonuc["deger2"] = sorgu.value("deger2");
        sonuc["tarih"] = sorgu.value("tarih");
        sonuc["mevcut"] = true;
    } else {
        sonuc["mevcut"] = false;
    }

    return sonuc;
}

void Database::kalibrasyonTarihiKaydet(const QString &sensor)
{
    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO KalibrasyonTarihleri (sensor, tarih) VALUES (:sensor, :tarih) "
        "ON CONFLICT(sensor) DO UPDATE SET tarih = :tarih"
    );

    sorgu.bindValue(":sensor", sensor);
    sorgu.bindValue(":tarih", QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm"));

    if (!sorgu.exec()) {
        qWarning() << "Kalibrasyon tarihi kaydedilemedi:" << sorgu.lastError().text();
    }
}

QString Database::kalibrasyonTarihiGetir(const QString &sensor)
{
    QSqlQuery sorgu;
    sorgu.prepare("SELECT tarih FROM KalibrasyonTarihleri WHERE sensor = :sensor");
    sorgu.bindValue(":sensor", sensor);

    if (sorgu.exec() && sorgu.next()) {
        return sorgu.value("tarih").toString();
    }

    return QString();
}

QVariantMap Database::olcumBilgisiGetir(int olcumId)
{
    QVariantMap sonuc;
```

```
    QSqlQuery sorgu;
    sorgu.prepare("SELECT tarih, musteri, recete, agirlik FROM Olcumler WHERE id = :id");
    sorgu.bindValue(":id", olcumId);

    if (sorgu.exec() && sorgu.next()) {
        sonuc["tarih"] = sorgu.value("tarih");
        sonuc["musteri"] = sorgu.value("musteri");
        sonuc["recete"] = sorgu.value("recete");
        sonuc["agirlik"] = sorgu.value("agirlik");
        sonuc["bulundu"] = true;
    } else {
        sonuc["bulundu"] = false;
    }

    return sonuc;
}

// Orijinal SLIPER'daki çok sayfalı Excel/XML dışa aktarıma (bkz. slipermanV13.pdf
// bölüm 5.6) yakın format: Office 2003 "SpreadsheetML" XML'i. Ek kütüphane
// gerektirmez, Excel bu formatı doğrudan açar. "Results" sayfasında temel
// veri + Bingham sonuçları, "Stroke_1".."Stroke_N" sayfalarında ham stroke
// verisi bulunur.
QString Database::xmlDisaAktar(int olcumId)
{
    QVariantMap bilgi = olcumBilgisiGetir(olcumId);
    if (!bilgi["bulundu"].toBool()) {
        qWarning() << "Olcum bulunamadi, xml olusturulamadi.";
        return QString();
    }

    QString klasor = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation) + "/SliperRaporlari";
    if (!QDir().mkpath(klasor)) {
        qWarning() << "Rapor klasoru olusturulamadi:" << klasor;
        return QString();
    }

    QString dosyaAdi = klasor + QString("/SLIPER_Export_%1_%2.xml")
        .arg(olcumId)
        .arg(QDateTime::currentDateTime().toString("ddMMyyyy_HHmmss"));

    QFile dosya(dosyaAdi);
    if (!dosya.open(QIODevice::WriteOnly | QIODevice::Text)) {
        qWarning() << "XML dosyasi acilamadi:" << dosyaAdi;
        return QString();
    }

    QVariantMap bingham = binghamHesapla(olcumId);
    QVariantList strokelar = strokeVerileriGetir(olcumId);

    auto kacisliMetin = [](const QString &s) {
        QString sonuc = s;
        sonuc.replace("&", "&amp;");
        sonuc.replace("<", "&lt;");
        sonuc.replace(">", "&gt;");
        return sonuc;
    };

    QTextStream akis(&dosya);
    akis.setEncoding(QStringConverter::Utf8);

    akis << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
    akis << "<?mso-application progid=\"Excel.Sheet\"?>\n";
    akis << "<Workbook xmlns=\"urn:schemas-microsoft-com:office:spreadsheet\" "
            "xmlns:o=\"urn:schemas-microsoft-com:office:office\" "
            "xmlns:x=\"urn:schemas-microsoft-com:office:excel\" "
            "xmlns:ss=\"urn:schemas-microsoft-com:office:spreadsheet\">\n";

    // --- Results sayfasi ---
    akis << "<Worksheet ss:Name=\"Results\">\n<Table>\n";
    akis << "<Row><Cell><Data ss:Type=\"String\">Olcum ID</Data></Cell><Cell><Data ss:Type=\"Number\">"
         << olcumId << "</Data></Cell></Row>\n";
    akis << "<Row><Cell><Data ss:Type=\"String\">Tarih</Data></Cell><Cell><Data ss:Type=\"String\">"
         << kacisliMetin(bilgi["tarih"].toString()) << "</Data></Cell></Row>\n";
    akis << "<Row><Cell><Data ss:Type=\"String\">Musteri</Data></Cell><Cell><Data ss:Type=\"String\">"
         << kacisliMetin(bilgi["musteri"].toString()) << "</Data></Cell></Row>\n";
    akis << "<Row><Cell><Data ss:Type=\"String\">Recete</Data></Cell><Cell><Data ss:Type=\"String\">"
         << kacisliMetin(bilgi["recete"].toString()) << "</Data></Cell></Row>\n";
    akis << "<Row><Cell><Data ss:Type=\"String\">Agirlik (kg)</Data></Cell><Cell><Data ss:Type=\"Number\">"
         << bilgi["agirlik"].toDouble() << "</Data></Cell></Row>\n";
```

```
    akis << "<Row><Cell><Data ss:Type=\"String\">Stroke Sayisi</Data></Cell><Cell><Data ss:Type=\"Number\">"
         << strokelar.size() << "</Data></Cell></Row>\n";
    akis << "<Row></Row>\n";
    akis << "<Row><Cell><Data ss:Type=\"String\">Akma Gerilmesi tau0 (mbar)</Data></Cell><Cell><Data ss:Type=\"Number\">"
         << bingham["tau0"].toDouble() << "</Data></Cell></Row>\n";
    akis << "<Row><Cell><Data ss:Type=\"String\">Plastik Viskozite mu (mbar.h/m3)</Data></Cell><Cell><Data ss:Type=\"Number\">"
         << bingham["mu"].toDouble() << "</Data></Cell></Row>\n";
    akis << "<Row><Cell><Data ss:Type=\"String\">Uyum Kalitesi R2</Data></Cell><Cell><Data ss:Type=\"Number\">"
         << bingham["r2"].toDouble() << "</Data></Cell></Row>\n";
    akis << "</Table>\n</Worksheet>\n";

    // --- Her stroke icin ayri sayfa ---
    for (int i = 0; i < strokelar.size(); ++i) {
        QVariantMap s = strokelar[i].toMap();
        akis << "<Worksheet ss:Name=\"Stroke_" << (i + 1) << "\">\n<Table>\n";
        akis << "<Row><Cell><Data ss:Type=\"String\">Basinc (mbar)</Data></Cell>"
                "<Cell><Data ss:Type=\"String\">Konum (mm)</Data></Cell>"
                "<Cell><Data ss:Type=\"String\">Debi (m3/h)</Data></Cell>"
                "<Cell><Data ss:Type=\"String\">Gecerli</Data></Cell></Row>\n";
        akis << "<Row><Cell><Data ss:Type=\"Number\">" << s["basinc"].toDouble() << "</Data></Cell>"
             << "<Cell><Data ss:Type=\"Number\">" << s["konum"].toDouble() << "</Data></Cell>"
             << "<Cell><Data ss:Type=\"Number\">" << s["debi"].toDouble() << "</Data></Cell>"
             << "<Cell><Data ss:Type=\"String\">" << (s["gecerli"].toBool() ? "Evet" : "Hayir") << "</Data></Cell></Row>\n";
        akis << "</Table>\n</Worksheet>\n";
    }

    akis << "</Workbook>\n";

    dosya.close();
    qDebug() << "XML disa aktarildi:" << dosyaAdi;
    return dosyaAdi;
}

// Orijinal SLIPER'daki (Expert Settings > Database Export, sifre korumali)
// veritabani disa aktarim ozelligine karsilik gelir. Aktif SQLite baglantisi
// dosya uzerinde kilit tutabileceginden, kopyalamadan once WAL/journal'in
// diske yazilmasi icin basit bir bekleme (checkpoint) yapilir.
QString Database::veriTabaniDisaAktar()
{
    if (m_veriTabaniDosyaYolu.isEmpty() || !QFile::exists(m_veriTabaniDosyaYolu)) {
        qWarning() << "Disa aktarilacak veritabani bulunamadi.";
        return QString();
    }

    QString klasor = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation) + "/SliperRaporlari";
    if (!QDir().mkpath(klasor)) {
        qWarning() << "Rapor klasoru olusturulamadi:" << klasor;
        return QString();
    }

    // Bekleyen yazmalarin diske gitmesini garantiye almak icin.
    QSqlQuery checkpoint(m_db);
    checkpoint.exec("PRAGMA wal_checkpoint(FULL)");

    QString hedefDosya = klasor + QString("/sliper_yedek_%1.db")
        .arg(QDateTime::currentDateTime().toString("ddMMyyyy_HHmmss"));

    if (!QFile::copy(m_veriTabaniDosyaYolu, hedefDosya)) {
        qWarning() << "Veritabani disa aktarilamadi.";
        return QString();
    }

    qDebug() << "Veritabani disa aktarildi:" << hedefDosya;
    return hedefDosya;
}

// Orijinal SLIPER'daki "Database Import" ozelligine karsilik gelir. Ice
// aktarmadan once mevcut veritabaninin bir yedegi otomatik alinir; sonra
// baglanti kapatilip yeni dosya yerine kopyalanir ve yeniden acilir.
bool Database::veriTabaniIcaAktar(const QString &kaynakDosyaYolu)
{
    if (!QFile::exists(kaynakDosyaYolu)) {
        qWarning() << "Ice aktarilacak dosya bulunamadi:" << kaynakDosyaYolu;
        return false;
    }

    // Guvenlik: mevcut veriyi kaybetmemek icin otomatik yedek al.
    veriTabaniDisaAktar();

    m_db.close();
```

```
    QFile hedef(m_veriTabaniDosyaYolu);
    if (hedef.exists() && !hedef.remove()) {
        qWarning() << "Eski veritabani silinemedi, ice aktarim iptal edildi.";
        m_db.open();
        return false;
    }

    if (!QFile::copy(kaynakDosyaYolu, m_veriTabaniDosyaYolu)) {
        qWarning() << "Veritabani ice aktarilamadi.";
        m_db.open();
        return false;
    }

    if (!m_db.open()) {
        qWarning() << "Ice aktarim sonrasi veritabani acilamadi:" << m_db.lastError().text();
        return false;
    }

    tablolariOlustur();
    qDebug() << "Veritabani ice aktarildi:" << kaynakDosyaYolu;
    return true;
}

// Orijinal SLIPER'daki "Database Delete" ozelligine karsilik gelir. Sadece
// olcum/stroke verilerini siler; sensor kalibrasyonlari (load cell, mesafe,
// egim) korunur, cunku bunlar donanimsal kalibrasyon olup ayri bir islemdir.
bool Database::tumVeriyiSil()
{
    if (!m_db.transaction()) {
        qWarning() << "Tum veriyi silme icin transaction baslatilamadi:" << m_db.lastError().text();
        return false;
    }

    QSqlQuery strokeSil;
    if (!strokeSil.exec("DELETE FROM Strokelar")) {
        qWarning() << "Strokelar silinemedi:" << strokeSil.lastError().text();
        m_db.rollback();
        return false;
    }

    QSqlQuery olcumSil;
    if (!olcumSil.exec("DELETE FROM Olcumler")) {
        qWarning() << "Olcumler silinemedi:" << olcumSil.lastError().text();
        m_db.rollback();
        return false;
    }

    if (!m_db.commit()) {
        qWarning() << "Tum veriyi silme commit edilemedi:" << m_db.lastError().text();
        return false;
    }

    qDebug() << "Tum olcum verileri silindi.";
    return true;
}

QString Database::csvDisaAktar(int olcumId)
{
    QVariantMap bilgi = olcumBilgisiGetir(olcumId);
    if (!bilgi["bulundu"].toBool()) {
        qWarning() << "Olcum bulunamadi, csv olusturulamadi.";
        return QString();
    }

    QString klasor = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation) + "/SliperRaporlari";
    if (!QDir().mkpath(klasor)) {
        qWarning() << "Rapor klasoru olusturulamadi:" << klasor;
        return QString();
    }

    QString dosyaAdi = klasor + QString("/SLIPER_Veri_%1_%2.csv")
        .arg(olcumId)
        .arg(QDateTime::currentDateTime().toString("ddMMyyyy_HHmmss"));

    QFile dosya(dosyaAdi);
    if (!dosya.open(QIODevice::WriteOnly | QIODevice::Text)) {
        qWarning() << "CSV dosyasi acilamadi:" << dosyaAdi;
        return QString();
    }
```

```

    QTextStream akis(&dosya);
    akis.setEncoding(QStringConverter::Utf8);

    akis << "Olcum ID;" << olcumId << "\n";
    akis << "Tarih;" << bilgi["tarih"].toString() << "\n";
    akis << "Musteri;" << bilgi["musteri"].toString() << "\n";
    akis << "Recete;" << bilgi["recete"].toString() << "\n";
    akis << "Agirlik (kg);" << bilgi["agirlik"].toDouble() << "\n";
    akis << "\n";
    akis << "Stroke;Basinc (mbar);Konum (mm);Debi (m3/h);Gecerli\n";

    QVariantList strokelar = strokeVerileriGetir(olcumId);
    for (const QVariant &v : strokelar) {
        QVariantMap s = v.toMap();
        akis << s["stroke"].toInt() << ";"
             << s["basinc"].toDouble() << ";"
             << s["konum"].toDouble() << ";"
             << s["debi"].toDouble() << ";"
             << (s["gecerli"].toBool() ? "Evet" : "Hayir") << "\n";
    }

    dosya.close();
    qDebug() << "CSV disa aktarildi:" << dosyaAdi;
    return dosyaAdi;
}

bool Database::loadCellNoktalariKaydet(const QVariantList &noktalar)
{
    if (!m_db.transaction()) {
        qWarning() << "Load cell noktalari icin transaction baslatilamadi:" << m_db.lastError().text();
        return false;
    }

    QSqlQuery temizle;
    if (!temizle.exec("DELETE FROM LoadCellNoktalari")) {
        qWarning() << "Load cell noktalari temizlenemedi:" << temizle.lastError().text();
        m_db.rollback();
        return false;
    }

    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO LoadCellNoktalari (hedefKg, hamDeger) "
        "VALUES (:hedefKg, :hamDeger)"
    );

    for (const QVariant &v : noktalar) {
        QVariantMap nokta = v.toMap();
        sorgu.bindValue(":hedefKg", nokta["hedefKg"].toDouble());
        sorgu.bindValue(":hamDeger", nokta["hamDeger"].toDouble());
        if (!sorgu.exec()) {
            qWarning() << "Load cell noktasi kaydedilemedi:" << sorgu.lastError().text();
            m_db.rollback();
            return false;
        }
    }

    if (!m_db.commit()) {
        qWarning() << "Load cell noktalari commit edilemedi:" << m_db.lastError().text();
        return false;
    }

    qDebug() << "Load cell" << noktalar.size() << "nokta kaydedildi.";
    kalibrasyonTarihiKaydet("loadcell");
    return true;
}

QVariantList Database::loadCellNoktalariGetir()
{
    QVariantList sonuclar;

    QSqlQuery sorgu("SELECT hedefKg, hamDeger FROM LoadCellNoktalari ORDER BY hamDeger ASC");

    while (sorgu.next()) {
        QVariantMap satir;
        satir["hedefKg"] = sorgu.value("hedefKg");
        satir["hamDeger"] = sorgu.value("hamDeger");
        sonuclar.append(satir);
    }
```

```
    return sonuclar;
}

bool Database::mesafeNoktalariKaydet(const QVariantList &noktalar)
{
    if (!m_db.transaction()) {
        qWarning() << "Mesafe noktalari icin transaction baslatilamadi:" << m_db.lastError().text();
        return false;
    }

    QSqlQuery temizle;
    if (!temizle.exec("DELETE FROM MesafeNoktalari")) {
        qWarning() << "Mesafe noktalari temizlenemedi:" << temizle.lastError().text();
        m_db.rollback();
        return false;
    }

    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO MesafeNoktalari (hedefMm, hamDeger) "
        "VALUES (:hedefMm, :hamDeger)"
    );

    for (const QVariant &v : noktalar) {
        QVariantMap nokta = v.toMap();
        sorgu.bindValue(":hedefMm", nokta["hedefMm"].toDouble());
        sorgu.bindValue(":hamDeger", nokta["hamDeger"].toDouble());
        if (!sorgu.exec()) {
            qWarning() << "Mesafe noktasi kaydedilemedi:" << sorgu.lastError().text();
            m_db.rollback();
            return false;
        }
    }

    if (!m_db.commit()) {
        qWarning() << "Mesafe noktalari commit edilemedi:" << m_db.lastError().text();
        return false;
    }

    qDebug() << "Mesafe" << noktalar.size() << "nokta kaydedildi.";
    kalibrasyonTarihiKaydet("mesafe_olcum");
    return true;
}

QVariantList Database::mesafeNoktalariGetir()
{
    QVariantList sonuclar;

    QSqlQuery sorgu("SELECT hedefMm, hamDeger FROM MesafeNoktalari ORDER BY hamDeger ASC");

    while (sorgu.next()) {
        QVariantMap satir;
        satir["hedefMm"] = sorgu.value("hedefMm");
        satir["hamDeger"] = sorgu.value("hamDeger");
        sonuclar.append(satir);
    }

    return sonuclar;
}

bool Database::egimKalibrasyonuKaydet(double biasX, double biasY, double biasZ,
                                       double gainX, double gainY, double gainZ)
{
    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT OR REPLACE INTO EgimKalibrasyon (id, biasX, biasY, biasZ, gainX, gainY, gainZ, tarih) "
        "VALUES (1, :biasX, :biasY, :biasZ, :gainX, :gainY, :gainZ, :tarih)"
    );

    sorgu.bindValue(":biasX", biasX);
    sorgu.bindValue(":biasY", biasY);
    sorgu.bindValue(":biasZ", biasZ);
    sorgu.bindValue(":gainX", gainX);
    sorgu.bindValue(":gainY", gainY);
    sorgu.bindValue(":gainZ", gainZ);
    sorgu.bindValue(":tarih", QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm"));

    if (!sorgu.exec()) {
        qWarning() << "Egim kalibrasyonu kaydedilemedi:" << sorgu.lastError().text();
```

```
        return false;
    }

    qDebug() << "Egim kalibrasyonu kaydedildi.";
    return true;
}

QVariantMap Database::egimKalibrasyonuGetir()
{
    QVariantMap sonuc;

    QSqlQuery sorgu("SELECT biasX, biasY, biasZ, gainX, gainY, gainZ, tarih FROM EgimKalibrasyon WHERE id = 1");

    if (sorgu.exec() && sorgu.next()) {
        sonuc["biasX"] = sorgu.value("biasX");
        sonuc["biasY"] = sorgu.value("biasY");
        sonuc["biasZ"] = sorgu.value("biasZ");
        sonuc["gainX"] = sorgu.value("gainX");
        sonuc["gainY"] = sorgu.value("gainY");
        sonuc["gainZ"] = sorgu.value("gainZ");
        sonuc["tarih"] = sorgu.value("tarih");
        sonuc["mevcut"] = true;
    } else {
        sonuc["mevcut"] = false;
    }

    return sonuc;
}
```

src/Database.h

```
#pragma once
#include <QObject>
#include <QSqlDatabase>
#include <QVariantList>
#include <QVariantMap>

class Database : public QObject
{
    Q_OBJECT

public:
    explicit Database(QObject *parent = nullptr);

    Q_INVOKABLE int olcumBaslat(const QString &musteri, const QString &recete, double agirlik);
    Q_INVOKABLE void strokeKaydet(int olcumId, double basinc, double konum, double debi, bool gecerli);
    Q_INVOKABLE bool olcumSil(int olcumId);
    Q_INVOKABLE QVariantList tumOlcumleriGetir();
    Q_INVOKABLE QVariantList strokeVerileriGetir(int olcumId);
    Q_INVOKABLE QVariantMap binghamHesapla(int olcumId);
    Q_INVOKABLE bool kalibrasyonKaydet(const QString &sensor, double deger1, double deger2);
    Q_INVOKABLE QVariantMap kalibrasyonGetir(const QString &sensor);
    Q_INVOKABLE QVariantMap olcumBilgisiGetir(int olcumId);
    Q_INVOKABLE QString csvDisaAktar(int olcumId);
    Q_INVOKABLE QString xmlDisaAktar(int olcumId);
    Q_INVOKABLE QString veriTabaniDisaAktar();
    Q_INVOKABLE bool veriTabaniIcaAktar(const QString &kaynakDosyaYolu);
    Q_INVOKABLE bool tumVeriyiSil();
    Q_INVOKABLE bool loadCellNoktalariKaydet(const QVariantList &noktalar);
    Q_INVOKABLE QVariantList loadCellNoktalariGetir();
    Q_INVOKABLE bool mesafeNoktalariKaydet(const QVariantList &noktalar);
    Q_INVOKABLE QVariantList mesafeNoktalariGetir();
    Q_INVOKABLE bool egimKalibrasyonuKaydet(double biasX, double biasY, double biasZ,
                                             double gainX, double gainY, double gainZ);
    Q_INVOKABLE QVariantMap egimKalibrasyonuGetir();
    Q_INVOKABLE void kalibrasyonTarihiKaydet(const QString &sensor);
    Q_INVOKABLE QString kalibrasyonTarihiGetir(const QString &sensor);

private:
    void tablolariOlustur();
    QSqlDatabase m_db;
    QString m_veriTabaniDosyaYolu;
};
```

src/ReportManager.cpp

```cpp
#include "ReportManager.h"
#include <QPrinter>
#include <QPainter>
#include <QTextDocument>
#include <QStandardPaths>
#include <QDir>
#include <QDateTime>
#include <QFile>
#include <QDebug>

ReportManager::ReportManager(QObject *parent)
    : QObject(parent)
{
}

// Önizleme ve gerçek PDF çıktısı bu tek fonksiyondan beslenir; ikisi arasında
// tasarım/veri tutarsızlığı oluşmasın diye HTML üretimi burada merkezileştirildi.
QString ReportManager::raporHtmlOlustur(int olcumId, const QString &musteri, const QString &recete,
                                          double tau0, double mu, double r2)
{
    const bool pompalanabilir = tau0 < 5 && mu < 10;
    const QString durumRengi = pompalanabilir ? "#16a34a" : "#dc2626";
    const QString durumMetni = pompalanabilir ? "POMPALANABİLİR" : "POMPALAMA SORUNU";

    return QString(
        "<html><body style='font-family:Segoe UI, Arial, sans-serif; color:#111827;'>"
        "<h1 style='color:#3b82f6; margin-bottom:4px;'>SLIPER Analiz Raporu</h1>"
        "<p style='color:#6b7280; font-size:11px; margin-top:0;'>Liya Laboratuvar Test Cihazları</p>"
        "<hr style='border:none; border-top:1px solid #d1d5db;'>"
        "<table style='width:100%; border-collapse:collapse; margin-top:10px;'>"
        "<tr><td style='padding:4px 0; width:140px;'><b>Ölçüm ID</b></td><td>%1</td></tr>"
        "<tr><td style='padding:4px 0;'><b>Müşteri</b></td><td>%2</td></tr>"
        "<tr><td style='padding:4px 0;'><b>Reçete</b></td><td>%3</td></tr>"
        "<tr><td style='padding:4px 0;'><b>Tarih</b></td><td>%4</td></tr>"
        "</table>"
        "<h2 style='color:#111827; margin-top:24px; margin-bottom:8px; font-size:16px;'>Bingham Model Sonuçları</h2>"
        "<table style='width:100%; border-collapse:collapse;' cellpadding='6'>"
        "<tr style='background:#f3f4f6;'>"
        "<th style='text-align:left; border:1px solid #d1d5db;'>Parametre</th>"
        "<th style='text-align:left; border:1px solid #d1d5db;'>Değer</th></tr>"
        "<tr><td style='border:1px solid #d1d5db;'>Akma Gerilmesi (τ₀)</td><td style='border:1px solid #d1d5db;'>%5 mbar</td></tr>"
        "<tr><td style='border:1px solid #d1d5db;'>Plastik Viskozite (μ)</td><td style='border:1px solid #d1d5db;'>%6 mbar·h/m³</td></tr>"
        "<tr><td style='border:1px solid #d1d5db;'>Uyum Kalitesi (R²)</td><td style='border:1px solid #d1d5db;'>%7</td></tr>"
        "</table>"
        "<p style='margin-top:20px;'><b>Durum:</b> "
        "<span style='color:%8; font-weight:bold;'>%9</span></p>"
        "<hr style='border:none; border-top:1px solid #d1d5db; margin-top:24px;'>"
        "<p style='color:#6b7280; font-size:10px;'>Liya Laboratuvar Test Cihazları - SLIPER Analiz Sistemi</p>"
        "</body></html>"
    ).arg(olcumId)
     .arg(musteri.toHtmlEscaped())
     .arg(recete.toHtmlEscaped())
     .arg(QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm"))
     .arg(QString::number(tau0, 'f', 2))
     .arg(QString::number(mu, 'f', 2))
     .arg(QString::number(r2, 'f', 2))
     .arg(durumRengi)
     .arg(durumMetni);
}

QString ReportManager::pdfOnizlemeHtml(int olcumId, const QString &musteri, const QString &recete,
                                         double tau0, double mu, double r2)
{
    return raporHtmlOlustur(olcumId, musteri, recete, tau0, mu, r2);
}

QString ReportManager::pdfOlustur(int olcumId, const QString &musteri, const QString &recete,
                                    double tau0, double mu, double r2)
{
    QString klasor = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation) + "/SliperRaporlari";
    if (!QDir().mkpath(klasor)) {
        qWarning() << "Rapor klasoru olusturulamadi:" << klasor;
        return QString();
    }

    QString dosyaAdi = klasor + QString("/SLIPER_Rapor_%1_%2.pdf")
        .arg(olcumId)
```

```
        .arg(QDateTime::currentDateTime().toString("ddMMyyyy_HHmmss"));

    QTextDocument belge;
    belge.setHtml(raporHtmlOlustur(olcumId, musteri, recete, tau0, mu, r2));

    QPrinter yazici(QPrinter::HighResolution);
    yazici.setOutputFormat(QPrinter::PdfFormat);
    yazici.setOutputFileName(dosyaAdi);

    belge.print(&yazici);

    if (!QFile::exists(dosyaAdi)) {
        qWarning() << "PDF dosyasi yazilamadi:" << dosyaAdi;
        return QString();
    }

    qDebug() << "PDF rapor olusturuldu:" << dosyaAdi;
    return dosyaAdi;
}
```

src/ReportManager.h

```
#pragma once
#include <QObject>

class ReportManager : public QObject
{
    Q_OBJECT

public:
    explicit ReportManager(QObject *parent = nullptr);
    Q_INVOKABLE QString pdfOnizlemeHtml(int olcumId, const QString &musteri, const QString &recete,
                                         double tau0, double mu, double r2);
    Q_INVOKABLE QString pdfOlustur(int olcumId, const QString &musteri, const QString &recete,
                                     double tau0, double mu, double r2);

private:
    QString raporHtmlOlustur(int olcumId, const QString &musteri, const QString &recete,
                              double tau0, double mu, double r2);
};
```

src/SensorManager.cpp

```
#include "SensorManager.h"

SensorManager::SensorManager(QObject *parent)
    : QObject(parent)
{
}

double SensorManager::basinc() const { return m_basinc; }
double SensorManager::konum() const { return m_konum; }
double SensorManager::hiz() const { return m_hiz; }
double SensorManager::debi() const { return m_debi; }
double SensorManager::egimX() const { return m_egimX; }
double SensorManager::egimY() const { return m_egimY; }
double SensorManager::hamAgirlik() const { return m_hamAgirlik; }
double SensorManager::hamMesafe() const { return m_hamMesafe; }
double SensorManager::hamAccelX() const { return m_hamAccelX; }
double SensorManager::hamAccelY() const { return m_hamAccelY; }
double SensorManager::hamAccelZ() const { return m_hamAccelZ; }
bool SensorManager::veriGecerli() const { return m_veriGecerli; }
double SensorManager::bataryaVoltaj() const { return m_bataryaVoltaj; }

void SensorManager::veriGuncelle(double basinc, double konum, double hiz, double debi, double egimX, double egimY)
{
    const bool veriGecerliDegisti = !m_veriGecerli;
    m_veriGecerli = true;

    if (m_basinc != basinc) { m_basinc = basinc; emit basincChanged(); }
    if (m_konum != konum) { m_konum = konum; emit konumChanged(); }
    if (m_hiz != hiz) { m_hiz = hiz; emit hizChanged(); }
    if (m_debi != debi) { m_debi = debi; emit debiChanged(); }
    if (m_egimX != egimX) { m_egimX = egimX; emit egimXChanged(); }
    if (m_egimY != egimY) { m_egimY = egimY; emit egimYChanged(); }
    if (veriGecerliDegisti) { emit veriGecerliChanged(); }

    emit veriGuncellendi();
}

void SensorManager::hamAgirlikGuncelle(double hamAgirlik)
{
    if (m_hamAgirlik != hamAgirlik) {
        m_hamAgirlik = hamAgirlik;
        emit hamAgirlikChanged();
    }
}

void SensorManager::hamMesafeGuncelle(double hamMesafe)
{
    if (m_hamMesafe != hamMesafe) {
        m_hamMesafe = hamMesafe;
        emit hamMesafeChanged();
    }
}

void SensorManager::hamAccelGuncelle(double x, double y, double z)
{
    if (m_hamAccelX != x || m_hamAccelY != y || m_hamAccelZ != z) {
        m_hamAccelX = x;
        m_hamAccelY = y;
        m_hamAccelZ = z;
        emit hamAccelDegisti();
    }
}

void SensorManager::bataryaVoltajGuncelle(double voltaj)
{
    if (m_bataryaVoltaj != voltaj) {
        m_bataryaVoltaj = voltaj;
        emit bataryaVoltajChanged();
    }
}

void SensorManager::veriyiGecersizYap()
{
    if (!m_veriGecerli) return;
    m_veriGecerli = false;
    emit veriGecerliChanged();
}
```

src/SensorManager.h

```
#pragma once
#include <QObject>

class SensorManager : public QObject
{
    Q_OBJECT
    Q_PROPERTY(double basinc READ basinc NOTIFY basincChanged)
    Q_PROPERTY(double konum READ konum NOTIFY konumChanged)
    Q_PROPERTY(double hiz READ hiz NOTIFY hizChanged)
    Q_PROPERTY(double debi READ debi NOTIFY debiChanged)
    Q_PROPERTY(double egimX READ egimX NOTIFY egimXChanged)
    Q_PROPERTY(double egimY READ egimY NOTIFY egimYChanged)
    Q_PROPERTY(double hamAgirlik READ hamAgirlik NOTIFY hamAgirlikChanged)
    Q_PROPERTY(double hamMesafe READ hamMesafe NOTIFY hamMesafeChanged)
    Q_PROPERTY(double hamAccelX READ hamAccelX NOTIFY hamAccelDegisti)
    Q_PROPERTY(double hamAccelY READ hamAccelY NOTIFY hamAccelDegisti)
    Q_PROPERTY(double hamAccelZ READ hamAccelZ NOTIFY hamAccelDegisti)
    Q_PROPERTY(bool veriGecerli READ veriGecerli NOTIFY veriGecerliChanged)
    Q_PROPERTY(double bataryaVoltaj READ bataryaVoltaj NOTIFY bataryaVoltajChanged)

public:
    explicit SensorManager(QObject *parent = nullptr);

    double basinc() const;
    double konum() const;
    double hiz() const;
    double debi() const;
    double egimX() const;
    double egimY() const;
    double hamAgirlik() const;
    double hamMesafe() const;
    double hamAccelX() const;
    double hamAccelY() const;
    double hamAccelZ() const;
    bool veriGecerli() const;
    double bataryaVoltaj() const;

    Q_INVOKABLE void veriGuncelle(double basinc, double konum, double hiz, double debi, double egimX, double egimY);
    Q_INVOKABLE void hamAgirlikGuncelle(double hamAgirlik);
    Q_INVOKABLE void hamMesafeGuncelle(double hamMesafe);
    Q_INVOKABLE void hamAccelGuncelle(double x, double y, double z);
    Q_INVOKABLE void bataryaVoltajGuncelle(double voltaj);
    Q_INVOKABLE void veriyiGecersizYap();

signals:
    void basincChanged();
    void konumChanged();
    void hizChanged();
    void debiChanged();
    void egimXChanged();
    void egimYChanged();
    void hamAgirlikChanged();
    void hamMesafeChanged();
    void hamAccelDegisti();
    void veriGecerliChanged();
    void veriGuncellendi();
    void bataryaVoltajChanged();

private:
    double m_basinc = 0.0;
    double m_konum = 0.0;
    double m_hiz = 0.0;
    double m_debi = 0.0;
    double m_egimX = 0.0;
    double m_egimY = 0.0;
    double m_hamAgirlik = 0.0;
    double m_hamMesafe = 0.0;
    double m_hamAccelX = 0.0;
    double m_hamAccelY = 0.0;
    double m_hamAccelZ = 0.0;
    bool m_veriGecerli = false;
    double m_bataryaVoltaj = 0.0;
};
```

src/VoiceCommandManager.cpp

```cpp
#include "VoiceCommandManager.h"
#include "VoiceRecognitionWorker.h"

#include <QCoreApplication>
#include <QDir>
#include <QMetaObject>

VoiceCommandManager::VoiceCommandManager(QObject *parent)
    : QObject(parent)
{
    const QString modelYolu = QDir(QCoreApplication::applicationDirPath()).filePath("vosk-model-small-tr-0.3");

    m_worker = new VoiceRecognitionWorker(modelYolu);
    m_worker->moveToThread(&m_workerThread);

    connect(&m_workerThread, &QThread::finished, m_worker, &QObject::deleteLater);

    connect(m_worker, &VoiceRecognitionWorker::modelHazir, this, &VoiceCommandManager::workerModelHazir);
    connect(m_worker, &VoiceRecognitionWorker::dinliyorDegisti, this, &VoiceCommandManager::workerDinliyorDegisti);
    connect(m_worker, &VoiceRecognitionWorker::hataOlustu, this, &VoiceCommandManager::workerHataOlustu);
    connect(m_worker, &VoiceRecognitionWorker::komutAlgilandi, this, &VoiceCommandManager::workerKomutAlgilandi);
    connect(m_worker, &VoiceRecognitionWorker::anlikMetinDegisti, this, &VoiceCommandManager::workerAnlikMetinDegisti);

    m_workerThread.start();
    QMetaObject::invokeMethod(m_worker, "modeliYukle", Qt::QueuedConnection);
}

VoiceCommandManager::~VoiceCommandManager()
{
    m_workerThread.quit();
    m_workerThread.wait(3000);
}

bool VoiceCommandManager::etkin() const { return m_etkin; }

void VoiceCommandManager::setEtkin(bool etkin)
{
    if (m_etkin == etkin)
        return;
    m_etkin = etkin;
    emit etkinChanged();

    if (m_etkin)
        QMetaObject::invokeMethod(m_worker, "dinlemeyeBasla", Qt::QueuedConnection);
    else
        QMetaObject::invokeMethod(m_worker, "dinlemeyiDurdur", Qt::QueuedConnection);
}

bool VoiceCommandManager::dinliyor() const { return m_dinliyor; }
bool VoiceCommandManager::modelHazir() const { return m_modelHazir; }
QString VoiceCommandManager::sonKomutMetni() const { return m_sonKomutMetni; }
QString VoiceCommandManager::anlikMetin() const { return m_anlikMetin; }
QString VoiceCommandManager::sonHata() const { return m_sonHata; }

void VoiceCommandManager::workerModelHazir(bool hazir)
{
    m_modelHazir = hazir;
    emit modelHazirChanged();

    if (hazir && m_etkin)
        QMetaObject::invokeMethod(m_worker, "dinlemeyeBasla", Qt::QueuedConnection);
}

void VoiceCommandManager::workerDinliyorDegisti(bool dinliyor)
{
    m_dinliyor = dinliyor;
    emit dinliyorChanged();
}

void VoiceCommandManager::workerHataOlustu(const QString &mesaj)
{
    m_sonHata = mesaj;
    emit sonHataChanged();
}

void VoiceCommandManager::workerKomutAlgilandi(int komutTuru, const QString &metin)
{
```

```
    m_sonKomutMetni = metin;
    emit sonKomutMetniChanged();

    switch (static_cast<VoiceRecognitionWorker::KomutTuru>(komutTuru)) {
    case VoiceRecognitionWorker::KomutBaslat: emit baslatKomutu(); break;
    case VoiceRecognitionWorker::KomutDurdur: emit durdurKomutu(); break;
    case VoiceRecognitionWorker::KomutBitir:  emit bitirKomutu();  break;
    case VoiceRecognitionWorker::KomutDevam:  emit devamKomutu();  break;
    case VoiceRecognitionWorker::KomutEkle:   emit ekleKomutu();   break;
    }
}

void VoiceCommandManager::workerAnlikMetinDegisti(const QString &metin)
{
    m_anlikMetin = metin;
    emit anlikMetinChanged();
}
```

src/VoiceCommandManager.h

```cpp
#pragma once
#include <QObject>
#include <QString>
#include <QThread>

class VoiceRecognitionWorker;

// "Başlat", "Durdur", "Bitir" gibi sesli komutları dinlemek için kullanılır.
// Laboratuvar ortamında eller kirli/meşgulken dokunmadan ölçüm kontrolü
// yapılabilmesi amacıyla eklendi.
//
// Tanıma tamamen cihaz üzerinde (offline) Vosk kütüphanesi ile yapılır,
// internet bağlantısı gerekmez. Yanlışlıkla tetiklenmeyi önlemek için
// tanıyıcı sadece bu komut kelimelerini içeren kısıtlı bir sözlükle
// (grammar) çalışır; çevredeki gündelik konuşma bu dar kelime setine
// düşmediği için genelde göz ardı edilir (bkz. VoiceRecognitionWorker).
class VoiceCommandManager : public QObject
{
    Q_OBJECT
    Q_PROPERTY(bool etkin READ etkin WRITE setEtkin NOTIFY etkinChanged)
    Q_PROPERTY(bool dinliyor READ dinliyor NOTIFY dinliyorChanged)
    Q_PROPERTY(bool modelHazir READ modelHazir NOTIFY modelHazirChanged)
    Q_PROPERTY(QString sonKomutMetni READ sonKomutMetni NOTIFY sonKomutMetniChanged)
    Q_PROPERTY(QString anlikMetin READ anlikMetin NOTIFY anlikMetinChanged)
    Q_PROPERTY(QString sonHata READ sonHata NOTIFY sonHataChanged)

public:
    explicit VoiceCommandManager(QObject *parent = nullptr);
    ~VoiceCommandManager() override;

    bool etkin() const;
    void setEtkin(bool etkin);
    bool dinliyor() const;
    bool modelHazir() const;
    QString sonKomutMetni() const;
    QString anlikMetin() const;
    QString sonHata() const;

signals:
    void etkinChanged();
    void dinliyorChanged();
    void modelHazirChanged();
    void sonKomutMetniChanged();
    void anlikMetinChanged();
    void sonHataChanged();

    // Sayfalar bu sinyallere baglanip ilgili aksiyonu tetikler.
    void baslatKomutu();
    void durdurKomutu();
    void bitirKomutu();
    void devamKomutu();
    void ekleKomutu();

private slots:
    void workerModelHazir(bool hazir);
    void workerDinliyorDegisti(bool dinliyor);
    void workerHataOlustu(const QString &mesaj);
    void workerKomutAlgilandi(int komutTuru, const QString &metin);
    void workerAnlikMetinDegisti(const QString &metin);

private:
    QThread m_workerThread;
    VoiceRecognitionWorker *m_worker = nullptr;

    bool m_etkin = false;
    bool m_dinliyor = false;
    bool m_modelHazir = false;
    QString m_sonKomutMetni;
    QString m_anlikMetin;
    QString m_sonHata;
};
```

src/VoiceRecognitionWorker.cpp

```cpp
#include "VoiceRecognitionWorker.h"
#include "vosk_api.h"

#include <QAudioSource>
#include <QAudioFormat>
#include <QMediaDevices>
#include <QAudioDevice>
#include <QIODevice>
#include <QJsonDocument>
#include <QJsonObject>
#include <QJsonArray>
#include <QDebug>
#include <QLocale>
#include <QVector>
#include <algorithm>
#include <cstdlib>

namespace {

// Vosk'un tanıyabileceği komut kelimeleri. Tanıyıcı, dinlemeye başlarken
// bu listeyle "grammar" (kısıtlı sözlük) moduna alınır: motor SADECE bu
// kelimeleri ya da "[unk]" (bilinmeyen) çıktısını üretebilir. Bu sayede
// etraftaki gündelik konuşma bu dar kelime setine düşmediği icin genelde
// [unk] olarak elenir, ama gercek komut soylenince net yakalanir - ayrica
// arama uzayi kuculdugu icin tanima cok daha hizli/dogru calisir.
const QStringList &grammarKelimeleri()
{
    static const QStringList kelimeler = {
        "başlat", "başla",
        "durdur",
        "bitir", "bitti", "bitirdi", "bitiriyor", "bitirelim", "tamamla",
        "devam",
        "ekle"
    };
    return kelimeler;
}

QByteArray grammarJsonUret()
{
    QJsonArray dizi;
    for (const QString &kelime : grammarKelimeleri())
        dizi.append(kelime);
    dizi.append(QStringLiteral("[unk]"));
    return QJsonDocument(dizi).toJson(QJsonDocument::Compact);
}

// Turkce karakterleri ASCII'ye indirger, kucuk harfe cevirir; boylece
// Vosk ciktisi ile aday kelime listeleri arasinda esnek karsilastirma
// yapilabilir (aksan/karakter farkindan etkilenmez).
QString sadelestir(const QString &metin)
{
    QString kucuk = QLocale(QLocale::Turkish).toLower(metin);
    QString sonuc;
    sonuc.reserve(kucuk.size());
    for (const QChar &ch : kucuk) {
        switch (ch.unicode()) {
        case u'ş': sonuc += 's'; break;
        case u'ç': sonuc += 'c'; break;
        case u'ğ': sonuc += 'g'; break;
        case u'ü': sonuc += 'u'; break;
        case u'ö': sonuc += 'o'; break;
        case u'ı': sonuc += 'i'; break;
        case u'İ': sonuc += 'i'; break;
        default:
            if (ch.isLetterOrNumber() || ch.isSpace())
                sonuc += ch;
            break;
        }
    }
    return sonuc;
}

int levenshtein(const QString &a, const QString &b)
{
    const int n = a.size();
    const int m = b.size();
    QVector<int> onceki(m + 1), simdiki(m + 1);
```

```cpp
    for (int j = 0; j <= m; ++j) onceki[j] = j;

    for (int i = 1; i <= n; ++i) {
        simdiki[0] = i;
        for (int j = 1; j <= m; ++j) {
            int maliyet = (a[i - 1] == b[j - 1]) ? 0 : 1;
            simdiki[j] = std::min({ onceki[j] + 1, simdiki[j - 1] + 1, onceki[j - 1] + maliyet });
        }
        onceki.swap(simdiki);
    }
    return onceki[m];
}

bool kelimeUyuyorMu(const QString &token, const QStringList &adaylar, int tolerans)
{
    if (adaylar.contains(token))
        return true;
    for (const QString &aday : adaylar) {
        if (std::abs(token.size() - aday.size()) <= tolerans && levenshtein(token, aday) <= tolerans)
            return true;
    }
    return false;
}

} // namespace

VoiceRecognitionWorker::VoiceRecognitionWorker(const QString &modelYolu, QObject *parent)
    : QObject(parent)
    , m_modelYolu(modelYolu)
{
    m_sonKomutZamani.start();
}

VoiceRecognitionWorker::~VoiceRecognitionWorker()
{
    dinlemeyiDurdur();
    if (m_recognizer) vosk_recognizer_free(m_recognizer);
    if (m_model) vosk_model_free(m_model);
}

void VoiceRecognitionWorker::modeliYukle()
{
    vosk_set_log_level(-1);
    m_model = vosk_model_new(m_modelYolu.toUtf8().constData());
    if (!m_model) {
        emit hataOlustu(QStringLiteral("Ses tanima modeli yuklenemedi: ") + m_modelYolu);
        emit modelHazir(false);
        return;
    }
    emit modelHazir(true);
}

void VoiceRecognitionWorker::dinlemeyeBasla()
{
    if (!m_model) {
        emit hataOlustu(QStringLiteral("Ses tanima modeli hazir degil."));
        return;
    }
    if (m_audioSource)
        return; // zaten dinliyor

    QAudioFormat format;
    format.setSampleRate(16000);
    format.setChannelCount(1);
    format.setSampleFormat(QAudioFormat::Int16);

    const QAudioDevice girisAygiti = QMediaDevices::defaultAudioInput();
    if (girisAygiti.isNull()) {
        emit hataOlustu(QStringLiteral("Mikrofon bulunamadi."));
        return;
    }

    if (!m_recognizer) {
        const QByteArray grammar = grammarJsonUret();
        m_recognizer = vosk_recognizer_new_grm(m_model, 16000.0f, grammar.constData());
        if (!m_recognizer) {
            emit hataOlustu(QStringLiteral("Ses tanima motoru baslatilamadi."));
            return;
        }
        vosk_recognizer_set_words(m_recognizer, 0);
```

```
    } else {
        vosk_recognizer_reset(m_recognizer);
    }

    m_audioSource = new QAudioSource(girisAygiti, format, this);
    m_audioDevice = m_audioSource->start();
    if (!m_audioDevice) {
        emit hataOlustu(QStringLiteral("Mikrofon acilamadi."));
        delete m_audioSource;
        m_audioSource = nullptr;
        return;
    }

    connect(m_audioDevice, &QIODevice::readyRead, this, &VoiceRecognitionWorker::sesVerisiHazir);
    emit dinliyorDegisti(true);
}

void VoiceRecognitionWorker::dinlemeyiDurdur()
{
    if (!m_audioSource)
        return;

    m_audioSource->stop();
    m_audioSource->deleteLater();
    m_audioSource = nullptr;
    m_audioDevice = nullptr;

    if (m_recognizer)
        vosk_recognizer_reset(m_recognizer);

    emit dinliyorDegisti(false);
    emit anlikMetinDegisti(QString());
}

void VoiceRecognitionWorker::sesVerisiHazir()
{
    if (!m_audioDevice || !m_recognizer)
        return;

    const QByteArray veri = m_audioDevice->readAll();
    if (veri.isEmpty())
        return;

    const int bitti = vosk_recognizer_accept_waveform(m_recognizer, veri.constData(), veri.size());
    if (bitti == 1) {
        sonucuIsle(QString::fromUtf8(vosk_recognizer_result(m_recognizer)));
    } else if (bitti == 0) {
        const QString kismi = QString::fromUtf8(vosk_recognizer_partial_result(m_recognizer));
        QJsonObject obj = QJsonDocument::fromJson(kismi.toUtf8()).object();
        emit anlikMetinDegisti(obj.value(QStringLiteral("partial")).toString());
    }
}

void VoiceRecognitionWorker::sonucuIsle(const QString &json)
{
    QJsonObject obj = QJsonDocument::fromJson(json.toUtf8()).object();
    const QString metin = obj.value(QStringLiteral("text")).toString().trimmed();
    emit anlikMetinDegisti(QString());
    if (metin.isEmpty())
        return;

    metniDegerlendir(metin);
}

void VoiceRecognitionWorker::metniDegerlendir(const QString &metin)
{
    static const QStringList baslatAdaylari = { "baslat", "basla" };
    static const QStringList durdurAdaylari = { "durdur" };
    static const QStringList bitirAdaylari = { "bitir", "bitti", "bitirdi", "bitiriyor", "bitirelim", "tamamla" };
    static const QStringList devamAdaylari = { "devam" };
    static const QStringList ekleAdaylari = { "ekle" };

    const QString sade = sadelestir(metin);
    const QStringList tokenlar = sade.split(' ', Qt::SkipEmptyParts);

    for (const QString &token : tokenlar) {
        if (token == "unk")
            continue; // grammar disina cikan / eslesmeyen ses

        int komutTuru = -1;
```

```
        if (kelimeUyuyorMu(token, bitirAdaylari, 1)) komutTuru = VoiceRecognitionWorker::KomutBitir;
        else if (kelimeUyuyorMu(token, durdurAdaylari, 1)) komutTuru = VoiceRecognitionWorker::KomutDurdur;
        else if (kelimeUyuyorMu(token, devamAdaylari, 1)) komutTuru = VoiceRecognitionWorker::KomutDevam;
        else if (kelimeUyuyorMu(token, ekleAdaylari, 1)) komutTuru = VoiceRecognitionWorker::KomutEkle;
        else if (kelimeUyuyorMu(token, baslatAdaylari, 1)) komutTuru = VoiceRecognitionWorker::KomutBaslat;

        if (komutTuru >= 0) {
            if (m_sonKomutZamani.elapsed() < KOMUT_BEKLEME_MS)
                return; // cok yakin zamanda baska komut tetiklendi, tekrari engelle
            m_sonKomutZamani.restart();
            emit komutAlgilandi(komutTuru, metin);
            return;
        }
    }
}
```

src/VoiceRecognitionWorker.h

```cpp
#pragma once
#include <QObject>
#include <QString>
#include <QElapsedTimer>

class QAudioSource;
class QIODevice;
struct VoskModel;
struct VoskRecognizer;

// VoiceCommandManager'in ayri bir QThread icinde calistirdigi gercek isci.
// Mikrofon yakalama ve Vosk ile konusma tanima burada, arayuz thread'ini
// bloklamadan yapilir.
class VoiceRecognitionWorker : public QObject
{
    Q_OBJECT

public:
    enum KomutTuru {
        KomutBaslat = 0,
        KomutDurdur = 1,
        KomutBitir = 2,
        KomutDevam = 3,
        KomutEkle = 4
    };
    Q_ENUM(KomutTuru)

    explicit VoiceRecognitionWorker(const QString &modelYolu, QObject *parent = nullptr);
    ~VoiceRecognitionWorker() override;

public slots:
    void modeliYukle();
    void dinlemeyeBasla();
    void dinlemeyiDurdur();

signals:
    void modelHazir(bool basarili);
    void dinliyorDegisti(bool dinliyor);
    void hataOlustu(const QString &mesaj);
    void komutAlgilandi(int komutTuru, const QString &metin);
    void anlikMetinDegisti(const QString &metin);

private slots:
    void sesVerisiHazir();

private:
    void sonucuIsle(const QString &json);
    void metniDegerlendir(const QString &metin);

    QString m_modelYolu;
    VoskModel *m_model = nullptr;
    VoskRecognizer *m_recognizer = nullptr;
    QAudioSource *m_audioSource = nullptr;
    QIODevice *m_audioDevice = nullptr;

    QElapsedTimer m_sonKomutZamani;
    static constexpr qint64 KOMUT_BEKLEME_MS = 1200;
};
```

src/WifiManager.cpp

```cpp
#include "WifiManager.h"
#include <QJsonDocument>
#include <QJsonObject>
#include <QDebug>
#include <cmath>

WifiManager::WifiManager(SensorManager *sensorManager, Database *database, QObject *parent)
    : QObject(parent)
    , m_sensorManager(sensorManager)
    , m_database(database)
{
    m_soket = new QTcpSocket(this);

    connect(m_soket, &QTcpSocket::connected, this, &WifiManager::soketBaglandi);
    connect(m_soket, &QTcpSocket::disconnected, this, &WifiManager::soketAyrildi);
    connect(m_soket, &QTcpSocket::errorOccurred, this, &WifiManager::soketHata);
    connect(m_soket, &QTcpSocket::readyRead, this, &WifiManager::veriHazir);

    // Hareketsizlik kontrolu: baglanti acikken 2 dakikada bir kontrol edilir,
    // en son veri geldigi zamandan itibaren HAREKETSIZLIK_LIMIT_MS gecmisse
    // baglanti otomatik kesilir (orijinal cihazin "2 saat sonra otomatik
    // kapanma" davranisinin yazilimsal karsiligi).
    m_hareketsizlikTimer.setInterval(2 * 60 * 1000);
    connect(&m_hareketsizlikTimer, &QTimer::timeout, this, &WifiManager::hareketsizlikKontrolEt);
}

bool WifiManager::baglandi() const { return m_baglandi; }
bool WifiManager::baglaniyor() const { return m_baglaniyor; }
QString WifiManager::durumMesaji() const { return m_durumMesaji; }

void WifiManager::baglan()
{
    if (m_baglandi || m_baglaniyor) return;

    m_baglaniyor = true;
    emit baglaniyorChanged();
    durumGuncelle("SLIPER-ESP32'ye baglaniliyor...");

    m_soket->connectToHost(QString(ESP32_IP), ESP32_PORT);
}

void WifiManager::baglantiyiKes()
{
    m_soket->disconnectFromHost();
}

void WifiManager::kalibrasyonYenidenYukle()
{
    kalibrasyonYukle();
}

void WifiManager::kalibrasyonYukle()
{
    if (!m_database) return;

    m_loadCellHamDegerleri.clear();
    m_loadCellKiloDegerleri.clear();
    const QVariantList noktalar = m_database->loadCellNoktalariGetir();
    for (const QVariant &v : noktalar) {
        QVariantMap n = v.toMap();
        m_loadCellHamDegerleri.append(n["hamDeger"].toDouble());
        m_loadCellKiloDegerleri.append(n["hedefKg"].toDouble());
    }
    qDebug() << "Load cell" << m_loadCellHamDegerleri.size() << "nokta yuklendi.";

    m_mesafeHamDegerleri.clear();
    m_mesafeMmDegerleri.clear();
    const QVariantList mesafeNoktalari = m_database->mesafeNoktalariGetir();
    for (const QVariant &v : mesafeNoktalari) {
        QVariantMap n = v.toMap();
        m_mesafeHamDegerleri.append(n["hamDeger"].toDouble());
        m_mesafeMmDegerleri.append(n["hedefMm"].toDouble());
    }
    qDebug() << "Mesafe" << m_mesafeHamDegerleri.size() << "nokta yuklendi.";

    QVariantMap egimKal = m_database->egimKalibrasyonuGetir();
    if (egimKal.value("mevcut").toBool()) {
```

```
        m_egimBiasX = egimKal.value("biasX").toDouble();
        m_egimBiasY = egimKal.value("biasY").toDouble();
        m_egimBiasZ = egimKal.value("biasZ").toDouble();
        m_egimGainX = egimKal.value("gainX").toDouble();
        m_egimGainY = egimKal.value("gainY").toDouble();
        m_egimGainZ = egimKal.value("gainZ").toDouble();
        qDebug() << "Egim ivme kalibrasyonu yuklendi.";
    }
}

void WifiManager::soketBaglandi()
{
    m_baglandi = true;
    m_baglaniyor = false;
    emit baglandiChanged();
    emit baglaniyorChanged();
    durumGuncelle("SLIPER-ESP32 Bağlı");

    m_ilkPaket = true;
    m_zamanlayici.restart();
    m_hareketsizlikZamanlayici.restart();
    m_hareketsizlikTimer.start();
    kalibrasyonYukle();

    if (m_sensorManager) {
        m_sensorManager->veriyiGecersizYap();
    }

    qDebug() << "ESP32'ye baglanildi:" << ESP32_IP << ESP32_PORT;
}

void WifiManager::soketAyrildi()
{
    m_baglandi = false;
    m_baglaniyor = false;
    m_hareketsizlikTimer.stop();
    emit baglandiChanged();
    emit baglaniyorChanged();
    durumGuncelle("Bağlı Değil");

    if (m_sensorManager) {
        m_sensorManager->veriyiGecersizYap();
    }

    qDebug() << "ESP32 baglantisi kesildi.";
}

void WifiManager::hareketsizlikKontrolEt()
{
    if (!m_baglandi) return;

    if (m_hareketsizlikZamanlayici.elapsed() >= HAREKETSIZLIK_LIMIT_MS) {
        qWarning() << "2 saatten uzun suredir veri akisi yok, baglanti otomatik kesiliyor.";
        durumGuncelle("Hareketsizlik nedeniyle baglanti kesildi");
        emit hareketsizlikNedeniyleBaglantiKesildi();
        baglantiyiKes();
    }
}

void WifiManager::soketHata(QAbstractSocket::SocketError hata)
{
    m_baglandi = false;
    m_baglaniyor = false;
    emit baglandiChanged();
    emit baglaniyorChanged();
    durumGuncelle("Baglanti hatasi: " + m_soket->errorString());

    qWarning() << "Wifi soket hatasi:" << hata << m_soket->errorString();
}

void WifiManager::veriHazir()
{
    if (m_hareketsizlikZamanlayici.isValid()) {
        m_hareketsizlikZamanlayici.restart();
    }

    m_tamponVeri.append(m_soket->readAll());

    int satirSonu;
    while ((satirSonu = m_tamponVeri.indexOf('\n')) != -1) {
```

```cpp
        QByteArray satir = m_tamponVeri.left(satirSonu).trimmed();
        m_tamponVeri.remove(0, satirSonu + 1);

        if (!satir.isEmpty()) {
            jsonSatiriIsle(satir);
        }
    }
}

void WifiManager::egimHesapla(double accelX, double accelY, double accelZ, double &egimX, double &egimY) const
{
    egimX = std::atan2(accelY, accelZ) * 180.0 / M_PI;
    egimY = std::atan2(-accelX, std::sqrt(accelY * accelY + accelZ * accelZ)) * 180.0 / M_PI;
}

void WifiManager::jsonSatiriIsle(const QByteArray &satir)
{
    QJsonParseError hata;
    QJsonDocument belge = QJsonDocument::fromJson(satir, &hata);

    if (hata.error != QJsonParseError::NoError || !belge.isObject()) {
        qWarning() << "JSON parse hatasi:" << hata.errorString() << satir;
        return;
    }

    QJsonObject obj = belge.object();

    const double hamAgirlik = obj.value("hamAgirlik").toDouble();
    const double hamMesafe = obj.contains("hamMesafe")
        ? obj.value("hamMesafe").toDouble()
        : obj.value("konum").toDouble();
    const double accelX = obj.value("accelX").toDouble();
    const double accelY = obj.value("accelY").toDouble();
    const double accelZ = obj.value("accelZ").toDouble();

    // Kalibrasyon ekraninda gerekiyor: ham deger her zaman guncellensin
    if (m_sensorManager) {
        m_sensorManager->hamAgirlikGuncelle(hamAgirlik);
        m_sensorManager->hamMesafeGuncelle(hamMesafe);
        m_sensorManager->hamAccelGuncelle(accelX, accelY, accelZ);

        // Batarya voltaji: firmware "hamBatarya" alanini gondermiyorsa
        // (henuz donanim baglanmadiysa) 0 olarak kalir, QML tarafinda
        // gosterge "bilinmiyor" durumunda birakilmalidir.
        if (obj.contains("hamBatarya")) {
            const double hamBatarya = obj.value("hamBatarya").toDouble();
            const double voltaj = hamBatarya * ADS1115_LSB_VOLT * BATARYA_BOLUCU_ORANI;
            m_sensorManager->bataryaVoltajGuncelle(voltaj);
        }
    }

    const double konum = m_mesafeHamDegerleri.isEmpty()
        ? hamMesafe
        : mesafeInterpolasyon(hamMesafe);

    // --- Basinc: kayitli kalibrasyon ile ---
    const double agirlikKg = loadCellInterpolasyon(hamAgirlik);
    const double basinc = agirlikKg * 98.1;

    // --- Hiz ve debi: konum farkindan turetiliyor ---
    double hiz = 0.0;
    double debi = 0.0;

    const qint64 simdikiZamanMs = m_zamanlayici.elapsed();

    if (!m_ilkPaket) {
        double dt = (simdikiZamanMs - m_oncekiZamanMs) / 1000.0;
        if (dt <= 0.0) dt = 0.001;

        const double boruAlaniM2 = M_PI * (BORU_CAPI_M / 2.0) * (BORU_CAPI_M / 2.0);
        hiz = std::fabs(konum - m_oncekiKonum) / 1000.0 / dt;
        debi = hiz * boruAlaniM2 * 3600.0;
    } else {
        m_ilkPaket = false;
    }

    m_oncekiKonum = konum;
    m_oncekiZamanMs = simdikiZamanMs;

    // --- Egim: once ham ivme bias/gain ile duzeltilir, sonra aciya cevrilir ---
```

```
    const double duzeltilmisX = (accelX - m_egimBiasX) / m_egimGainX;
    const double duzeltilmisY = (accelY - m_egimBiasY) / m_egimGainY;
    const double duzeltilmisZ = (accelZ - m_egimBiasZ) / m_egimGainZ;

    double egimX = 0.0, egimY = 0.0;
    egimHesapla(duzeltilmisX, duzeltilmisY, duzeltilmisZ, egimX, egimY);

    if (m_sensorManager) {
        m_sensorManager->veriGuncelle(basinc, konum, hiz, debi, egimX, egimY);
    }
}

void WifiManager::durumGuncelle(const QString &mesaj)
{
    if (m_durumMesaji != mesaj) {
        m_durumMesaji = mesaj;
        emit durumMesajiChanged();
    }
}

double WifiManager::loadCellInterpolasyon(double hamDeger) const
{
    const int n = m_loadCellHamDegerleri.size();
    if (n == 0) return 0.0;
    if (n == 1) return m_loadCellKiloDegerleri[0];

    int i;
    if (hamDeger <= m_loadCellHamDegerleri[0]) {
        i = 0;
    } else if (hamDeger >= m_loadCellHamDegerleri[n - 1]) {
        i = n - 2;
    } else {
        for (i = 0; i < n - 1; ++i) {
            if (hamDeger <= m_loadCellHamDegerleri[i + 1]) break;
        }
    }

    const double ham1 = m_loadCellHamDegerleri[i];
    const double ham2 = m_loadCellHamDegerleri[i + 1];
    const double kg1 = m_loadCellKiloDegerleri[i];
    const double kg2 = m_loadCellKiloDegerleri[i + 1];

    if (ham2 == ham1) return kg1;
    return kg1 + (hamDeger - ham1) * (kg2 - kg1) / (ham2 - ham1);
}

double WifiManager::mesafeInterpolasyon(double hamDeger) const
{
    const int n = m_mesafeHamDegerleri.size();
    if (n == 0) return 0.0;
    if (n == 1) return m_mesafeMmDegerleri[0];

    int i;
    if (hamDeger <= m_mesafeHamDegerleri[0]) {
        i = 0;
    } else if (hamDeger >= m_mesafeHamDegerleri[n - 1]) {
        i = n - 2;
    } else {
        for (i = 0; i < n - 1; ++i) {
            if (hamDeger <= m_mesafeHamDegerleri[i + 1]) break;
        }
    }

    const double ham1 = m_mesafeHamDegerleri[i];
    const double ham2 = m_mesafeHamDegerleri[i + 1];
    const double mm1 = m_mesafeMmDegerleri[i];
    const double mm2 = m_mesafeMmDegerleri[i + 1];

    if (ham2 == ham1) return mm1;
    return mm1 + (hamDeger - ham1) * (mm2 - mm1) / (ham2 - ham1);
}
```

src/WifiManager.h

```
#pragma once
#include <QObject>
#include <QTcpSocket>
#include <QElapsedTimer>
#include <QTimer>
#include <QVector>
#include "SensorManager.h"
#include "Database.h"

class WifiManager : public QObject
{
    Q_OBJECT
    Q_PROPERTY(bool baglandi READ baglandi NOTIFY baglandiChanged)
    Q_PROPERTY(bool baglaniyor READ baglaniyor NOTIFY baglaniyorChanged)
    Q_PROPERTY(QString durumMesaji READ durumMesaji NOTIFY durumMesajiChanged)

public:
    explicit WifiManager(SensorManager *sensorManager, Database *database, QObject *parent = nullptr);

    bool baglandi() const;
    bool baglaniyor() const;
    QString durumMesaji() const;

    Q_INVOKABLE void baglan();
    Q_INVOKABLE void baglantiyiKes();
    Q_INVOKABLE void kalibrasyonYenidenYukle();

signals:
    void baglandiChanged();
    void baglaniyorChanged();
    void durumMesajiChanged();
    // Orijinal SLIPER'daki "2 saat hareketsizlikten sonra otomatik kapanma"
    // davranışının yazılımsal karşılığı: gerçek donanımsal güç kesme yerine,
    // uzun süre veri akışı olmayınca bağlantı otomatik kesilip kullanıcı
    // bilgilendirilir.
    void hareketsizlikNedeniyleBaglantiKesildi();

private slots:
    void soketBaglandi();
    void soketAyrildi();
    void soketHata(QAbstractSocket::SocketError hata);
    void veriHazir();
    void hareketsizlikKontrolEt();

private:
    void durumGuncelle(const QString &mesaj);
    void jsonSatiriIsle(const QByteArray &satir);
    void egimHesapla(double accelX, double accelY, double accelZ, double &egimX, double &egimY) const;
    void kalibrasyonYukle();
    double loadCellInterpolasyon(double hamDeger) const;
    double mesafeInterpolasyon(double hamDeger) const;

    QTcpSocket *m_soket = nullptr;
    SensorManager *m_sensorManager = nullptr;
    Database *m_database = nullptr;
    QByteArray m_tamponVeri;

    bool m_baglandi = false;
    bool m_baglaniyor = false;
    QString m_durumMesaji = "Bağlı Değil";

    bool m_ilkPaket = true;
    double m_oncekiKonum = 0.0;
    qint64 m_oncekiZamanMs = 0;
    QElapsedTimer m_zamanlayici;
    QElapsedTimer m_hareketsizlikZamanlayici;

    // --- Kayitli kalibrasyon degerleri (Database'den yuklenir) ---

    QVector<double> m_loadCellHamDegerleri;
    QVector<double> m_loadCellKiloDegerleri;

    QVector<double> m_mesafeHamDegerleri;
    QVector<double> m_mesafeMmDegerleri;

    double m_egimBiasX = 0.0, m_egimBiasY = 0.0, m_egimBiasZ = 0.0;
    double m_egimGainX = 1.0, m_egimGainY = 1.0, m_egimGainZ = 1.0;
```

```

    static constexpr const char *ESP32_IP = "192.168.4.1";
    static constexpr quint16 ESP32_PORT = 8888;

    static constexpr double BORU_CAPI_M = 0.126;

    // --- Batarya izleme ---
    // NOT: ESP32 firmware'inde henuz gercek bir voltaj bolucu devre
    // kalibrasyonu yapilmadi (bkz. sliper_esp32.ino icindeki TODO notu).
    // Asagidaki katsayi, ADS1115'in GAIN_ONE modunda ( +-4.096V, 16 bit )
    // okudugu ham degeri, varsayimsal 1:2 oranli bir bolucu ile pil
    // voltajina cevirir. Gercek donanim baglaninca BATARYA_BOLUCU_ORANI
    // olcum yapilarak duzeltilmelidir.
    static constexpr double ADS1115_LSB_VOLT = 0.000125; // 4.096V / 32768
    static constexpr double BATARYA_BOLUCU_ORANI = 2.0;
    static constexpr double BATARYA_UYARI_VOLTAJ = 14.0;
    static constexpr double BATARYA_IYI_VOLTAJ = 14.5;

    QTimer m_hareketsizlikTimer;
    static constexpr int HAREKETSIZLIK_LIMIT_MS = 2 * 60 * 60 * 1000; // 2 saat
};
```

third_party/vosk/vosk_api.h

```
// Copyright 2020-2021 Alpha Cephei Inc.
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//       http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/* This header contains the C API for Vosk speech recognition system */

#ifndef VOSK_API_H
#define VOSK_API_H

#ifdef __cplusplus
extern "C" {
#endif

/** Model stores all the data required for recognition
 *  it contains static data and can be shared across processing
 *  threads. */
typedef struct VoskModel VoskModel;


/** Speaker model is the same as model but contains the data
 *  for speaker identification. */
typedef struct VoskSpkModel VoskSpkModel;


/** Recognizer object is the main object which processes data.
 *  Each recognizer usually runs in own thread and takes audio as input.
 *  Once audio is processed recognizer returns JSON object as a string
 *  which represent decoded information - words, confidences, times, n-best lists,
 *  speaker information and so on */
typedef struct VoskRecognizer VoskRecognizer;


/**
 * Batch model object
 */
typedef struct VoskBatchModel VoskBatchModel;

/**
 * Batch recognizer object
 */
typedef struct VoskBatchRecognizer VoskBatchRecognizer;


/** Loads model data from the file and returns the model object
 *
 * @param model_path: the path of the model on the filesystem
 * @returns model object or NULL if problem occured */
VoskModel *vosk_model_new(const char *model_path);


/** Releases the model memory
 *
 *  The model object is reference-counted so if some recognizer
 *  depends on this model, model might still stay alive. When
 *  last recognizer is released, model will be released too. */
void vosk_model_free(VoskModel *model);


/** Check if a word can be recognized by the model
 * @param word: the word
 * @returns the word symbol if @param word exists inside the model
 * or -1 otherwise.
 * Reminding that word symbol 0 is for <epsilon> */
int vosk_model_find_word(VoskModel *model, const char *word);


/** Loads speaker model data from the file and returns the model object
```

```
 *
 * @param model_path: the path of the model on the filesystem
 * @returns model object or NULL if problem occured */
VoskSpkModel *vosk_spk_model_new(const char *model_path);


/** Releases the model memory
 *
 *  The model object is reference-counted so if some recognizer
 *  depends on this model, model might still stay alive. When
 *  last recognizer is released, model will be released too. */
void vosk_spk_model_free(VoskSpkModel *model);

/** Creates the recognizer object
 *
 *  The recognizers process the speech and return text using shared model data
 *  @param model       VoskModel containing static data for recognizer. Model can be
 *                     shared across recognizers, even running in different threads.
 *  @param sample_rate The sample rate of the audio you going to feed into the recognizer.
 *                     Make sure this rate matches the audio content, it is a common
 *                     issue causing accuracy problems.
 *  @returns recognizer object or NULL if problem occured */
VoskRecognizer *vosk_recognizer_new(VoskModel *model, float sample_rate);


/** Creates the recognizer object with speaker recognition
 *
 *  With the speaker recognition mode the recognizer not just recognize
 *  text but also return speaker vectors one can use for speaker identification
 *
 *  @param model       VoskModel containing static data for recognizer. Model can be
 *                     shared across recognizers, even running in different threads.
 *  @param sample_rate The sample rate of the audio you going to feed into the recognizer.
 *                     Make sure this rate matches the audio content, it is a common
 *                     issue causing accuracy problems.
 *  @param spk_model speaker model for speaker identification
 *  @returns recognizer object or NULL if problem occured */
VoskRecognizer *vosk_recognizer_new_spk(VoskModel *model, float sample_rate, VoskSpkModel *spk_model);


/** Creates the recognizer object with the phrase list
 *
 *  Sometimes when you want to improve recognition accuracy and when you don't need
 *  to recognize large vocabulary you can specify a list of phrases to recognize. This
 *  will improve recognizer speed and accuracy but might return [unk] if user said
 *  something different.
 *
 *  Only recognizers with lookahead models support this type of quick configuration.
 *  Precompiled HCLG graph models are not supported.
 *
 *  @param model       VoskModel containing static data for recognizer. Model can be
 *                     shared across recognizers, even running in different threads.
 *  @param sample_rate The sample rate of the audio you going to feed into the recognizer.
 *                     Make sure this rate matches the audio content, it is a common
 *                     issue causing accuracy problems.
 *  @param grammar The string with the list of phrases to recognize as JSON array of strings,
 *                 for example "["one two three four five", "[unk]"]".
 *
 *  @returns recognizer object or NULL if problem occured */
VoskRecognizer *vosk_recognizer_new_grm(VoskModel *model, float sample_rate, const char *grammar);


/** Adds speaker model to already initialized recognizer
 *
 * Can add speaker recognition model to already created recognizer. Helps to initialize
 * speaker recognition for grammar-based recognizer.
 *
 * @param spk_model Speaker recognition model */
void vosk_recognizer_set_spk_model(VoskRecognizer *recognizer, VoskSpkModel *spk_model);


/** Reconfigures recognizer to use grammar
 *
 * @param recognizer   Already running VoskRecognizer
 * @param grammar      Set of phrases in JSON array of strings or "[]" to use default model graph.
 *                     See also vosk_recognizer_new_grm
 */
void vosk_recognizer_set_grm(VoskRecognizer *recognizer, char const *grammar);
```

```c
/** Configures recognizer to output n-best results
 *
 * <pre>
 *   {
 *      "alternatives": [
 *          { "text": "one two three four five", "confidence": 0.97 },
 *          { "text": "one two three for five", "confidence": 0.03 },
 *      ]
 *   }
 * </pre>
 *
 * @param max_alternatives - maximum alternatives to return from recognition results
 */
void vosk_recognizer_set_max_alternatives(VoskRecognizer *recognizer, int max_alternatives);


/** Enables words with times in the output
 *
 * <pre>
 *   "result" : [{
 *       "conf" : 1.000000,
 *       "end" : 1.110000,
 *       "start" : 0.870000,
 *       "word" : "what"
 *     }, {
 *       "conf" : 1.000000,
 *       "end" : 1.530000,
 *       "start" : 1.110000,
 *       "word" : "zero"
 *     }, {
 *       "conf" : 1.000000,
 *       "end" : 1.950000,
 *       "start" : 1.530000,
 *       "word" : "zero"
 *     }, {
 *       "conf" : 1.000000,
 *       "end" : 2.340000,
 *       "start" : 1.950000,
 *       "word" : "zero"
 *     }, {
 *       "conf" : 1.000000,
 *       "end" : 2.610000,
 *       "start" : 2.340000,
 *       "word" : "one"
 *     }],
 * </pre>
 *
 * @param words - boolean value
 */
void vosk_recognizer_set_words(VoskRecognizer *recognizer, int words);

/** Like above return words and confidences in partial results
 *
 * @param partial_words - boolean value
 */
void vosk_recognizer_set_partial_words(VoskRecognizer *recognizer, int partial_words);

/** Set NLSML output
 * @param nlsml - boolean value
 */
void vosk_recognizer_set_nlsml(VoskRecognizer *recognizer, int nlsml);


/** Accept voice data
 *
 *  accept and process new chunk of voice data
 *
 *  @param data - audio data in PCM 16-bit mono format
 *  @param length - length of the audio data
 *  @returns 1 if silence is occured and you can retrieve a new utterance with result method
 *           0 if decoding continues
 *           -1 if exception occured */
int vosk_recognizer_accept_waveform(VoskRecognizer *recognizer, const char *data, int length);


/** Same as above but the version with the short data for language bindings where you have
 *  audio as array of shorts */
int vosk_recognizer_accept_waveform_s(VoskRecognizer *recognizer, const short *data, int length);
```

```c
/** Same as above but the version with the float data for language bindings where you have
 *  audio as array of floats */
int vosk_recognizer_accept_waveform_f(VoskRecognizer *recognizer, const float *data, int length);


/** Returns speech recognition result
 *
 * @returns the result in JSON format which contains decoded line, decoded
 *          words, times in seconds and confidences. You can parse this result
 *          with any json parser
 *
 * <pre>
 *  {
 *    "text" : "what zero zero zero one"
 *  }
 * </pre>
 *
 * If alternatives enabled it returns result with alternatives, see also vosk_recognizer_set_max_alternatives().
 *
 * If word times enabled returns word time, see also vosk_recognizer_set_word_times().
 */
const char *vosk_recognizer_result(VoskRecognizer *recognizer);


/** Returns partial speech recognition
 *
 * @returns partial speech recognition text which is not yet finalized.
 *          result may change as recognizer process more data.
 *
 * <pre>
 * {
 *    "partial" : "cyril one eight zero"
 * }
 * </pre>
 */
const char *vosk_recognizer_partial_result(VoskRecognizer *recognizer);


/** Returns speech recognition result. Same as result, but doesn't wait for silence
 *  You usually call it in the end of the stream to get final bits of audio. It
 *  flushes the feature pipeline, so all remaining audio chunks got processed.
 *
 *  @returns speech result in JSON format.
 */
const char *vosk_recognizer_final_result(VoskRecognizer *recognizer);


/** Resets the recognizer
 *
 *  Resets current results so the recognition can continue from scratch */
void vosk_recognizer_reset(VoskRecognizer *recognizer);


/** Releases recognizer object
 *
 *  Underlying model is also unreferenced and if needed released */
void vosk_recognizer_free(VoskRecognizer *recognizer);

/** Set log level for Kaldi messages
 *
 *  @param log_level the level
 *     0 - default value to print info and error messages but no debug
 *     less than 0 - don't print info messages
 *     greather than 0 - more verbose mode
 */
void vosk_set_log_level(int log_level);

/**
 *  Init, automatically select a CUDA device and allow multithreading.
 *  Must be called once from the main thread.
 *  Has no effect if HAVE_CUDA flag is not set.
 */
void vosk_gpu_init();

/**
 *  Init CUDA device in a multi-threaded environment.
 *  Must be called for each thread.
 *  Has no effect if HAVE_CUDA flag is not set.
 */
void vosk_gpu_thread_init();
```

```
/** Creates the batch recognizer object
 *
 *  @returns model object or NULL if problem occured */
VoskBatchModel *vosk_batch_model_new(const char *model_path);

/** Releases batch model object */
void vosk_batch_model_free(VoskBatchModel *model);

/** Wait for the processing */
void vosk_batch_model_wait(VoskBatchModel *model);

/** Creates batch recognizer object
 *  @returns recognizer object or NULL if problem occured */
VoskBatchRecognizer *vosk_batch_recognizer_new(VoskBatchModel *model, float sample_rate);

/** Releases batch recognizer object */
void vosk_batch_recognizer_free(VoskBatchRecognizer *recognizer);

/** Accept batch voice data */
void vosk_batch_recognizer_accept_waveform(VoskBatchRecognizer *recognizer, const char *data, int length);

/** Set NLSML output
 * @param nlsml - boolean value
 */
void vosk_batch_recognizer_set_nlsml(VoskBatchRecognizer *recognizer, int nlsml);

/** Closes the stream */
void vosk_batch_recognizer_finish_stream(VoskBatchRecognizer *recognizer);

/** Return results */
const char *vosk_batch_recognizer_front_result(VoskBatchRecognizer *recognizer);

/** Release and free first retrieved result */
void vosk_batch_recognizer_pop(VoskBatchRecognizer *recognizer);

/** Get amount of pending chunks for more intelligent waiting */
int vosk_batch_recognizer_get_pending_chunks(VoskBatchRecognizer *recognizer);

#ifdef __cplusplus
}
#endif

#endif /* VOSK_API_H */
```